OpenClaw Memory Sidecar 工作手册

OpenClaw 记忆侧车系统：运作、维修、代码与状态全说明

当前集成版

2026-03-18

目录

快速索引 **iv**

阅读说明 **vi**

0.1 收录范围 vi

0.2 当前结论 vi

第一章 系统全貌 **1**

1.1 系统由哪两部分组成 1

1.1.1 Windows 侧：记忆内核 1

1.1.2 WSL 侧：真实 OpenClaw 运行时 1

1.2 为什么 `D:\openclaw` 是系统的窝 1

1.3 真实运行链 1

1.4 本次真实验收结果 2

第二章 结构总表 **3**

2.1 仓库顶层结构 3

2.2 Windows 侧手册快照结构 3

2.3 WSL 侧手册快照结构 4

第三章 文件职责总览 **5**

3.1 Windows 侧文件职责 5

3.2 WSL 侧文件职责 5

第四章 逐文件解释 **7**

4.1 Windows 侧解释 7

4.1.1 `AGENTS.md` 7

4.1.2 `main.py` 7

4.1.3 `Start-OpenClaw-WSL-Interactive.cmd` 7

4.1.4 `core` 目录 7

4.1.5 `memory` 目录 7

4.2 WSL 侧解释 7

4.2.1 `bin/openclaw` 7

4.2.2 `package.json` 7

4.2.3 `src/cli/gateway-cli/run.ts` 8

4.2.4 src/cli/gateway-cli/run-loop.ts 8
4.2.5 src/infra/external-memory-kernel.ts 8
4.2.6 src/gateway/server-methods/chat.ts 8
4.2.7 src/gateway/server-methods/agent.ts 8
4.2.8 src/auto-reply/reply/agent-runner.ts 8

第五章 如何维修 9
5.1 最常见问题 9
5.1.1 发了消息，但 memory 不更新 9
5.1.2 Gateway 服务起不来 9
5.1.3 系统记不住“窝” 9
5.2 推荐维修流程 9

附录 A Windows 侧完整文件快照 10
A.1 AGENTS.md 10
A.2 main.py 14
A.3 Start-OpenClaw-WSL-Interactive.cmd 21
A.4 core/utils.py 21
A.5 core/memory_manager.py 23
A.6 core/router.py 29
A.7 core/retriever.py 30
A.8 core/strategy_manager.py 33
A.9 core/reflection_manager.py 35
A.10 core/cleaner.py 36
A.11 core/executor.py 38
A.12 core/selfcheck_manager.py 40
A.13 memory/working.json 47
A.14 memory/summary.json 49
A.15 memory/strategy.json 50
A.16 memory/reflection.json 51
A.17 memory/selfcheck.json 52
A.18 memory/runtime.json 62
A.19 memory/meta.json 65
A.20 memory/knowledge/programming.json 67
A.21 memory/knowledge/general.json 69
A.22 memory/knowledge/science.json 74
A.23 memory/knowledge/finance.json 75

附录 B WSL 侧完整文件快照 76
B.1 bin/openclaw 76
B.2 package.json 76
B.3 src/cli/gateway-cli/run.ts 88

B.4 src/cli/gateway-cli/run-loop.ts 99
B.5 src/infra/external-memory-kernel.ts 105
B.6 src/gateway/server-methods/chat.ts 107
B.7 src/gateway/server-methods/agent.ts 136
B.8 src/auto-reply/reply/agent-runner.ts 153

最后说明 **170**

快速索引

先看哪里

- 想快速理解整套系统：先看“系统全貌”。
- 想看目录和文件分布：看“结构总表”。
- 想知道每个关键文件干什么：看“文件职责总览”。
- 想逐个读懂当前系统：看“逐文件解释”。
- 想排障、维修、回退：看“如何维修”。
- 想直接查完整代码和当前状态：看“附录 A / 附录 B”。

最关键路径

主题	去哪里看
系统入口与家目录	`selfstart/Start-OpenClaw-WSL-Interactive.cmd`，以及“系统全貌”中的运行链说明
Windows 侧 OpenClaw Memory Sidecar	`main.py`、`core/*`、`memory/*`，以及“逐文件解释”
WSL 侧真实 OpenClaw 入口	`manual_sources/wsl/bin/openclaw`、`src/cli/gateway-cli/run.ts`
真实 chat 主路径	`src/gateway/server-methods/chat.ts`
真实 agent 主路径	`src/gateway/server-methods/agent.ts`、`src/auto-reply/reply/agent-runner.ts`
WSL 到 Windows 的 bridge	`src/infra/external-memory-kernel.ts`
当前运行状态	`memory/working.json`、`memory/runtime.json`、`memory/selfcheck.json`
长期摘要与知识	`memory/summary.json`、`memory/meta.json`、`memory/knowledge/*.json`
如何确认系统还活着	看“如何维修”中的检查顺序

维修顺序建议

1. 先确认 `D:\openclaw` 仍是 canonical home。

2. 再确认 WSL 启动链仍导出 `OPENCLAW_EXTERNAL_MEMORY_ROOT=/mnt/d/openclaw`。
3. 再确认真实请求会推进 `memory/working.json` 和 `memory/runtime.json`。
4. 最后才去看附录中的完整代码和配置细节。

阅读说明

这份手册的目标不是做一个简短摘要，而是把当前 OpenClaw 集成版的**结构、运行方式、维修入口、关键文件解释以及完整源码/内容快照**整理成一本能交接、能排障、能维护的工作手册。

0.1 收录范围

本手册收录两部分：

- Windows 侧 `D:\openclaw` 中与当前记忆内核直接相关的全部关键文件
- WSL 侧 `/home/park/openclaw` 中与真实 OpenClaw 运行时集成直接相关的全部关键文件

为了保持这份手册可读、可修、可打印，以下内容**不作为源码正文收录对象**：

- `node_modules`
- `dist` 编译产物全文
- 日志目录全文
- 缓存、`__pycache__`、临时文件、备份压缩包

这些被排除的内容不是系统核心源码，而是生成物或噪音。维修时应优先看源码、配置和状态文件，而不是先淹没在二进制与日志堆里。

0.2 当前结论

当前系统已经达到以下状态：

- 真实 Gateway 主路径已接入 OpenClaw Memory Sidecar
- 真实 `chat.send` 已验证会触发 Windows 侧记忆更新
- `D:\openclaw` 已被明确视为 canonical home/workspace
- 维修时可以根据本手册直接定位文件、路径和运行链

第 1 章　系统全貌

1.1　系统由哪两部分组成

1.1.1　Windows 侧：记忆内核

Windows 侧位于 `D:\openclaw`，它不是 Gateway 本体，而是一个轻量 memory kernel。它负责：

- working memory
- summary / reflection / selfcheck
- knowledge partition
- runtime snapshot
- 规则化清理与压缩

1.1.2　WSL 侧：真实 OpenClaw 运行时

WSL 侧位于 `/home/park/openclaw`，它负责：

- CLI 入口
- Gateway 服务
- chat / agent 方法分发
- auto-reply / agent turn 执行链
- 通过 sidecar bridge 调用 Windows 侧记忆内核

1.2　为什么 `D:\openclaw` 是系统的窝

这是当前系统的**约定根目录**。理由如下：

1. Windows 侧 memory kernel 全部在这里
2. 启动脚本会把它映射成 WSL 可见的 `/mnt/d/openclaw`
3. WSL 侧 sidecar bridge 默认也指向这个路径
4. memory 里的 `working.json` 和 `meta.json` 都已经把它记成 canonical home

1.3　真实运行链

当前真实运行链如下：

1. 用户启动 `Start-OpenClaw-WSL-Interactive.cmd`
2. 脚本进入 WSL，并导出 `OPENCLAW_EXTERNAL_MEMORY_ROOT=/mnt/d/openclaw`
3. WSL 调用 `/home/park/.local/bin/openclaw gateway start`

4. Gateway 服务运行
5. 用户发起 `chat.send` 或相关 agent 流程
6. OpenClaw 完成一次真实聊天或 agent turn
7. sidecar bridge 触发 `D:\openclaw\main.py`
8. Windows 侧 memory 执行一轮 compact cycle
9. `working.json`、`runtime.json` 等状态更新

1.4 本次真实验收结果

本手册生成前已经做过一次真实聊天主路径验收：

- 使用真实 `chat.send`
- Gateway 返回 `status = started`
- 随后检查 `working.json`
- `step_count` 从 42 增加到 43
- `runtime.json` 新增了 `step = 43` 的记录

这说明当前不是“只有单独运行 `main.py` 才有效”，而是**真实聊天路径已经带动了这套 memory**。

第 2 章　结构总表

2.1　仓库顶层结构

```
openclaw/
|- AGENTS.md
|- main.py
|- core/
|- memory/
|- selfstart/
|- agent-reach/
\- memory organization/
```

这一节只保留“看结构的方法”和“标准格式示例”，不再直接铺开整棵真实仓库树。真实全量代码与状态仍保留在附录和快照目录中。

2.2　Windows 侧手册快照结构

```
manual_sources/windows/
|- AGENTS.md
|- main.py
|- Start-OpenClaw-WSL-Interactive.cmd
|- core/
|  |- utils.py
|  |- memory_manager.py
|  |- router.py
|  |- retriever.py
|  |- strategy_manager.py
|  |- reflection_manager.py
|  |- cleaner.py
|  |- executor.py
|  \- selfcheck_manager.py
\- memory/
   |- working.json
   |- summary.json
```

```
   |- strategy.json
   |- reflection.json
   |- selfcheck.json
   |- runtime.json
   |- meta.json
   \- knowledge/
```

格式原则：

- 只展示层级、角色和代表性文件。
- 不在“结构总表”里铺开杂项缓存、备份、临时文件和历史噪音。
- 真正需要逐字核对代码时，再去看附录。

2.3 WSL 侧手册快照结构

```
manual_sources/wsl/
|- bin/
|  \- openclaw
|- package.json
\- src/
   |- cli/
   |  \- gateway-cli/
   |     |- run.ts
   |     \- run-loop.ts
   |- infra/
   |  \- external-memory-kernel.ts
   |- gateway/
   |  \- server-methods/
   |     |- chat.ts
   |     \- agent.ts
   \- auto-reply/
      \- reply/
         \- agent-runner.ts
```

这一页的目标是让维护者先知道“应该去哪一层找什么”，而不是先被真实仓库里的全部名字淹没。

第 3 章　文件职责总览

3.1　Windows 侧文件职责

文件	职责
AGENTS.md	当前 OpenClaw Memory Sidecar 的操作约束和规则文本
main.py	单轮 memory cycle 入口，串联各模块
selfstart/Start-OpenClaw-WSL-Interactive.cmd	启动链、显式导出 external memory root
core/utils.py	基础工具、归一化、JSON 与 token 辅助
core/memory_manager.py	memory 文件初始化、读写、知识写入、runtime 记录
core/router.py	query 到 partition 的确定性路由
core/retriever.py	轻量检索、缓存复用、top-k 控制
core/strategy_manager.py	策略选择、偏斜控制、使用统计
core/reflection_manager.py	周期性短反思
core/cleaner.py	去重、近重复合并、summary 压缩
core/executor.py	最小执行器
core/selfcheck_manager.py	数值评估、模式切换、建议与补丁守卫
memory/working.json	当前工作状态与步数
memory/summary.json	持久摘要
memory/strategy.json	可复用策略
memory/reflection.json	简短可执行 insight
memory/selfcheck.json	health、checks、suggestions、patch history
memory/runtime.json	运行快照
memory/meta.json	稳定规则与系统约束
memory/knowledge/programming.json	编程和系统知识
memory/knowledge/general.json	一般规则和偏好
memory/knowledge/science.json	科学分区
memory/knowledge/finance.json	金融分区

3.2　WSL 侧文件职责

文件	职责
`bin/openclaw`	WSL 实际 CLI 包装器
`package.json`	包元数据和 CLI 定义
`src/cli/gateway-cli/run.ts`	`gateway run` 启动命令逻辑
`src/cli/gateway-cli/run-loop.ts`	Gateway 生命周期主循环
`src/infra/external-memory-kernel.ts`	sidecar bridge，负责调用 Windows 侧 OpenClaw Memory Sidecar
`src/gateway/server-methods/chat.ts`	真实 `chat.send` 主路径，已接 sidecar
`src/gateway/server-methods/agent.ts`	真实 `agent` RPC 主路径，已接 sidecar
`src/auto-reply/reply/agent-runner.ts`	真实 reply-agent 主路径，已接 sidecar

第 4 章　逐文件解释

4.1　Windows 侧解释

4.1.1　`AGENTS.md`

这是当前 OpenClaw Memory Sidecar 的操作契约。它不是算法代码，但它定义了这套系统该如何理解任务、如何控制行为边界、如何避免无关扩张。

4.1.2　`main.py`

这是 OpenClaw Memory Sidecar 的总入口。任何一次 memory cycle 都从这里开始。它负责调用各个 core 模块，并决定本轮是否进行检索、反思、summary 压缩、自检、runtime 记录。

4.1.3　`Start-OpenClaw-WSL-Interactive.cmd`

这不是 Gateway 主实现,但它是用户最直观的启动入口。当前它有一个关键职责:把 `/mnt/d/openclaw` 作为 external memory root 显式导出给 WSL 运行时。

4.1.4　`core` 目录

`core` 目录是 Windows 侧记忆内核的逻辑层。这里的每个文件都尽量保持单一职责，避免把所有逻辑压进 `main.py`。

4.1.5　`memory` 目录

`memory` 目录是持久化状态层。它既不是原始日志仓库，也不是 transcript 仓库，而是压缩后的长期状态与知识。

4.2　WSL 侧解释

4.2.1　`bin/openclaw`

这是 WSL 里用户最终调用到的 `openclaw` 命令。它本身不是核心逻辑，只是把命令转发给 Node CLI 入口。

4.2.2　`package.json`

这里定义了 `openclaw` 作为 CLI 包如何暴露给系统，是 CLI 身份和构建脚本的根。

4.2.3 src/cli/gateway-cli/run.ts

这是前台运行 Gateway 的命令逻辑。它负责解析运行参数，并把启动动作交给真正的 server 启动函数。

4.2.4 src/cli/gateway-cli/run-loop.ts

这是 Gateway 服务的生命周期主循环。它负责服务启动、关停、重启、锁管理、信号处理、重启后的继续运行。

4.2.5 src/infra/external-memory-kernel.ts

这是本次集成的核心 bridge。它不直接实现 memory，而是作为 WSL 侧的 sidecar 调用器，去触发 `D:\openclaw` 中的 OpenClaw Memory Sidecar。

4.2.6 src/gateway/server-methods/chat.ts

这是最重要的真实用户主路径之一。用户发一句话，本质上会走到这里。现在这个文件在聊天成功或失败后会触发一次 sidecar memory run。

4.2.7 src/gateway/server-methods/agent.ts

当系统通过 RPC 形式直接触发 agent 流程时，会走这个文件。它现在也会在完成后触发 memory sidecar。

4.2.8 src/auto-reply/reply/agent-runner.ts

这是 reply-agent 的实际执行主路径。对于真实 agent turn，这是最关键的完成点之一。它现在在非 heartbeat 情况下会触发 memory sidecar。

第 5 章　如何维修

5.1　最常见问题

5.1.1　发了消息，但 memory 不更新

优先按下面顺序查：

1. `python main.py` 是否能独立运行
2. WSL 源码改完后是否重新 `pnpm build`
3. Gateway 服务是否真的重启到新构建
4. `chat.send` 实际是否走到当前接入路径
5. `D:\openclaw` 是否仍存在、是否仍包含 `main.py`

5.1.2　Gateway 服务起不来

1. 查 `journalctl --user -u openclaw-gateway.service -n 200 --no-pager`
2. 查 `/tmp/openclaw/openclaw-YYYY-MM-DD.log`
3. 判断是服务没起来、起来后秒退、还是 UI 构建卡住

5.1.3　系统记不住“窝”

查三处：

1. `Start-OpenClaw-WSL-Interactive.cmd` 是否仍导出 `OPENCLAW_EXTERNAL_MEMORY_ROOT=/mnt/d/openclaw`
2. `src/infra/external-memory-kernel.ts` 默认 root 是否仍是 `/mnt/d/openclaw`
3. `memory/meta.json` 与 `memory/working.json` 是否仍把 `D:\openclaw` 视为 canonical home

5.2　推荐维修流程

1. 先备份
2. 只改一个挂点或一个模块
3. 跑类型检查
4. 重新构建 WSL 侧 dist
5. 重启 gateway
6. 发一条真实消息
7. 检查 `working.json` 的 `step_count` 是否增长
8. 检查 `runtime.json` 是否新增记录

第 A 章　Windows 侧完整文件快照

A.1　AGENTS.md

作用说明

这份文件定义了当前 Windows 侧 OpenClaw Memory Sidecar 的行为契约，是整个侧车系统的规则起点。

完整内容

```
 AUTONOMOUS MEMORY AGENT SYSTEM (FOR CODEX / OPENCLAW)

You are an execution-focused AI agent with persistent memory, strict control logic, and self
    -improving behavior.

Your goal:

- COMPLETE tasks efficiently
- MINIMIZE unnecessary actions
- AVOID instability (exec, loops, memory overload)

---

 MEMORY STRUCTURE

You operate with 5 memory layers:

1. WORKING MEMORY (working.json)

- goal
- current_step
- next_action
- focus
- step_history
- step_count

2. SUMMARY MEMORY (summary.json)

```

```
- facts
- decisions
- failures
- progress

3. STRATEGY MEMORY (strategy.json)

- strategies with score + usage

4. KNOWLEDGE MEMORY (partitioned)

- programming
- science
- finance
- general

Each memory item:

- content
- tags
- importance (0~1)

5. REFLECTION MEMORY (reflection.json)

- insights (behavior improvements)

---

 CORE EXECUTION LOOP

Follow STRICT order:

1. LOAD working memory
2. SELECT best strategy (highest score)
3. CHECK step repetition:
   - if repeated  2 → force new approach
4. APPLY retrieval cooldown:
   - do NOT retrieve within 2 steps
5. ROUTE memory:
   - choose ONE partition only
6. RETRIEVE top_k=3 only
7. APPLY attention budget:
   - working  400 tokens
   - retrieved  600 tokens
8. BUILD context
9. INJECT:
   - strategy
```

```
   - failures
   - top insights
10. EXECUTE current step directly
11. UPDATE memory:

- step_history
- step_count
- last_action
- next_action

12. SCORE strategy:

- success → +0.1
- failure → -0.2

13. LOG failures if any
14. EVERY 3 steps:

- perform reflection

---

 HARD RULES (NON-NEGOTIABLE)

- NEVER use exec()
- NEVER use eval()
- NEVER generate dynamic execution code
- NEVER loop on same step
- NEVER retrieve full memory
- NEVER exceed attention budget
- ALWAYS produce usable output
- ALWAYS move forward

---

 MEMORY ROUTER

Select ONE:

- programming → code, bugs, dev
- science → math, physics, logic
- finance → money, trading, economy
- general → everything else

---

 MEMORY CLEANING
```

```

Trigger if:

- memory too large
- noise high

Actions:

- delete importance < 0.3
- merge duplicates
- boost frequently used memory

---

 SELF-REFLECTION

Every 3 steps, analyze:

- what worked
- what failed
- what to avoid
- what to improve

Store 1 insight:

- short
- actionable
- high value

---

 BEHAVIOR MODEL

You are NOT a chatbot.

You are:

- a task executor
- a memory optimizer
- a strategy learner

---

 PRIORITY ORDER

1. Complete task
2. Stay stable
```

```
3. Minimize memory usage
4. Improve future performance

---

 FAILURE PREVENTION

If unsure:

- do simplest valid action
- DO NOT fallback to exec
- DO NOT stall

---

 OUTPUT STYLE

- Direct
- Minimal
- Actionable
- No unnecessary explanation

---

END SYSTEM
```

A.2　main.py

作用说明

这是单轮 memory cycle 的总入口，也是从 sidecar bridge 最终会调用到的 OpenClaw Memory Sidecar 核心入口。

完整内容

```
from __future__ import annotations

import os
from pathlib import Path

from core.cleaner import clean_partition, light_compress_summary, needs_cleaning
from core.executor import build_context, detect_repetition, execute_step
from core.memory_manager import MemoryManager
from core.reflection_manager import generate_reflection_insight, should_reflect,
    store_reflection
from core.retriever import (
```

```
    mark_retrieval_used,
    rebuild_index,
    retrieval_allowed,
    retrieve_top_k,
    reuse_cached_results,
    touch_retrieved_items,
)
from core.router import select_partition
from core.selfcheck_manager import (
    evaluate_health,
    generate_optimization_suggestions,
    generate_patch_proposals,
    apply_safe_patch_proposals,
    run_self_check,
    select_mode,
    store_selfcheck,
    update_no_benefit_patch_state,
)
from core.strategy_manager import select_best_strategy, update_strategy_score
from core.utils import compact_step_label, now_iso


def _append_if_new(manager: MemoryManager, summary: dict, section: str, text: str,
    recent_window: int = 5) -> bool:
    cleaned = str(text).strip()
    if not cleaned:
        return False
    generic_entries = {
        "Review results and continue with the next minimal improvement",
        "Completed step: Review results and continue with the next minimal improvement",
    }
    if cleaned in generic_entries:
        return False
    recent = summary.get(section, [])[-recent_window:]
    if cleaned in recent:
        return False
    manager.append_summary(section, cleaned)
    summary.setdefault(section, []).append(cleaned)
    return True


def print_summary() -> None:
    print("File tree:")
    for path in [
        "AGENTS.md",
        "main.py",
        "core/memory_manager.py",
```

```
        "core/strategy_manager.py",
        "core/router.py",
        "core/retriever.py",
        "core/reflection_manager.py",
        "core/cleaner.py",
        "core/executor.py",
        "core/selfcheck_manager.py",
        "core/utils.py",
        "memory/working.json",
        "memory/summary.json",
        "memory/strategy.json",
        "memory/reflection.json",
        "memory/selfcheck.json",
        "memory/runtime.json",
        "memory/meta.json",
        "memory/knowledge/programming.json",
        "memory/knowledge/science.json",
        "memory/knowledge/finance.json",
        "memory/knowledge/general.json",
    ]:
        print(f"- {path}")

    print("\nModule summary:")
    print("- utils.py: JSON helpers, normalization, and compact labels.")
    print("- memory_manager.py: file initialization and persistence.")
    print("- router.py: deterministic partition selection.")
    print("- retriever.py: lexical retrieval capped at 3 items.")
    print("- strategy_manager.py: small strategy selection and scoring.")
    print("- reflection_manager.py: every-3-step reflection handling.")
    print("- cleaner.py: light deduplication and size control.")
    print("- executor.py: builds minimal context and completes one step.")
    print("- selfcheck_manager.py: lightweight numeric health check.")
    print("- main.py: runs one full execution cycle.")

    print("\nHow to run:")
    print("- python main.py")


def main() -> None:
    base_dir = Path(__file__).resolve().parent
    manager = MemoryManager(base_dir)
    manager.initialize()

    working = manager.load_working()
    summary = manager.load_summary()
    strategy_data = manager.load_strategy()
    reflection_data = manager.load_reflection()
```

```
    selfcheck_data = manager.load_selfcheck()
    runtime_data = manager.load_runtime()
    meta = manager.load_meta()

    query_text = " ".join(
        part for part in [working.get("goal", ""), working.get("focus", ""), working.get("current_step", "")] if part
    )
    partition = select_partition(query_text)
    partition_data = manager.load_partition(partition)
    rebuild_index(partition_data)

    repeated = detect_repetition(
        working.get("step_history", []),
        working.get("current_step", ""),
        working.get("next_action", ""),
    )
    strategy = select_best_strategy(
        strategy_data,
        current_step=working.get("current_step", ""),
        last_strategy=working.get("last_strategy"),
        force_new=repeated,
        step_count=int(working.get("step_count", 0)),
    )

    mode = str(working.get("mode", "normal"))
    retrieved_items = []
    retrieval_used = False
    cached_items = reuse_cached_results(partition_data, query_text, working, partition)
    if cached_items:
        retrieved_items = cached_items[:2] if mode == "convergence" else cached_items[:3]
        touch_retrieved_items(retrieved_items, int(working.get("step_count", 0)))
    elif retrieval_allowed(working) and not (mode == "stability" and int(working.get("step_count", 0)) % 2 == 1):
        top_k = 2 if mode in {"stability", "convergence"} else 3
        retrieved_items = retrieve_top_k(partition_data, query_text, k=top_k)
        touch_retrieved_items(retrieved_items, int(working.get("step_count", 0)))
        retrieval_used = True
    working["last_result_keys"] = [item.get("key", "") for item in retrieved_items[:3]]
    working["last_query"] = query_text

    context = build_context(working, strategy, summary, reflection_data, retrieved_items, meta)
    result = execute_step(context)

    old_current_step = working.get("current_step", "")
    next_step = result["decision"] or "Review results and continue with the next minimal
```

```
    improvement"
    working["step_history"] = (working.get("step_history", []) + [compact_step_label(old_current_step or next_step)])[-20:]
    working["step_count"] = int(working.get("step_count", 0)) + 1
    working["last_action"] = old_current_step
    working["current_step"] = next_step
    working["next_action"] = "Review results and continue with the next minimal improvement"
    working["last_partition"] = partition
    working["last_strategy"] = strategy.get("name", "direct_execution")
    if retrieval_used:
        mark_retrieval_used(working)

    _append_if_new(manager, summary, "progress", result["output"])
    _append_if_new(manager, summary, "decisions", result["decision"])
    allow_knowledge_write = mode != "convergence" and not (mode == "stability" and int(working["step_count"]) % 2 == 0)
    if result.get("new_fact") and allow_knowledge_write:
        _append_if_new(manager, summary, "facts", result["new_fact"])
        added_knowledge = manager.add_knowledge(
            partition,
            result["new_fact"],
            [partition, str(working.get("focus", "")), compact_step_label(next_step)],
            importance=0.6,
            current_step_count=int(working["step_count"]),
            working=working,
        )
        if added_knowledge:
            working = manager.load_working()
            partition_data = manager.load_partition(partition)
    if result.get("failure"):
        manager.append_summary("failures", result["failure"])

    update_strategy_score(strategy_data, working["last_strategy"], bool(result.get("success", False)))

    if should_reflect(int(working["step_count"])):
        insight = generate_reflection_insight(working, manager.load_summary())
        store_reflection(reflection_data, insight)

    latest_summary = manager.load_summary()
    maintenance_due = int(working["step_count"]) % 5 == 0
    should_clean = needs_cleaning(partition_data) or mode in {"stability", "convergence"}
    if should_clean:
        partition_data = clean_partition(partition_data, int(working["step_count"]))
    else:
        rebuild_index(partition_data)
    if maintenance_due:
```

```
        latest_summary = light_compress_summary(latest_summary, aggressive=mode == "convergence")

    suggestions: list[str] = []
    proposals: list[dict] = []
    applied_patches: list[dict] = []
    patch_applied = False
    health_report = {
        "health": float(working.get("last_health", 1.0)),
        "delta": 0.0,
        "pressure": 0.0,
        "noise": 0.0,
        "complexity": 0.0,
    }
    if maintenance_due:
        partition_map = {name: manager.load_partition(name) for name in ("programming", "science", "finance", "general")}
        check_report = run_self_check(working, latest_summary, strategy_data, reflection_data, partition_map)
        previous_eval = selfcheck_data.get("evaluations", [])[-1] if selfcheck_data.get("evaluations") else None
        health_report = evaluate_health(check_report, previous_eval)
        update_no_benefit_patch_state(working, selfcheck_data, health_report)
        working["mode"] = select_mode(health_report, check_report, mode)
        suggestions = generate_optimization_suggestions(check_report, health_report, working, max_suggestions=2)
        if suggestions:
            working["recent_suggestion_steps"] = (
                working.get("recent_suggestion_steps", []) + [int(working["step_count"])]
            )[-10:]
        skip_evolution = os.environ.get("OPENCLAW_SKIP_EVOLUTION") == "1" or working.get("mode") == "convergence"
        proposals = [] if skip_evolution else generate_patch_proposals(check_report, health_report, working, base_dir)
        patch_applied, applied_patches = (False, []) if skip_evolution else apply_safe_patch_proposals(base_dir, proposals, working)
        selfcheck_data = store_selfcheck(selfcheck_data, check_report, health_report, suggestions, proposals, applied_patches)
        working["last_health"] = health_report["health"]
    else:
        working["mode"] = mode


    manager.save_working(working)
    manager.save_summary(latest_summary)
    manager.save_strategy(strategy_data)
    manager.save_reflection(reflection_data)
```

```
    manager.save_selfcheck(selfcheck_data)
    total_knowledge_count = sum(
        len(manager.load_partition(name).get("items", []))
        for name in ("programming", "science", "finance", "general")
    )
    total_summary_count = sum(len(value) for value in latest_summary.values() if isinstance(value, list))
    reflection_count = len(reflection_data.get("insights", []))
    runtime_record = {
        "step": int(working["step_count"]),
        "mode": working.get("mode", "normal"),
        "partition": partition,
        "strategy": working.get("last_strategy", ""),
        "health": float(working.get("last_health", 1.0)),
        "knowledge_count": total_knowledge_count,
        "summary_count": total_summary_count,
        "reflection_count": reflection_count,
        "selfcheck_triggered": maintenance_due,
        "cleaner_triggered": should_clean,
        "timestamp": now_iso(),
    }
    records = runtime_data.setdefault("records", [])
    records.append(runtime_record)
    runtime_data["records"] = records[-50:]
    manager.save_runtime(runtime_data)
    manager.save_partition(partition, partition_data)

    print(f"Execution result: {result['output']}")
    print(f"Current step: {working['current_step']}")
    print(f"Step count: {working['step_count']}")
    print(f"Partition: {partition}")
    print(f"Retrieved items: {len(retrieved_items[:3])}")
    print(f"Health: {health_report['health']}")
    print(f"Mode: {working.get('mode', 'normal')}")
    print(f"Suggestions: {suggestions if suggestions else []}")
    print(f"Proposals: {proposals if proposals else []}")
    print(f"Patch applied: {patch_applied}")
    print()
    print_summary()


if __name__ == "__main__":
    main()
```

A.3　Start-OpenClaw-WSL-Interactive.cmd

作用说明

这是启动链条文件，当前显式把 `/mnt/d/openclaw` 作为 external memory root 传给 WSL。

完整内容

```
@echo off
setlocal
title OpenClaw (WSL Interactive)

REM Open an interactive Ubuntu shell and keep it open.
REM Runs OpenClaw once, then drops you into a normal bash prompt (Linux side).
REM D:\openclaw is the canonical external memory home exposed to WSL as /mnt/d/openclaw.
wsl.exe -d Ubuntu -- bash --login -i -c "set -e; export OPENCLAW_EXTERNAL_MEMORY_ROOT=/mnt/d
    /openclaw; /home/park/.local/bin/openclaw gateway start || true; echo; echo 'OpenClaw:
    try  openclaw gateway status'; echo 'Dashboard: http://127.0.0.1:18789/'; echo; exec
    bash --login -i"
```

A.4　core/utils.py

作用说明

提供 JSON 处理、文本归一化、token 处理和通用基础工具。

完整内容

```
from __future__ import annotations

import json
import re
import time
from datetime import datetime, timezone
from pathlib import Path
from typing import Any, Iterable


def ensure_file(path: str | Path, default_data: Any) -> None:
    """Create a JSON file with default data when it does not exist."""
    file_path = Path(path)
    file_path.parent.mkdir(parents=True, exist_ok=True)
    if not file_path.exists():
        save_json(file_path, default_data)


```

```
def load_json(path: str | Path) -> Any:
    """Load and parse JSON from a file."""
    with Path(path).open("r", encoding="utf-8") as handle:
        return json.load(handle)


def save_json(path: str | Path, data: Any) -> None:
    """Write JSON data to a file using UTF-8 and stable formatting."""
    file_path = Path(path)
    file_path.parent.mkdir(parents=True, exist_ok=True)
    temp_path = file_path.with_suffix(f"{file_path.suffix}.tmp")
    with temp_path.open("w", encoding="utf-8") as handle:
        json.dump(data, handle, ensure_ascii=False, indent=2, sort_keys=False)
        handle.write("\n")
    last_error: OSError | None = None
    for _ in range(5):
        try:
            temp_path.replace(file_path)
            return
        except PermissionError as exc:
            last_error = exc
            time.sleep(0.05)
    if last_error is not None:
        raise last_error


def now_iso() -> str:
    """Return the current UTC timestamp in ISO 8601 format."""
    return datetime.now(timezone.utc).replace(microsecond=0).isoformat()


def clamp(value: float, min_value: float, max_value: float) -> float:
    """Clamp a numeric value into the given inclusive range."""
    return max(min_value, min(max_value, value))


def unique_preserve_order(items: Iterable[str]) -> list[str]:
    """Deduplicate items while preserving original order."""
    seen: set[str] = set()
    result: list[str] = []
    for item in items:
        if item not in seen:
            seen.add(item)
            result.append(item)
    return result


```

```
def normalize_text(text: str) -> str:
    """Lowercase text, trim it, and collapse internal whitespace."""
    lowered = text.lower().strip()
    return re.sub(r"\s+", " ", lowered)


def compact_step_label(text: str) -> str:
    """Convert a step description into a short repetition-friendly label."""
    cleaned = normalize_text(text)
    cleaned = re.sub(r"[^a-z0-9\u4e00-\u9fff\s]", " ", cleaned)
    cleaned = re.sub(r"\s+", " ", cleaned).strip()
    if re.search(r"[\u4e00-\u9fff]", cleaned) and " " not in cleaned:
        return cleaned[:12]
    words = [word for word in cleaned.split(" ") if word][:6]
    return " ".join(words)
```

A.5　core/memory_manager.py

作用说明

负责 memory 文件创建、读写、知识写入、summary 追加、runtime 快照持久化。

完整内容

```
from __future__ import annotations

from pathlib import Path
from typing import Any

from core.retriever import rebuild_index, tokenize
from core.utils import clamp, ensure_file, load_json, normalize_text, now_iso, save_json,
    unique_preserve_order


DEFAULT_WORKING = {
    "goal": "Build a stable minimal memory system for OpenClaw using Python and JSON.",
    "current_step": "Initialize memory files and managers",
    "next_action": "Create modular memory components",
    "focus": "stability and simplicity",
    "step_history": [],
    "step_count": 0,
    "last_action": "",
    "retrieval_cooldown_until": 0,
    "knowledge_write_cooldown_until": 0,
    "last_partition": "",
    "last_strategy": "",
```

```
    "last_query": "",
    "last_result_keys": [],
    "recent_suggestion_steps": [],
    "mode": "normal",
    "last_health": 1.0,
    "evolution_budget": {
        "max_suggestions_per_10_steps": 2,
        "max_patches_per_20_steps": 1,
        "disable_patch_after_failures": 2,
        "patch_failures": 0,
        "last_patch_step": 0,
        "no_benefit_patch_count": 0,
        "auto_patch_disabled": False,
    },
}

DEFAULT_SUMMARY = {
    "facts": [],
    "decisions": [],
    "failures": [],
    "progress": [],
    "anti_patterns": [],
}

DEFAULT_STRATEGY = {
    "strategies": [
        {
            "name": "direct_execution",
            "description": "Do the simplest valid next action directly",
            "score": 1.0,
            "usage": 0,
            "last_used_step": 0,
        },
        {
            "name": "decompose_task",
            "description": "Break task into smaller steps when needed",
            "score": 0.8,
            "usage": 0,
            "last_used_step": 0,
        },
        {
            "name": "fallback_simplify",
            "description": "Reduce complexity after repetition or failure",
            "score": 0.7,
            "usage": 0,
            "last_used_step": 0,
        },
```

```
    ]
}

DEFAULT_REFLECTION = {"insights": []}
DEFAULT_SELFCHECK = {
    "checks": [],
    "evaluations": [],
    "optimizations": [],
    "patch_proposals": [],
    "applied_patches": [],
    "last_run": "",
}
DEFAULT_RUNTIME = {"records": []}
DEFAULT_META = {
    "rules": [
        {"rule": "Only store high-value reusable knowledge", "confidence": 1.0, "last_validated_step": 0},
        {"rule": "Never retrieve more than top 3 items", "confidence": 1.0, "last_validated_step": 0},
        {"rule": "Avoid repeated steps", "confidence": 1.0, "last_validated_step": 0},
        {"rule": "Prefer direct execution when next action is obvious", "confidence": 1.0, "last_validated_step": 0},
        {"rule": "Reduce memory growth whenever possible", "confidence": 1.0, "last_validated_step": 0},
    ],
    "patterns": [
        {"pattern": "After repeated failure, switch strategy", "confidence": 1.0, "last_validated_step": 0},
        {"pattern": "Avoid frequent knowledge writes", "confidence": 1.0, "last_validated_step": 0},
        {"pattern": "Use summary instead of knowledge when possible", "confidence": 1.0, "last_validated_step": 0},
    ],
    "constraints": [
        "Keep memory small",
        "Keep retrieval fast",
        "Keep disk usage low",
        "Keep evaluation lightweight",
        "Prefer parameter tuning over code patching",
    ],
    "evaluation_rules": [
        "If duplicate ratio rises, prefer cleaning over new learning",
        "If retrieval effectiveness drops, reduce retrieval frequency and clean the active partition",
        "If a strategy is overused without strong success, diversify",
        "If memory pressure rises, prioritize compression and deletion over storage",
    ],
```

```
}
DEFAULT_PARTITION = {"items": [], "index": {}}
VALID_PARTITIONS = ("programming", "science", "finance", "general")


class MemoryManager:
    def __init__(self, base_dir: str | Path) -> None:
        self.base_dir = Path(base_dir)
        self.memory_dir = self.base_dir / "memory"
        self.knowledge_dir = self.memory_dir / "knowledge"

    def initialize(self) -> None:
        ensure_file(self.memory_dir / "working.json", DEFAULT_WORKING)
        ensure_file(self.memory_dir / "summary.json", DEFAULT_SUMMARY)
        ensure_file(self.memory_dir / "strategy.json", DEFAULT_STRATEGY)
        ensure_file(self.memory_dir / "reflection.json", DEFAULT_REFLECTION)
        ensure_file(self.memory_dir / "selfcheck.json", DEFAULT_SELFCHECK)
        ensure_file(self.memory_dir / "runtime.json", DEFAULT_RUNTIME)
        ensure_file(self.memory_dir / "meta.json", DEFAULT_META)
        for partition in VALID_PARTITIONS:
            ensure_file(self.knowledge_dir / f"{partition}.json", DEFAULT_PARTITION)

    def load_working(self) -> dict[str, Any]:
        return self._merge_defaults(load_json(self.memory_dir / "working.json"),
    DEFAULT_WORKING)

    def save_working(self, data: dict[str, Any]) -> None:
        save_json(self.memory_dir / "working.json", data)

    def load_summary(self) -> dict[str, Any]:
        return self._merge_defaults(load_json(self.memory_dir / "summary.json"),
    DEFAULT_SUMMARY)

    def save_summary(self, data: dict[str, Any]) -> None:
        save_json(self.memory_dir / "summary.json", data)

    def load_strategy(self) -> dict[str, Any]:
        return self._merge_defaults(load_json(self.memory_dir / "strategy.json"),
    DEFAULT_STRATEGY)

    def save_strategy(self, data: dict[str, Any]) -> None:
        save_json(self.memory_dir / "strategy.json", data)

    def load_reflection(self) -> dict[str, Any]:
        return self._merge_defaults(load_json(self.memory_dir / "reflection.json"),
    DEFAULT_REFLECTION)

```

```
    def save_reflection(self, data: dict[str, Any]) -> None:
        save_json(self.memory_dir / "reflection.json", data)

    def load_selfcheck(self) -> dict[str, Any]:
        data = self._merge_defaults(load_json(self.memory_dir / "selfcheck.json"),
    DEFAULT_SELFCHECK)
        for check in data.get("checks", []):
            if "disk_usage_bytes" in check and "disk_usage" not in check:
                check["disk_usage"] = int(check.pop("disk_usage_bytes"))
        return data

    def save_selfcheck(self, data: dict[str, Any]) -> None:
        save_json(self.memory_dir / "selfcheck.json", data)

    def load_runtime(self) -> dict[str, Any]:
        return self._merge_defaults(load_json(self.memory_dir / "runtime.json"),
    DEFAULT_RUNTIME)

    def save_runtime(self, data: dict[str, Any]) -> None:
        save_json(self.memory_dir / "runtime.json", data)

    def load_meta(self) -> dict[str, Any]:
        return self._merge_defaults(load_json(self.memory_dir / "meta.json"), DEFAULT_META)

    def save_meta(self, data: dict[str, Any]) -> None:
        save_json(self.memory_dir / "meta.json", data)

    def load_partition(self, partition: str) -> dict[str, Any]:
        self._validate_partition(partition)
        return self._merge_defaults(load_json(self.knowledge_dir / f"{partition}.json"),
    DEFAULT_PARTITION)

    def save_partition(self, partition: str, data: dict[str, Any]) -> None:
        self._validate_partition(partition)
        save_json(self.knowledge_dir / f"{partition}.json", data)

    def append_summary(self, section: str, text: str, max_items: int = 50) -> None:
        cleaned = text.strip()
        if not cleaned:
            return
        summary = self.load_summary()
        if section not in summary:
            summary[section] = []
        summary[section].append(cleaned)
        summary[section] = summary[section][-max_items:]
        self.save_summary(summary)

```

```
    def add_knowledge(
        self,
        partition: str,
        content: str,
        tags: list[str],
        importance: float,
        current_step_count: int = 0,
        working: dict[str, Any] | None = None,
    ) -> bool:
        self._validate_partition(partition)
        normalized = normalize_text(content)
        tokens = [token for token in tokenize(normalized) if len(token) > 2]
        if len(normalized) < 20 or len(tokens) < 3:
            return False

        active_working = working if working is not None else self.load_working()
        if not self.knowledge_write_allowed(active_working):
            return False

        content_short = content.strip()
        if len(content_short) > 120:
            content_short = f"{content_short[:117].rstrip()}..."
        tag_list = unique_preserve_order(normalize_text(tag) for tag in tags if
    normalize_text(tag))
        key = "-".join(tokens[:4])

        partition_data = self.load_partition(partition)
        token_set = set(tokens)
        for item in partition_data["items"]:
            existing = normalize_text(item.get("content", ""))
            if existing == normalized:
                return False
            existing_tokens = set(item.get("tokens", []))
            overlap = len(token_set & existing_tokens) / max(1, len(token_set |
    existing_tokens))
            if overlap > 0.7:
                return False

        entry = {
            "content": content_short,
            "tags": tag_list[:5],
            "importance": clamp(importance, 0.0, 1.0),
            "created_at": now_iso(),
            "last_accessed_step": current_step_count,
            "access_count": 0,
            "tokens": tokens[:8],
            "key": key,
```

```
        }
        partition_data["items"].append(entry)
        rebuild_index(partition_data)
        self.save_partition(partition, partition_data)
        self.mark_knowledge_write_used(active_working)
        self.save_working(active_working)
        return True

    def update_working_step(self, current_step: str, next_action: str, focus: str, last_action: str = "") -> None:
        working = self.load_working()
        working["current_step"] = current_step
        working["next_action"] = next_action
        working["focus"] = focus
        working["last_action"] = last_action
        self.save_working(working)

    def knowledge_write_allowed(self, working: dict[str, Any]) -> bool:
        return int(working.get("step_count", 0)) >= int(working.get("knowledge_write_cooldown_until", 0))

    def mark_knowledge_write_used(self, working: dict[str, Any]) -> None:
        working["knowledge_write_cooldown_until"] = int(working.get("step_count", 0)) + 2

    def _validate_partition(self, partition: str) -> None:
        if partition not in VALID_PARTITIONS:
            raise ValueError(f"Invalid partition: {partition}")

    def _merge_defaults(self, data: Any, defaults: Any) -> Any:
        if isinstance(defaults, dict) and isinstance(data, dict):
            merged = {key: self._merge_defaults(data.get(key), value) for key, value in defaults.items()}
            for key, value in data.items():
                if key not in merged:
                    merged[key] = value
            return merged
        if data is None:
            return defaults
        return data
```

A.6　core/router.py

作用说明

负责把输入路由到 programming / science / finance / general 分区。

完整内容

```python
from __future__ import annotations


def select_partition(text: str) -> str:
    lowered = text.lower()
    programming = (
        "code", "bug", "python", "memory", "dev", "function", "class", "json", "module",
        "代码", "报错", "编程", "开发", "函数", "类", "模块", "内存", "系统", "调试",
    )
    science = (
        "math", "physics", "logic", "theorem", "proof",
        "数学", "物理", "逻辑", "定理", "证明", "科学",
    )
    finance = (
        "money", "market", "stock", "trading", "economy",
        "钱", "市场", "股票", "交易", "经济", "金融",
    )
    if any(keyword in lowered for keyword in programming):
        return "programming"
    if any(keyword in lowered for keyword in science):
        return "science"
    if any(keyword in lowered for keyword in finance):
        return "finance"
    return "general"
```

A.7　core/retriever.py

作用说明

负责检索、query 复用、top-k 控制和索引读取。

完整内容

```python
from __future__ import annotations

import re
from typing import Any

from core.utils import normalize_text, unique_preserve_order

STOPWORDS = {
    "a", "an", "and", "are", "as", "at", "be", "by", "for", "from", "in", "is",
    "it", "of", "on", "or", "the", "to", "with", "this", "that", "into", "only",
}
```

```
EARLY_RETURN_SCORE = 5.2
MAX_RETRIEVE_K = 3
CACHE_SIMILARITY_THRESHOLD = 0.75


def tokenize(text: str) -> list[str]:
    normalized = normalize_text(text)
    tokens: list[str] = []
    for token in re.findall(r"[a-z0-9]+|[\u4e00-\u9fff]+", normalized):
        if re.fullmatch(r"[a-z0-9]+", token):
            if token not in STOPWORDS:
                tokens.append(token)
            continue
        if len(token) == 1:
            tokens.append(token)
            continue
        tokens.append(token)
        tokens.extend(list(token))
        for idx in range(len(token) - 1):
            tokens.append(token[idx : idx + 2])
    return unique_preserve_order(tokens)


def compute_overlap_score(query_tokens: list[str], item: dict[str, Any]) -> float:
    item_tokens = set(item.get("tokens", []))
    query_set = set(query_tokens)
    token_overlap = len(query_set & item_tokens) / max(1, len(query_set))
    tag_overlap = len(query_set & set(item.get("tags", [])))
    key_tokens = set(str(item.get("key", "")).split("-"))
    key_overlap = len(query_set & key_tokens)
    access_bonus = min(float(item.get("access_count", 0)) * 0.05, 0.3)
    return (
        tag_overlap * 3.0
        + token_overlap * 1.5
        + key_overlap * 2.0
        + float(item.get("importance", 0.0))
        + access_bonus
    )


def retrieve_top_k(partition_data: dict[str, Any], query: str, k: int = 3) -> list[dict[str,
     Any]]:
    query_tokens = tokenize(query)
    if not query_tokens:
        return []

    index = partition_data.get("index", {})
```

```
    candidate_indices: set[int] = set()
    for token in query_tokens:
        candidate_indices.update(index.get(token, []))
    if not candidate_indices:
        candidate_indices = set(range(len(partition_data.get("items", []))))

    scored: list[tuple[float, float, int, dict[str, Any]]] = []
    for index_value in sorted(candidate_indices):
        item = partition_data["items"][index_value]
        score = compute_overlap_score(query_tokens, item)
        if score <= 0:
            continue
        scored.append((score, float(item.get("importance", 0.0)), len(item.get("content",
    "")), item))

    scored.sort(key=lambda row: (-row[0], -row[1], row[3].get("created_at", ""), row[2]))
    if scored and scored[0][0] >= EARLY_RETURN_SCORE:
        return [scored[0][3]]
    return [row[3] for row in scored[: max(1, min(k, MAX_RETRIEVE_K))]]


def query_similarity(left: str, right: str) -> float:
    left_tokens = set(tokenize(left))
    right_tokens = set(tokenize(right))
    if not left_tokens or not right_tokens:
        return 0.0
    return len(left_tokens & right_tokens) / max(1, len(left_tokens | right_tokens))


def reuse_cached_results(
    partition_data: dict[str, Any],
    query: str,
    working: dict[str, Any],
    partition: str,
) -> list[dict[str, Any]]:
    if working.get("last_partition") != partition:
        return []
    if query_similarity(query, str(working.get("last_query", ""))) <
    CACHE_SIMILARITY_THRESHOLD:
        return []

    items_by_key = {
        str(item.get("key", "")): item
        for item in partition_data.get("items", [])
        if item.get("key")
    }
    cached: list[dict[str, Any]] = []
```

```
    for key in working.get("last_result_keys", [])[:MAX_RETRIEVE_K]:
        item = items_by_key.get(str(key))
        if item is not None:
            cached.append(item)
    return cached


def retrieval_allowed(working: dict[str, Any]) -> bool:
    return int(working.get("step_count", 0)) >= int(working.get("retrieval_cooldown_until",
    0))


def mark_retrieval_used(working: dict[str, Any]) -> None:
    working["retrieval_cooldown_until"] = int(working.get("step_count", 0)) + 2


def touch_retrieved_items(items: list[dict[str, Any]], step_count: int) -> None:
    for item in items:
        item["access_count"] = int(item.get("access_count", 0)) + 1
        item["last_accessed_step"] = step_count


def rebuild_index(partition_data: dict[str, Any]) -> dict[str, list[int]]:
    index: dict[str, list[int]] = {}
    for idx, item in enumerate(partition_data.get("items", [])):
        useful_tokens = []
        for token in item.get("tokens", []):
            if token in STOPWORDS or token in useful_tokens:
                continue
            useful_tokens.append(token)
            if len(useful_tokens) >= 5:
                break
        for token in useful_tokens:
            index.setdefault(token, []).append(idx)
    partition_data["index"] = index
    return index
```

A.8　core/strategy_manager.py

作用说明

负责策略选择、使用统计、偏斜控制。

完整内容

```
from __future__ import annotations
```

```python

from typing import Any

from core.utils import clamp

RECENCY_BONUS = 0.03
FORCE_NEW_PENALTY = 0.5


def select_best_strategy(
    strategy_data: dict[str, Any],
    current_step: str = "",
    last_strategy: str | None = None,
    force_new: bool = False,
    step_count: int = 0,
) -> dict[str, Any]:
    strategies = strategy_data.get("strategies", [])
    chosen: dict[str, Any] | None = None
    chosen_score = -1.0
    lowered = current_step.lower()

    for strategy in strategies:
        name = strategy.get("name", "")
        score = float(strategy.get("score", 0.0))
        recency_gap = max(step_count - int(strategy.get("last_used_step", 0)), 0)
        score += min(recency_gap, 3) * RECENCY_BONUS
        if "repeat" in lowered or "failure" in lowered:
            if name == "fallback_simplify":
                score += 0.3
        elif any(word in lowered for word in ("build", "implement", "create")):
            if name == "direct_execution":
                score += 0.2
        else:
            if name == "decompose_task":
                score += 0.1

        if force_new and name == last_strategy:
            score -= FORCE_NEW_PENALTY

        if score > chosen_score:
            chosen = strategy
            chosen_score = score

    if chosen is None and strategies:
        chosen = strategies[0]
    if chosen is None:
        chosen = {"name": "direct_execution", "score": 1.0, "usage": 0, "last_used_step": 0}
```

```

    chosen["usage"] = int(chosen.get("usage", 0)) + 1
    chosen["last_used_step"] = step_count
    return chosen


def update_strategy_score(strategy_data: dict[str, Any], strategy_name: str, success: bool)
    -> None:
    for strategy in strategy_data.get("strategies", []):
        if strategy.get("name") == strategy_name:
            delta = 0.1 if success else -0.2
            strategy["score"] = clamp(float(strategy.get("score", 0.0)) + delta, 0.1, 3.0)
            return
```

A.9　core/reflection_manager.py

作用说明

负责周期性生成简短可执行的反思。

完整内容

```
from __future__ import annotations

from typing import Any


def should_reflect(step_count: int) -> bool:
    return step_count > 0 and step_count % 3 == 0


def generate_reflection_insight(working: dict[str, Any], summary: dict[str, Any]) -> str:
    failures = summary.get("failures", [])[-2:]
    if failures:
        return "Simplify sooner after a failure instead of repeating the same approach."
    history = working.get("step_history", [])[-3:]
    if len(history) >= 2 and len(set(history)) < len(history):
        return "Change strategy when recent step labels start repeating."
    if working.get("next_action"):
        return "Prefer direct execution when the next action is already obvious."
    return "Keep the active context small and focused."


def store_reflection(reflection_data: dict[str, Any], insight: str, max_items: int = 20) ->
    None:
    text = insight.strip()
```

```
    if not text:
        return
    insights = reflection_data.setdefault("insights", [])
    if text in insights:
        return
    insights.append(text)
    reflection_data["insights"] = insights[-max_items:]
```

A.10 core/cleaner.py

作用说明

负责 exact duplicate 与 near duplicate 清理，以及 summary 压缩。

完整内容

```
from __future__ import annotations

import re

from core.retriever import rebuild_index, tokenize
from core.utils import clamp, normalize_text, unique_preserve_order

LOW_IMPORTANCE_THRESHOLD = 0.3
STALE_REMOVE_STEPS = 10
STALE_DECAY_STEPS = 8
SUMMARY_SECTION_CAP = 50
SUMMARY_KEEP_NEWEST = 30
SUMMARY_KEEP_HIGHEST = 20
NEAR_DUPLICATE_OVERLAP = 0.7


def needs_cleaning(partition_data: dict) -> bool:
    items = partition_data.get("items", [])
    if len(items) > 100:
        return True
    if not items:
        return False
    low_ratio = sum(1 for item in items if float(item.get("importance", 0.0)) <
    LOW_IMPORTANCE_THRESHOLD) / len(items)
    return low_ratio > 0.3 or len(items) > 120


def clean_partition(partition_data: dict, current_step_count: int = 0) -> dict:
    merged_items: list[dict] = []
    for item in partition_data.get("items", []):
```

```
        importance = float(item.get("importance", 0.0))
        stale_steps = current_step_count - int(item.get("last_accessed_step", 0))
        if importance < LOW_IMPORTANCE_THRESHOLD and int(item.get("access_count", 0)) == 0:
            continue
        if int(item.get("access_count", 0)) == 0 and stale_steps > STALE_REMOVE_STEPS:
            continue
        if int(item.get("access_count", 0)) >= 3:
            item["importance"] = clamp(importance + 0.05, 0.0, 1.0)
        elif stale_steps > STALE_DECAY_STEPS:
            item["importance"] = clamp(importance - 0.05, 0.0, 1.0)
        if len(item.get("content", "")) > 120:
            item["content"] = f"{item['content'][:117].rstrip()}..."
        item_tokens = set(item.get("tokens", []))

        merged = False
        for existing in merged_items:
            existing_tokens = set(existing.get("tokens", []))
            overlap = len(item_tokens & existing_tokens) / max(1, len(item_tokens | existing_tokens))
            exact_match = normalize_text(existing.get("content", "")) == normalize_text(item.get("content", ""))
            if not exact_match and overlap <= NEAR_DUPLICATE_OVERLAP:
                continue
            if float(item.get("importance", 0.0)) > float(existing.get("importance", 0.0)):
                existing["content"] = item["content"]
                existing["importance"] = item["importance"]
                existing["created_at"] = item.get("created_at", existing.get("created_at", ""))
                existing["key"] = item.get("key", existing.get("key", ""))
                existing["tokens"] = item.get("tokens", existing.get("tokens", []))
            existing["tags"] = unique_preserve_order([*existing.get("tags", []), *item.get("tags", [])])[:5]
            existing["access_count"] = max(int(existing.get("access_count", 0)), int(item.get("access_count", 0)))
            existing["last_accessed_step"] = max(
                int(existing.get("last_accessed_step", 0)),
                int(item.get("last_accessed_step", 0)),
            )
            merged = True
            break
        if not merged:
            merged_items.append(item)

    partition_data["items"] = merged_items[:120]
    rebuild_index(partition_data)
    return partition_data

```

```python

def light_compress_summary(summary_data: dict, aggressive: bool = False) -> dict:
    keep_newest = 20 if aggressive else SUMMARY_KEEP_NEWEST
    keep_highest = 10 if aggressive else SUMMARY_KEEP_HIGHEST
    for key, value in summary_data.items():
        if isinstance(value, list):
            deduped = unique_preserve_order(str(item).strip() for item in value if str(item)
    .strip())
            if len(deduped) > SUMMARY_SECTION_CAP:
                recent = deduped[-keep_newest:]
                older = deduped[:-keep_newest]
                older_sorted = sorted(
                    older,
                    key=lambda text: (-_summary_value(text), older.index(text)),
                )
                kept = recent + older_sorted[:keep_highest]
                deduped = unique_preserve_order(kept)
            summary_data[key] = deduped
    return summary_data


def _summary_value(text: str) -> int:
    normalized = normalize_text(text)
    tokens = [token for token in tokenize(normalized) if len(token) > 1]
    unique_count = len(set(tokens))
    signal_bonus = 2 if any(char.isdigit() for char in text) else 0
    signal_bonus += 1 if "\\" in text or "/" in text else 0
    signal_bonus += 1 if normalized.startswith("do not") else 0
    signal_bonus += 1 if normalized.startswith("不要") else 0
    return min(unique_count, 12) + signal_bonus
```

A.11　core/executor.py

作用说明

负责最小执行上下文的构建和一步执行结果。

完整内容

```python
from __future__ import annotations

from core.utils import compact_step_label

REPETITION_WINDOW = 5
REPETITION_LIMIT = 2

```

```

def detect_repetition(step_history: list[str], current_step: str, next_action: str) -> bool:
    recent = step_history[-REPETITION_WINDOW:]
    labels = recent + [compact_step_label(current_step), compact_step_label(next_action)]
    target = labels[-2]
    return labels.count(target) >= REPETITION_LIMIT


def build_context(
    working: dict,
    strategy: dict,
    summary: dict,
    reflections: dict,
    retrieved_items: list[dict],
    meta: dict,
) -> dict:
    return {
        "goal": working.get("goal", ""),
        "current_step": working.get("current_step", ""),
        "next_action": working.get("next_action", ""),
        "focus": working.get("focus", ""),
        "strategy": strategy.get("name", "direct_execution"),
        "recent_failures": summary.get("failures", [])[-3:],
        "reflections": reflections.get("insights", [])[-3:],
        "retrieved": [item.get("content", "") for item in retrieved_items[:3]],
        "rules": [rule.get("rule", "") for rule in meta.get("rules", [])[:3]],
    }


def execute_step(context: dict) -> dict:
    current_step = str(context.get("current_step", "")).strip()
    next_action = str(context.get("next_action", "")).strip()

    if not current_step:
        return {
            "success": True,
            "output": "Initialized the minimal memory kernel.",
            "new_fact": "A small deterministic memory loop is easier to keep stable.",
            "decision": "Start from the simplest valid next action.",
            "failure": "",
        }

    return {
        "success": True,
        "output": f"Completed step: {current_step}",
        "new_fact": "",
        "decision": next_action or "Review results and continue with the next minimal
```

```
    improvement",
        "failure": "",
    }
```

A.12　core/selfcheck_manager.py

作用说明

负责数值型 selfcheck、mode、suggestions 和 patch safeguard。

完整内容

```
from __future__ import annotations

import os
import subprocess
import sys
from pathlib import Path
from typing import Any

from core.utils import clamp, now_iso


def _read_numeric_setting(base_dir: str | Path, target_file: str, setting: str) -> float |
    int | None:
    file_path = Path(base_dir) / target_file
    if not file_path.exists():
        return None
    prefix = f"{setting} = "
    for line in file_path.read_text(encoding="utf-8").splitlines():
        stripped = line.strip()
        if not stripped.startswith(prefix):
            continue
        raw_value = stripped[len(prefix):].strip()
        try:
            return int(raw_value) if raw_value.isdigit() else float(raw_value)
        except ValueError:
            return None
    return None


def run_self_check(
    working: dict[str, Any],
    summary: dict[str, Any],
    strategy_data: dict[str, Any],
    reflection_data: dict[str, Any],
```

```
    partition_data_map: dict[str, dict[str, Any]],
) -> dict[str, float | int]:
    all_items = [item for data in partition_data_map.values() for item in data.get("items", []))]
    item_count = len(all_items)
    unique_contents = len({item.get("content", "") for item in all_items})
    low_importance = sum(1 for item in all_items if float(item.get("importance", 0.0)) < 0.3)
    strategy_usage = [int(strategy.get("usage", 0)) for strategy in strategy_data.get("strategies", [])]
    total_strategy_usage = sum(strategy_usage) or 1
    max_strategy_usage = max(strategy_usage) if strategy_usage else 0
    disk_usage = (
        sum(len(str(data)) for data in partition_data_map.values())
        + len(str(summary))
        + len(str(reflection_data))
        + len(str(strategy_data))
    )
    retrieval_effectiveness = 1.0 if working.get("last_result_keys") else 0.5
    complexity_score = clamp(
        (
            len(summary.get("progress", []))
            + len(summary.get("failures", []))
            + len(reflection_data.get("insights", []))
        ) / 60.0,
        0.0,
        1.0,
    )

    return {
        "memory_pressure": round(min(item_count / 100.0, 1.0), 3),
        "duplicate_ratio": round(0.0 if item_count == 0 else 1 - (unique_contents / item_count), 3),
        "low_importance_ratio": round(0.0 if item_count == 0 else low_importance / item_count, 3),
        "strategy_skew": round(max_strategy_usage / total_strategy_usage, 3),
        "retrieval_effectiveness": round(retrieval_effectiveness, 3),
        "disk_usage": int(disk_usage),
        "complexity_score": round(complexity_score, 3),
    }


def evaluate_health(current_check: dict[str, Any], previous_check: dict[str, Any] | None = None) -> dict[str, float]:
    pressure = float(current_check["memory_pressure"])
    noise = float(current_check["duplicate_ratio"]) + float(current_check["low_importance_ratio"])
```

```
    complexity = float(current_check["complexity_score"])
    disk_factor = clamp(float(current_check["disk_usage"]) / 20000.0, 0.0, 1.0)
    health = 1.0
    health -= pressure * 0.25
    health -= noise * 0.25
    health -= float(current_check["strategy_skew"]) * 0.15
    health -= (1.0 - float(current_check["retrieval_effectiveness"])) * 0.1
    health -= complexity * 0.15
    health -= disk_factor * 0.1
    health = round(clamp(health, 0.0, 1.0), 3)
    previous_health = float(previous_check.get("health", health)) if previous_check else health
    return {
        "health": health,
        "delta": round(health - previous_health, 3),
        "pressure": round(pressure, 3),
        "noise": round(noise, 3),
        "complexity": round(complexity, 3),
    }


def select_mode(health_report: dict[str, Any], check_report: dict[str, Any], current_mode: str) -> str:
    if (
        check_report["memory_pressure"] >= 0.3
        or health_report["complexity"] >= 0.45
        or float(check_report.get("disk_usage", 0)) >= 15000
    ):
        return "convergence"
    if current_mode == "convergence" and health_report["health"] >= 0.85 and health_report["complexity"] < 0.35:
        return "normal"
    if health_report["health"] < 0.8 or health_report["delta"] < -0.03 or check_report["memory_pressure"] > 0.35:
        return "stability"
    if current_mode == "stability" and health_report["health"] >= 0.82:
        return "normal"
    return "normal"


def generate_optimization_suggestions(
    check_report: dict[str, Any],
    health_report: dict[str, Any],
    working: dict[str, Any],
    max_suggestions: int = 2,
) -> list[str]:
    budget = working.get("evolution_budget", {})
```

```
    recent_count = sum(
        1
        for item in working.get("recent_suggestion_steps", [])
        if int(working.get("step_count", 0)) - int(item) < 10
    )
    allowed = max(0, min(max_suggestions, int(budget.get("max_suggestions_per_10_steps",
    max_suggestions)) - recent_count))
    suggestions: list[str] = []
    if health_report["pressure"] > 0.2:
        suggestions.append(f"clean {working.get('last_partition', 'general')} earlier")
    if check_report["retrieval_effectiveness"] < 0.75:
        suggestions.append("reduce retrieval frequency")
    if check_report["strategy_skew"] > 0.7:
        suggestions.append("diversify strategy usage")
    return suggestions[:allowed]


def generate_patch_proposals(
    check_report: dict[str, Any],
    health_report: dict[str, Any],
    working: dict[str, Any],
    base_dir: str | Path,
) -> list[dict[str, Any]]:
    budget = working.get("evolution_budget", {})
    if bool(budget.get("auto_patch_disabled", False)):
        return []
    if int(budget.get("patch_failures", 0)) >= int(budget.get("disable_patch_after_failures
    ", 2)):
        return []
    if working.get("mode") == "convergence":
        return []
    proposals: list[dict[str, Any]] = []
    if check_report["retrieval_effectiveness"] < 0.75 and working.get("mode") == "stability
    ":
        current = _read_numeric_setting(base_dir, "core/retriever.py", "EARLY_RETURN_SCORE")
        if current is not None and float(current) > 5.2:
            proposals.append(
                {
                    "type": "threshold_change",
                    "target_file": "core/retriever.py",
                    "setting": "EARLY_RETURN_SCORE",
                    "old": float(current),
                    "new": 5.2,
                    "reason": "allow stronger reuse under stability mode",
                }
            )
    elif check_report["strategy_skew"] > 0.72:
```

```
        current = _read_numeric_setting(base_dir, "core/strategy_manager.py", "FORCE_NEW_PENALTY")
        if current is not None and float(current) < 0.6:
            proposals.append(
                {
                    "type": "threshold_change",
                    "target_file": "core/strategy_manager.py",
                    "setting": "FORCE_NEW_PENALTY",
                    "old": float(current),
                    "new": 0.6,
                    "reason": "encourage strategy diversification",
                }
            )
    elif health_report["pressure"] > 0.2:
        current = _read_numeric_setting(base_dir, "core/cleaner.py", "STALE_REMOVE_STEPS")
        if current is not None and float(current) > 9:
            proposals.append(
                {
                    "type": "threshold_change",
                    "target_file": "core/cleaner.py",
                    "setting": "STALE_REMOVE_STEPS",
                    "old": int(current),
                    "new": 9,
                    "reason": "shrink stale unused items earlier",
                }
            )
    return proposals[:1]


def apply_safe_patch_proposals(
    base_dir: str | Path,
    proposals: list[dict[str, Any]],
    working: dict[str, Any],
) -> tuple[bool, list[dict[str, Any]]]:
    if not proposals:
        return False, []
    budget = working.get("evolution_budget", {})
    step_count = int(working.get("step_count", 0))
    if step_count - int(budget.get("last_patch_step", 0)) < 20:
        return False, []
    if int(budget.get("patch_failures", 0)) >= int(budget.get("disable_patch_after_failures", 2)):
        return False, []
    if bool(budget.get("auto_patch_disabled", False)):
        return False, []

    proposal = proposals[0]
```

```
    allowlist = {
        ("core/retriever.py", "EARLY_RETURN_SCORE"),
        ("core/cleaner.py", "STALE_REMOVE_STEPS"),
        ("core/strategy_manager.py", "FORCE_NEW_PENALTY"),
        ("core/executor.py", "REPETITION_LIMIT"),
    }
    if (proposal.get("target_file"), proposal.get("setting")) not in allowlist:
        return False, []

    file_path = Path(base_dir) / str(proposal["target_file"])
    original = file_path.read_text(encoding="utf-8")
    old_line = f"{proposal['setting']} = {proposal['old']}"
    new_line = f"{proposal['setting']} = {proposal['new']}"
    if old_line not in original:
        return False, []

    updated = original.replace(old_line, new_line, 1)
    try:
        compile(updated, str(file_path), "exec")
        temp_path = file_path.with_suffix(f"{file_path.suffix}.tmp")
        temp_path.write_text(updated, encoding="utf-8")
        temp_path.replace(file_path)
        result = subprocess.run(
            [sys.executable, "-m", "py_compile", str(file_path)],
            cwd=str(base_dir),
            capture_output=True,
            text=True,
            check=False,
        )
        if result.returncode != 0:
            file_path.write_text(original, encoding="utf-8")
            budget["patch_failures"] = int(budget.get("patch_failures", 0)) + 1
            return False, []
        runtime_env = dict(os.environ)
        runtime_env["OPENCLAW_SKIP_EVOLUTION"] = "1"
        runtime_result = subprocess.run(
            [sys.executable, "main.py"],
            cwd=str(base_dir),
            capture_output=True,
            text=True,
            check=False,
            env=runtime_env,
        )
        if runtime_result.returncode != 0:
            file_path.write_text(original, encoding="utf-8")
            budget["patch_failures"] = int(budget.get("patch_failures", 0)) + 1
            return False, []
```

```
    except Exception:
        file_path.write_text(original, encoding="utf-8")
        budget["patch_failures"] = int(budget.get("patch_failures", 0)) + 1
        return False, []

    budget["last_patch_step"] = step_count
    applied = {
        "at": now_iso(),
        "step": step_count,
        "target_file": proposal["target_file"],
        "setting": proposal["setting"],
        "old": proposal["old"],
        "new": proposal["new"],
        "reason": proposal["reason"],
        "baseline_health": float(working.get("last_health", 0.0)),
        "benefit_checked": False,
    }
    return True, [applied]


def update_no_benefit_patch_state(
    working: dict[str, Any],
    selfcheck_data: dict[str, Any],
    health_report: dict[str, Any],
) -> None:
    budget = working.setdefault("evolution_budget", {})
    current_step = int(working.get("step_count", 0))
    no_benefit = int(budget.get("no_benefit_patch_count", 0))
    for patch in selfcheck_data.get("applied_patches", []):
        if patch.get("benefit_checked"):
            continue
        if current_step - int(patch.get("step", current_step)) < 5:
            continue
        baseline = float(patch.get("baseline_health", 0.0))
        improved = float(health_report.get("health", 0.0)) > baseline + 0.01
        patch["benefit_checked"] = True
        patch["benefit_health"] = float(health_report.get("health", 0.0))
        patch["benefit"] = improved
        if improved:
            no_benefit = 0
        else:
            no_benefit += 1
    budget["no_benefit_patch_count"] = no_benefit
    if no_benefit >= 3:
        budget["auto_patch_disabled"] = True


```

```
def store_selfcheck(
    selfcheck_data: dict[str, Any],
    report: dict[str, Any],
    health_report: dict[str, Any],
    suggestions: list[str],
    proposals: list[dict[str, Any]],
    applied_patches: list[dict[str, Any]],
    max_items: int = 30,
) -> dict[str, Any]:
    checks = selfcheck_data.setdefault("checks", [])
    evaluations = selfcheck_data.setdefault("evaluations", [])
    optimizations = selfcheck_data.setdefault("optimizations", [])
    patch_proposals = selfcheck_data.setdefault("patch_proposals", [])
    applied = selfcheck_data.setdefault("applied_patches", [])
    checks.append(report)
    evaluations.append(health_report)
    if suggestions:
        optimizations.append({"at": now_iso(), "items": suggestions})
    if proposals:
        patch_proposals.append({"at": now_iso(), "items": proposals})
    if applied_patches:
        applied.extend(applied_patches)
    selfcheck_data["checks"] = checks[-max_items:]
    selfcheck_data["evaluations"] = evaluations[-max_items:]
    selfcheck_data["optimizations"] = optimizations[-max_items:]
    selfcheck_data["patch_proposals"] = patch_proposals[-max_items:]
    selfcheck_data["applied_patches"] = applied[-max_items:]
    selfcheck_data["last_run"] = now_iso()
    return selfcheck_data
```

A.13　memory/working.json

作用说明

当前工作状态，包含 step、mode、goal、last query、last result keys 等。

完整内容

```
{
  "goal": "Keep OpenClaw stable, compact, and operational in its canonical home D:\\openclaw
    .",
  "current_step": "Review results and continue with the next minimal improvement",
  "next_action": "Review results and continue with the next minimal improvement",
  "focus": "operational stability and memory quality",
  "step_history": [
    "review results and continue with the",
```

```
    "review results and continue with the",
    "review results and continue with the",
    "review results and continue with the",
    "review results and continue with the",
    "review results and continue with the",
    "review results and continue with the",
    "review results and continue with the",
    "review results and continue with the",
    "review results and continue with the",
    "review results and continue with the",
    "review results and continue with the",
    "review results and continue with the",
    "review results and continue with the",
    "review results and continue with the",
    "review results and continue with the",
    "review results and continue with the",
    "review results and continue with the",
    "review results and continue with the",
    "review results and continue with the",
    "review results and continue with the"
  ],
  "step_count": 43,
  "last_action": "Review results and continue with the next minimal improvement",
  "retrieval_cooldown_until": 31,
  "knowledge_write_cooldown_until": 17,
  "last_partition": "programming",
  "last_strategy": "decompose_task",
  "last_query": "Keep OpenClaw stable, compact, and operational in its canonical home D:\\
    openclaw. operational stability and memory quality Review results and continue with the
    next minimal improvement",
  "last_result_keys": [
    "socket-mouse-bridge-host",
    "xiaohongshu-mcp-troubleshooting-order"
  ],
  "recent_suggestion_steps": [],
  "mode": "normal",
  "last_health": 0.817,
  "evolution_budget": {
    "max_suggestions_per_10_steps": 2,
    "max_patches_per_20_steps": 1,
    "disable_patch_after_failures": 2,
    "patch_failures": 0,
    "last_patch_step": 20,
    "no_benefit_patch_count": 0,
    "auto_patch_disabled": false
  }
}
```

A.14　memory/summary.json

作用说明

长期摘要，包含 facts、decisions、progress、failures、anti-patterns。

完整内容


```
{
  "facts": [
    "OpenClaw runs in WSL and primarily operates on D:\\openclaw.",
    "Windows paths should be handled via their WSL mappings rather than treated as inaccessible.",
    "Mouse control uses the socket-based windows-bridge flow with host 172.18.160.1.",
    "WhatsApp cannot be tested by sending messages from the same logged-in number to itself.",
    "ChatGPT and other logged-in pages are more reliable in a real browser than in headless mode.",
    "The user prefers direct execution, concise explanations, and real verification output."
  ],
  "decisions": [
    "Keep long-lived reusable assets in D:\\openclaw and keep transient workspace files elsewhere when possible.",
    "Do not configure tray, mouse bridge, or related scripts to auto-start on Windows.",
    "Treat D:\\openclaw\\selfstart as a manual-start directory, not a startup directory.",
    "Use the fixed Xiaohongshu MCP troubleshooting order before trying extra background checks.",
    "Use backend or non-visible page reads before opening a visible browser for Bilibili-style content retrieval.",
    "If startup issues reappear, check logs and system startup sources before changing scripts."
  ],
  "failures": [
    "Startup behavior was previously implemented in the wrong direction by treating selfstart and helper scripts as an auto-start chain.",
    "WhatsApp reliability issues can come from expired web sessions, relogin mismatch, or testing the wrong inbound path.",
    "MiniMax highspeed was observed as an unstable fallback choice."
  ],
  "progress": [
    "Initialized OpenClaw and installed core skills between 2026-03-09 and 2026-03-10.",
    "Configured Gmail, GitHub CLI, and the daily briefing pipeline on 2026-03-13.",
    "Moved durable assets to D:\\openclaw and switched mouse control to the socket bridge on 2026-03-14.",
    "Recovered Xiaohongshu MCP, registered local Ollama models, and fixed briefing delivery on 2026-03-15.",
    "Cleaned caches and clarified the no-auto-start boundary on 2026-03-17 and 2026-03-18.",
```

```
    "Completed step: No active legacy task. Keep the system stable and review startup behavior only when needed.",
    "Completed step: If startup issues reappear, check logs and system startup sources before changing scripts."
  ],
  "anti_patterns": [
    "Do not rebuild Windows auto-start chains unless the user explicitly authorizes it.",
    "Do not keep raw logs or cache inventories as long-term memory when a concise conclusion is enough.",
    "Do not assume Startup folder entries are the only possible source of Windows auto-start behavior."
  ]
}
```


A.15 memory/strategy.json

作用说明

可复用策略及其使用统计。

完整内容


```
{
  "strategies": [
    {
      "name": "direct_execution",
      "description": "Do the simplest valid next action directly when the next step is obvious.",
      "score": 3.0,
      "usage": 22,
      "last_used_step": 41,
      "when_to_use": [
        "obvious_next_step",
        "low_uncertainty"
      ]
    },
    {
      "name": "decompose_task",
      "description": "Break larger tasks into smaller steps when uncertainty or scope grows.",
      "score": 2.5000000000000013,
      "usage": 17,
      "last_used_step": 42,
      "when_to_use": [
        "complex_task",
        "high_uncertainty"
```

```
      ]
    },
    {
      "name": "fallback_simplify",
      "description": "Reduce complexity after repeated failure or noisy context.",
      "score": 0.9,
      "usage": 1,
      "last_used_step": 0,
      "when_to_use": [
        "repetition",
        "failure_recovery"
      ]
    },
    {
      "name": "fixed_order_troubleshooting",
      "description": "Follow the known troubleshooting order before trying new branches.",
      "score": 0.95,
      "usage": 0,
      "last_used_step": 0,
      "when_to_use": [
        "service_recovery",
        "known_runbook"
      ]
    },
    {
      "name": "clean_regenerable_first",
      "description": "Delete regenerable cache and failed queue data before touching live state.",
      "score": 0.9,
      "usage": 0,
      "last_used_step": 0,
      "when_to_use": [
        "cleanup",
        "disk_pressure"
      ]
    }
  ]
}
```


A.16　memory/reflection.json

作用说明

简短、可执行的 reflection 列表。

完整内容


```
{
  "insights": [
    "Treat selfstart as manual-only unless the user explicitly asks for Windows auto-start.",
    "When startup behavior looks wrong, inspect system startup sources before editing helper scripts again.",
    "Delete regenerable cache before touching live session or browser state.",
    "Use a fixed troubleshooting order for Xiaohongshu MCP instead of adding extra background checks.",
    "Simplify sooner after a failure instead of repeating the same approach."
  ]
}
```


A.17　memory/selfcheck.json

作用说明

当前 health 历史、自检记录、建议和 patch 审计。

完整内容


```
{
  "checks": [
    {
      "memory_pressure": 0.01,
      "duplicate_ratio": 0.0,
      "low_importance_ratio": 0.0,
      "strategy_skew": 0.714,
      "retrieval_effectiveness": 0.5,
      "complexity_score": 0.18,
      "disk_usage": 2371
    },
    {
      "memory_pressure": 0.02,
      "duplicate_ratio": 0.0,
      "low_importance_ratio": 0.0,
      "strategy_skew": 0.625,
      "retrieval_effectiveness": 1.0,
      "complexity_score": 0.22,
      "disk_usage": 3185
    },
    {
      "memory_pressure": 0.02,
      "duplicate_ratio": 0.0,
```

```
      "low_importance_ratio": 0.0,
      "strategy_skew": 0.667,
      "retrieval_effectiveness": 0.5,
      "complexity_score": 0.24,
      "disk_usage": 3533
    },
    {
      "memory_pressure": 0.02,
      "duplicate_ratio": 0.0,
      "low_importance_ratio": 0.0,
      "strategy_skew": 0.6,
      "retrieval_effectiveness": 0.5,
      "complexity_score": 0.26,
      "disk_usage": 3764
    },
    {
      "memory_pressure": 0.02,
      "duplicate_ratio": 0.0,
      "low_importance_ratio": 0.0,
      "strategy_skew": 0.636,
      "retrieval_effectiveness": 1.0,
      "complexity_score": 0.28,
      "disk_usage": 3997
    },
    {
      "memory_pressure": 0.15,
      "duplicate_ratio": 0.0,
      "low_importance_ratio": 0.0,
      "strategy_skew": 0.7,
      "retrieval_effectiveness": 1.0,
      "disk_usage": 11288,
      "complexity_score": 0.217
    },
    {
      "memory_pressure": 0.15,
      "duplicate_ratio": 0.0,
      "low_importance_ratio": 0.0,
      "strategy_skew": 0.727,
      "retrieval_effectiveness": 0.5,
      "disk_usage": 11474,
      "complexity_score": 0.233
    },
    {
      "memory_pressure": 0.16,
      "duplicate_ratio": 0.0,
      "low_importance_ratio": 0.0,
      "strategy_skew": 0.667,
```

```
      "retrieval_effectiveness": 0.5,
      "disk_usage": 12251,
      "complexity_score": 0.267
    },
    {
      "memory_pressure": 0.16,
      "duplicate_ratio": 0.0,
      "low_importance_ratio": 0.0,
      "strategy_skew": 0.692,
      "retrieval_effectiveness": 0.5,
      "disk_usage": 12423,
      "complexity_score": 0.283
    },
    {
      "memory_pressure": 0.16,
      "duplicate_ratio": 0.0,
      "low_importance_ratio": 0.0,
      "strategy_skew": 0.643,
      "retrieval_effectiveness": 1.0,
      "disk_usage": 12669,
      "complexity_score": 0.3
    },
    {
      "memory_pressure": 0.16,
      "duplicate_ratio": 0.0,
      "low_importance_ratio": 0.0,
      "strategy_skew": 0.667,
      "retrieval_effectiveness": 0.5,
      "disk_usage": 12816,
      "complexity_score": 0.317
    },
    {
      "memory_pressure": 0.16,
      "duplicate_ratio": 0.0,
      "low_importance_ratio": 0.0,
      "strategy_skew": 0.625,
      "retrieval_effectiveness": 0.5,
      "disk_usage": 13047,
      "complexity_score": 0.333
    },
    {
      "memory_pressure": 0.16,
      "duplicate_ratio": 0.0,
      "low_importance_ratio": 0.0,
      "strategy_skew": 0.647,
      "retrieval_effectiveness": 0.5,
      "disk_usage": 13193,
```

```
      "complexity_score": 0.35
    },
    {
      "memory_pressure": 0.16,
      "duplicate_ratio": 0.0,
      "low_importance_ratio": 0.0,
      "strategy_skew": 0.611,
      "retrieval_effectiveness": 1.0,
      "disk_usage": 13424,
      "complexity_score": 0.367
    },
    {
      "memory_pressure": 0.16,
      "duplicate_ratio": 0.0,
      "low_importance_ratio": 0.0,
      "strategy_skew": 0.6,
      "retrieval_effectiveness": 0.5,
      "disk_usage": 12230,
      "complexity_score": 0.267
    },
    {
      "memory_pressure": 0.1,
      "duplicate_ratio": 0.0,
      "low_importance_ratio": 0.0,
      "strategy_skew": 0.619,
      "retrieval_effectiveness": 0.5,
      "disk_usage": 9440,
      "complexity_score": 0.267
    },
    {
      "memory_pressure": 0.09,
      "duplicate_ratio": 0.0,
      "low_importance_ratio": 0.0,
      "strategy_skew": 0.591,
      "retrieval_effectiveness": 1.0,
      "disk_usage": 8949,
      "complexity_score": 0.267
    },
    {
      "memory_pressure": 0.09,
      "duplicate_ratio": 0.0,
      "low_importance_ratio": 0.0,
      "strategy_skew": 0.609,
      "retrieval_effectiveness": 0.5,
      "disk_usage": 8948,
      "complexity_score": 0.267
    },
```

```
    {
      "memory_pressure": 0.09,
      "duplicate_ratio": 0.0,
      "low_importance_ratio": 0.0,
      "strategy_skew": 0.583,
      "retrieval_effectiveness": 0.5,
      "disk_usage": 8776,
      "complexity_score": 0.267
    },
    {
      "memory_pressure": 0.08,
      "duplicate_ratio": 0.0,
      "low_importance_ratio": 0.0,
      "strategy_skew": 0.6,
      "retrieval_effectiveness": 0.5,
      "disk_usage": 8097,
      "complexity_score": 0.25
    },
    {
      "memory_pressure": 0.08,
      "duplicate_ratio": 0.0,
      "low_importance_ratio": 0.0,
      "strategy_skew": 0.593,
      "retrieval_effectiveness": 1.0,
      "disk_usage": 8097,
      "complexity_score": 0.25
    },
    {
      "memory_pressure": 0.08,
      "duplicate_ratio": 0.0,
      "low_importance_ratio": 0.0,
      "strategy_skew": 0.562,
      "retrieval_effectiveness": 1.0,
      "disk_usage": 8096,
      "complexity_score": 0.25
    },
    {
      "memory_pressure": 0.08,
      "duplicate_ratio": 0.0,
      "low_importance_ratio": 0.0,
      "strategy_skew": 0.568,
      "retrieval_effectiveness": 1.0,
      "disk_usage": 8099,
      "complexity_score": 0.25
    }
  ],
  "evaluations": [
```

```
    {
      "health": 0.854,
      "delta": -0.015,
      "pressure": 0.01,
      "noise": 0.0,
      "complexity": 0.18
    },
    {
      "health": 0.857,
      "delta": 0.003,
      "pressure": 0.02,
      "noise": 0.0,
      "complexity": 0.22
    },
    {
      "health": 0.847,
      "delta": -0.01,
      "pressure": 0.02,
      "noise": 0.0,
      "complexity": 0.24
    },
    {
      "health": 0.853,
      "delta": 0.006,
      "pressure": 0.02,
      "noise": 0.0,
      "complexity": 0.26
    },
    {
      "health": 0.844,
      "delta": -0.009,
      "pressure": 0.02,
      "noise": 0.0,
      "complexity": 0.28
    },
    {
      "health": 0.769,
      "delta": -0.075,
      "pressure": 0.15,
      "noise": 0.0,
      "complexity": 0.217
    },
    {
      "health": 0.711,
      "delta": -0.058,
      "pressure": 0.15,
      "noise": 0.0,
```

```
      "complexity": 0.233
    },
    {
      "health": 0.709,
      "delta": -0.002,
      "pressure": 0.16,
      "noise": 0.0,
      "complexity": 0.267
    },
    {
      "health": 0.702,
      "delta": -0.007,
      "pressure": 0.16,
      "noise": 0.0,
      "complexity": 0.283
    },
    {
      "health": 0.755,
      "delta": 0.053,
      "pressure": 0.16,
      "noise": 0.0,
      "complexity": 0.3
    },
    {
      "health": 0.698,
      "delta": -0.057,
      "pressure": 0.16,
      "noise": 0.0,
      "complexity": 0.317
    },
    {
      "health": 0.701,
      "delta": 0.003,
      "pressure": 0.16,
      "noise": 0.0,
      "complexity": 0.333
    },
    {
      "health": 0.694,
      "delta": -0.007,
      "pressure": 0.16,
      "noise": 0.0,
      "complexity": 0.35
    },
    {
      "health": 0.746,
      "delta": 0.052,
```

```
      "pressure": 0.16,
      "noise": 0.0,
      "complexity": 0.367
    },
    {
      "health": 0.719,
      "delta": -0.027,
      "pressure": 0.16,
      "noise": 0.0,
      "complexity": 0.267
    },
    {
      "health": 0.745,
      "delta": 0.026,
      "pressure": 0.1,
      "noise": 0.0,
      "complexity": 0.267
    },
    {
      "health": 0.804,
      "delta": 0.059,
      "pressure": 0.09,
      "noise": 0.0,
      "complexity": 0.267
    },
    {
      "health": 0.751,
      "delta": -0.053,
      "pressure": 0.09,
      "noise": 0.0,
      "complexity": 0.267
    },
    {
      "health": 0.756,
      "delta": 0.005,
      "pressure": 0.09,
      "noise": 0.0,
      "complexity": 0.267
    },
    {
      "health": 0.762,
      "delta": 0.006,
      "pressure": 0.08,
      "noise": 0.0,
      "complexity": 0.25
    },
    {
```

```
      "health": 0.813,
      "delta": 0.051,
      "pressure": 0.08,
      "noise": 0.0,
      "complexity": 0.25
    },
    {
      "health": 0.818,
      "delta": 0.005,
      "pressure": 0.08,
      "noise": 0.0,
      "complexity": 0.25
    },
    {
      "health": 0.817,
      "delta": -0.001,
      "pressure": 0.08,
      "noise": 0.0,
      "complexity": 0.25
    }
  ],
  "optimizations": [
    {
      "at": "2026-03-18T02:40:46+00:00",
      "items": [
        "If startup behavior reappears, verify logs and system startup sources before changing scripts.",
        "Prefer concise audit conclusions over storing raw log streams."
      ]
    },
    {
      "at": "2026-03-18T02:48:55+00:00",
      "items": [
        "reduce retrieval frequency"
      ]
    },
    {
      "at": "2026-03-18T02:48:55+00:00",
      "items": [
        "reduce retrieval frequency"
      ]
    },
    {
      "at": "2026-03-18T02:48:56+00:00",
      "items": [
        "reduce retrieval frequency"
      ]
```

```
    },
    {
      "at": "2026-03-18T02:52:59+00:00",
      "items": [
        "reduce retrieval frequency"
      ]
    },
    {
      "at": "2026-03-18T02:53:09+00:00",
      "items": [
        "reduce retrieval frequency"
      ]
    },
    {
      "at": "2026-03-18T02:54:41+00:00",
      "items": [
        "reduce retrieval frequency"
      ]
    },
    {
      "at": "2026-03-18T02:55:36+00:00",
      "items": [
        "reduce retrieval frequency"
      ]
    },
    {
      "at": "2026-03-18T02:55:57+00:00",
      "items": [
        "reduce retrieval frequency"
      ]
    }
  ],
  "patch_proposals": [
    {
      "at": "2026-03-18T02:48:55+00:00",
      "items": [
        {
          "type": "threshold_change",
          "target_file": "core/retriever.py",
          "setting": "EARLY_RETURN_SCORE",
          "old": 5.5,
          "new": 5.2,
          "reason": "allow stronger reuse under stability mode"
        }
      ]
    },
    {
```

```
      "at": "2026-03-18T02:48:55+00:00",
      "items": [
        {
          "type": "threshold_change",
          "target_file": "core/retriever.py",
          "setting": "EARLY_RETURN_SCORE",
          "old": 5.5,
          "new": 5.2,
          "reason": "allow stronger reuse under stability mode"
        }
      ]
    },
    {
      "at": "2026-03-18T02:48:56+00:00",
      "items": [
        {
          "type": "threshold_change",
          "target_file": "core/retriever.py",
          "setting": "EARLY_RETURN_SCORE",
          "old": 5.5,
          "new": 5.2,
          "reason": "allow stronger reuse under stability mode"
        }
      ]
    }
  ],
  "applied_patches": [
    {
      "at": "2026-03-18T02:48:56+00:00",
      "target_file": "core/retriever.py",
      "setting": "EARLY_RETURN_SCORE",
      "old": 5.5,
      "new": 5.2,
      "reason": "allow stronger reuse under stability mode"
    }
  ],
  "last_run": "2026-03-18T03:35:55+00:00"
}
```


A.18　memory/runtime.json

作用说明

每次执行后写入的紧凑运行快照。

完整内容


```
{
  "records": [
    {
      "step": 35,
      "mode": "normal",
      "partition": "programming",
      "strategy": "decompose_task",
      "health": 0.818,
      "knowledge_count": 8,
      "summary_count": 25,
      "reflection_count": 5,
      "selfcheck_triggered": true,
      "cleaner_triggered": false,
      "timestamp": "2026-03-18T03:09:57+00:00"
    },
    {
      "step": 36,
      "mode": "normal",
      "partition": "programming",
      "strategy": "direct_execution",
      "health": 0.818,
      "knowledge_count": 8,
      "summary_count": 25,
      "reflection_count": 5,
      "selfcheck_triggered": false,
      "cleaner_triggered": false,
      "timestamp": "2026-03-18T03:10:57+00:00"
    },
    {
      "step": 37,
      "mode": "normal",
      "partition": "programming",
      "strategy": "decompose_task",
      "health": 0.818,
      "knowledge_count": 8,
      "summary_count": 25,
      "reflection_count": 5,
      "selfcheck_triggered": false,
      "cleaner_triggered": false,
      "timestamp": "2026-03-18T03:10:57+00:00"
    },
    {
      "step": 38,
      "mode": "normal",
      "partition": "programming",
```

```
      "strategy": "direct_execution",
      "health": 0.818,
      "knowledge_count": 8,
      "summary_count": 25,
      "reflection_count": 5,
      "selfcheck_triggered": false,
      "cleaner_triggered": false,
      "timestamp": "2026-03-18T03:27:12+00:00"
    },
    {
      "step": 39,
      "mode": "normal",
      "partition": "programming",
      "strategy": "decompose_task",
      "health": 0.818,
      "knowledge_count": 8,
      "summary_count": 25,
      "reflection_count": 5,
      "selfcheck_triggered": false,
      "cleaner_triggered": false,
      "timestamp": "2026-03-18T03:34:43+00:00"
    },
    {
      "step": 40,
      "mode": "normal",
      "partition": "programming",
      "strategy": "direct_execution",
      "health": 0.817,
      "knowledge_count": 8,
      "summary_count": 25,
      "reflection_count": 5,
      "selfcheck_triggered": true,
      "cleaner_triggered": false,
      "timestamp": "2026-03-18T03:35:55+00:00"
    },
    {
      "step": 41,
      "mode": "normal",
      "partition": "programming",
      "strategy": "decompose_task",
      "health": 0.817,
      "knowledge_count": 8,
      "summary_count": 25,
      "reflection_count": 5,
      "selfcheck_triggered": false,
      "cleaner_triggered": false,
      "timestamp": "2026-03-18T03:37:45+00:00"
```

```
    },
    {
      "step": 42,
      "mode": "normal",
      "partition": "programming",
      "strategy": "direct_execution",
      "health": 0.817,
      "knowledge_count": 8,
      "summary_count": 25,
      "reflection_count": 5,
      "selfcheck_triggered": false,
      "cleaner_triggered": false,
      "timestamp": "2026-03-18T03:37:45+00:00"
    },
    {
      "step": 43,
      "mode": "normal",
      "partition": "programming",
      "strategy": "decompose_task",
      "health": 0.817,
      "knowledge_count": 8,
      "summary_count": 25,
      "reflection_count": 5,
      "selfcheck_triggered": false,
      "cleaner_triggered": false,
      "timestamp": "2026-03-18T03:48:39+00:00"
    }
  ]
}
```


A.19　memory/meta.json

作用说明

稳定规则、模式、约束、评估规则。

完整内容


```
{
  "rules": [
    {
      "rule": "Only store high-value reusable knowledge",
      "confidence": 1.0,
      "last_validated_step": 12
    },
    {
```

```
      "rule": "Prefer direct execution with real verification output",
      "confidence": 1.0,
      "last_validated_step": 12
    },
    {
      "rule": "Do not configure Windows auto-start without explicit user approval",
      "confidence": 1.0,
      "last_validated_step": 12
    },
    {
      "rule": "Treat Windows paths as accessible through WSL path mapping",
      "confidence": 1.0,
      "last_validated_step": 12
    },
    {
      "rule": "Keep durable assets on D:\\openclaw when possible",
      "confidence": 1.0,
      "last_validated_step": 12
    },
    {
      "rule": "Treat D:\\openclaw as the canonical OpenClaw home and workspace root",
      "confidence": 1.0,
      "last_validated_step": 40
    }
  ],
  "patterns": [
    {
      "pattern": "Check known startup sources before editing launch scripts again",
      "confidence": 1.0,
      "last_validated_step": 12
    },
    {
      "pattern": "Use fixed troubleshooting order for Xiaohongshu MCP",
      "confidence": 1.0,
      "last_validated_step": 12
    },
    {
      "pattern": "Delete regenerable cache before touching live browser or session state",
      "confidence": 0.95,
      "last_validated_step": 12
    },
    {
      "pattern": "Prefer visible browser sessions for logged-in sites when headless reads fail",
      "confidence": 0.95,
      "last_validated_step": 12
    }
```

```
  ],
  "constraints": [
    "Keep memory small",
    "Keep retrieval fast",
    "Keep disk usage low",
    "Avoid memory pollution from raw logs",
    "Prefer free and local tools when practical"
  ],
  "evaluation_rules": [
    "If startup-related noise returns, preserve only the conclusion and not the full transcript",
    "If memory contains repeated workflow notes, merge them into one reusable rule",
    "If a fact is tied to one domain, store it in exactly one partition",
    "If a workflow is fixed and repeatable, prefer strategy or meta over summary sprawl"
  ]
}
```

A.20　memory/knowledge/programming.json

作用说明

编程和系统操作相关知识。

完整内容

```
{
  "items": [
    {
      "content": "The socket mouse bridge host is 172.18.160.1 and the bridge must run in the user desktop session, not Session 0.",
      "tags": [
        "mouse-bridge",
        "socket",
        "windows-gui"
      ],
      "importance": 1.0,
      "created_at": "2026-03-18T02:41:12+00:00",
      "last_accessed_step": 42,
      "access_count": 18,
      "tokens": [
        "socket",
        "mouse",
        "bridge",
        "host",
        "172",
        "160",
```

```
        "bridge",
        "must"
      ],
      "key": "socket-mouse-bridge-host"
    },
    {
      "content": "Xiaohongshu MCP troubleshooting order: check mcporter status, ensure
    Docker Desktop is running, start the container,...",
      "tags": [
        "xiaohongshu",
        "mcp",
        "docker"
      ],
      "importance": 1.0,
      "created_at": "2026-03-18T02:41:12+00:00",
      "last_accessed_step": 42,
      "access_count": 18,
      "tokens": [
        "xiaohongshu",
        "mcp",
        "troubleshooting",
        "order",
        "check",
        "mcporter",
        "status",
        "ensure"
      ],
      "key": "xiaohongshu-mcp-troubleshooting-order"
    }
  ],
  "index": {
    "socket": [
      0
    ],
    "mouse": [
      0
    ],
    "bridge": [
      0
    ],
    "host": [
      0
    ],
    "172": [
      0
    ],
    "xiaohongshu": [
```

```
      1
    ],
    "mcp": [
      1
    ],
    "troubleshooting": [
      1
    ],
    "order": [
      1
    ],
    "check": [
      1
    ]
  }
}
```

A.21　memory/knowledge/general.json

作用说明

一般知识、偏好和非编程规则。

完整内容

```
{
  "items": [
    {
      "content": "The user prefers direct execution, concise explanations, and real
    verification output over ceremony.",
      "tags": [
        "user-preferences",
        "communication",
        "verification"
      ],
      "importance": 0.95,
      "created_at": "2026-03-18T02:41:12+00:00",
      "last_accessed_step": 12,
      "access_count": 0,
      "tokens": [
        "user",
        "prefers",
        "direct",
        "execution",
        "concise",
        "explanations",
```

```
        "real",
        "verification"
      ],
      "key": "user-prefers-direct-execution"
    },
    {
      "content": "Use Edge by default unless the target page is already clearly open in Chrome.",
      "tags": [
        "browser-preference",
        "edge",
        "chrome"
      ],
      "importance": 0.8,
      "created_at": "2026-03-18T02:41:12+00:00",
      "last_accessed_step": 12,
      "access_count": 0,
      "tokens": [
        "use",
        "edge",
        "default",
        "unless",
        "target",
        "page",
        "already",
        "clearly"
      ],
      "key": "use-edge-default-unless"
    },
    {
      "content": "WhatsApp inbound tests require another account; the logged-in number cannot test itself by messaging itself.",
      "tags": [
        "whatsapp",
        "testing",
        "inbound"
      ],
      "importance": 0.9,
      "created_at": "2026-03-18T02:41:12+00:00",
      "last_accessed_step": 12,
      "access_count": 0,
      "tokens": [
        "whatsapp",
        "inbound",
        "tests",
        "require",
        "another",
```

```
        "account",
        "logged",
        "number"
      ],
      "key": "whatsapp-inbound-tests-require"
    },
    {
      "content": "Gmail reminders should focus on account-security mail rather than generic OTP messages.",
      "tags": [
        "gmail",
        "security-mail",
        "notifications"
      ],
      "importance": 0.8,
      "created_at": "2026-03-18T02:41:12+00:00",
      "last_accessed_step": 12,
      "access_count": 0,
      "tokens": [
        "gmail",
        "reminders",
        "should",
        "focus",
        "account",
        "security",
        "mail",
        "rather"
      ],
      "key": "gmail-reminders-should-focus"
    },
    {
      "content": "Daily briefing generation runs at 08:00 Shanghai time and delivery should use announce mode.",
      "tags": [
        "daily-briefing",
        "schedule",
        "announce"
      ],
      "importance": 0.75,
      "created_at": "2026-03-18T02:41:12+00:00",
      "last_accessed_step": 12,
      "access_count": 0,
      "tokens": [
        "daily",
        "briefing",
        "generation",
        "runs",
```

```
        "shanghai",
        "time",
        "delivery",
        "should"
      ],
      "key": "daily-briefing-generation-runs"
    },
    {
      "content": "Windows startup behavior should be investigated through Startup sources,
    Run keys, and scheduled tasks before changin...",
      "tags": [
        "windows-startup",
        "troubleshooting",
        "boundary"
      ],
      "importance": 0.9,
      "created_at": "2026-03-18T02:41:12+00:00",
      "last_accessed_step": 12,
      "access_count": 0,
      "tokens": [
        "windows",
        "startup",
        "behavior",
        "should",
        "investigated",
        "through",
        "startup",
        "sources"
      ],
      "key": "windows-startup-behavior-should"
    }
  ],
  "index": {
    "user": [
      0
    ],
    "prefers": [
      0
    ],
    "direct": [
      0
    ],
    "execution": [
      0
    ],
    "concise": [
      0
```

```
    ],
    "use": [
      1
    ],
    "edge": [
      1
    ],
    "default": [
      1
    ],
    "unless": [
      1
    ],
    "target": [
      1
    ],
    "whatsapp": [
      2
    ],
    "inbound": [
      2
    ],
    "tests": [
      2
    ],
    "require": [
      2
    ],
    "another": [
      2
    ],
    "gmail": [
      3
    ],
    "reminders": [
      3
    ],
    "should": [
      3,
      5
    ],
    "focus": [
      3
    ],
    "account": [
      3
    ],
```

```
    "daily": [
      4
    ],
    "briefing": [
      4
    ],
    "generation": [
      4
    ],
    "runs": [
      4
    ],
    "shanghai": [
      4
    ],
    "windows": [
      5
    ],
    "startup": [
      5
    ],
    "behavior": [
      5
    ],
    "investigated": [
      5
    ]
  }
}
```


A.22　memory/knowledge/science.json

作用说明

科学分区当前快照。

完整内容


```
{
  "items": [],
  "index": {}
}
```

A.23　memory/knowledge/finance.json

作用说明

金融分区当前快照。

完整内容

```
{
  "items": [],
  "index": {}
}
```

第 B 章　WSL 侧完整文件快照

B.1　bin/openclaw

作用说明

这是 WSL 里用户最终调用到的 `openclaw` 命令包装器。

完整内容

```
#!/usr/bin/env bash
set -euo pipefail
exec node "/home/park/openclaw/dist/entry.js" "$@"
```

B.2　package.json

作用说明

定义 OpenClaw CLI 包身份、bin 映射、构建脚本和测试脚本。

完整内容

```
{
  "name": "openclaw",
  "version": "2026.3.9",
  "description": "Multi-channel AI gateway with extensible messaging integrations",
  "keywords": [],
  "homepage": "https://github.com/openclaw/openclaw#readme",
  "bugs": {
    "url": "https://github.com/openclaw/openclaw/issues"
  },
  "license": "MIT",
  "author": "",
  "repository": {
    "type": "git",
    "url": "git+https://github.com/openclaw/openclaw.git"
  },
  "bin": {
```

```
    "openclaw": "openclaw.mjs"
  },
  "directories": {
    "doc": "docs",
    "test": "test"
  },
  "files": [
    "CHANGELOG.md",
    "LICENSE",
    "openclaw.mjs",
    "README-header.png",
    "README.md",
    "assets/",
    "dist/",
    "docs/",
    "extensions/",
    "skills/"
  ],
  "type": "module",
  "main": "dist/index.js",
  "exports": {
    ".": "./dist/index.js",
    "./plugin-sdk": {
      "types": "./dist/plugin-sdk/index.d.ts",
      "default": "./dist/plugin-sdk/index.js"
    },
    "./plugin-sdk/core": {
      "types": "./dist/plugin-sdk/core.d.ts",
      "default": "./dist/plugin-sdk/core.js"
    },
    "./plugin-sdk/compat": {
      "types": "./dist/plugin-sdk/compat.d.ts",
      "default": "./dist/plugin-sdk/compat.js"
    },
    "./plugin-sdk/telegram": {
      "types": "./dist/plugin-sdk/telegram.d.ts",
      "default": "./dist/plugin-sdk/telegram.js"
    },
    "./plugin-sdk/discord": {
      "types": "./dist/plugin-sdk/discord.d.ts",
      "default": "./dist/plugin-sdk/discord.js"
    },
    "./plugin-sdk/slack": {
      "types": "./dist/plugin-sdk/slack.d.ts",
      "default": "./dist/plugin-sdk/slack.js"
    },
    "./plugin-sdk/signal": {
```

```
      "types": "./dist/plugin-sdk/signal.d.ts",
      "default": "./dist/plugin-sdk/signal.js"
    },
    "./plugin-sdk/imessage": {
      "types": "./dist/plugin-sdk/imessage.d.ts",
      "default": "./dist/plugin-sdk/imessage.js"
    },
    "./plugin-sdk/whatsapp": {
      "types": "./dist/plugin-sdk/whatsapp.d.ts",
      "default": "./dist/plugin-sdk/whatsapp.js"
    },
    "./plugin-sdk/line": {
      "types": "./dist/plugin-sdk/line.d.ts",
      "default": "./dist/plugin-sdk/line.js"
    },
    "./plugin-sdk/msteams": {
      "types": "./dist/plugin-sdk/msteams.d.ts",
      "default": "./dist/plugin-sdk/msteams.js"
    },
    "./plugin-sdk/acpx": {
      "types": "./dist/plugin-sdk/acpx.d.ts",
      "default": "./dist/plugin-sdk/acpx.js"
    },
    "./plugin-sdk/bluebubbles": {
      "types": "./dist/plugin-sdk/bluebubbles.d.ts",
      "default": "./dist/plugin-sdk/bluebubbles.js"
    },
    "./plugin-sdk/copilot-proxy": {
      "types": "./dist/plugin-sdk/copilot-proxy.d.ts",
      "default": "./dist/plugin-sdk/copilot-proxy.js"
    },
    "./plugin-sdk/device-pair": {
      "types": "./dist/plugin-sdk/device-pair.d.ts",
      "default": "./dist/plugin-sdk/device-pair.js"
    },
    "./plugin-sdk/diagnostics-otel": {
      "types": "./dist/plugin-sdk/diagnostics-otel.d.ts",
      "default": "./dist/plugin-sdk/diagnostics-otel.js"
    },
    "./plugin-sdk/diffs": {
      "types": "./dist/plugin-sdk/diffs.d.ts",
      "default": "./dist/plugin-sdk/diffs.js"
    },
    "./plugin-sdk/feishu": {
      "types": "./dist/plugin-sdk/feishu.d.ts",
      "default": "./dist/plugin-sdk/feishu.js"
    },
```

```
    "./plugin-sdk/google-gemini-cli-auth": {
      "types": "./dist/plugin-sdk/google-gemini-cli-auth.d.ts",
      "default": "./dist/plugin-sdk/google-gemini-cli-auth.js"
    },
    "./plugin-sdk/googlechat": {
      "types": "./dist/plugin-sdk/googlechat.d.ts",
      "default": "./dist/plugin-sdk/googlechat.js"
    },
    "./plugin-sdk/irc": {
      "types": "./dist/plugin-sdk/irc.d.ts",
      "default": "./dist/plugin-sdk/irc.js"
    },
    "./plugin-sdk/llm-task": {
      "types": "./dist/plugin-sdk/llm-task.d.ts",
      "default": "./dist/plugin-sdk/llm-task.js"
    },
    "./plugin-sdk/lobster": {
      "types": "./dist/plugin-sdk/lobster.d.ts",
      "default": "./dist/plugin-sdk/lobster.js"
    },
    "./plugin-sdk/matrix": {
      "types": "./dist/plugin-sdk/matrix.d.ts",
      "default": "./dist/plugin-sdk/matrix.js"
    },
    "./plugin-sdk/mattermost": {
      "types": "./dist/plugin-sdk/mattermost.d.ts",
      "default": "./dist/plugin-sdk/mattermost.js"
    },
    "./plugin-sdk/memory-core": {
      "types": "./dist/plugin-sdk/memory-core.d.ts",
      "default": "./dist/plugin-sdk/memory-core.js"
    },
    "./plugin-sdk/memory-lancedb": {
      "types": "./dist/plugin-sdk/memory-lancedb.d.ts",
      "default": "./dist/plugin-sdk/memory-lancedb.js"
    },
    "./plugin-sdk/minimax-portal-auth": {
      "types": "./dist/plugin-sdk/minimax-portal-auth.d.ts",
      "default": "./dist/plugin-sdk/minimax-portal-auth.js"
    },
    "./plugin-sdk/nextcloud-talk": {
      "types": "./dist/plugin-sdk/nextcloud-talk.d.ts",
      "default": "./dist/plugin-sdk/nextcloud-talk.js"
    },
    "./plugin-sdk/nostr": {
      "types": "./dist/plugin-sdk/nostr.d.ts",
      "default": "./dist/plugin-sdk/nostr.js"
```

```
    },
    "./plugin-sdk/open-prose": {
      "types": "./dist/plugin-sdk/open-prose.d.ts",
      "default": "./dist/plugin-sdk/open-prose.js"
    },
    "./plugin-sdk/phone-control": {
      "types": "./dist/plugin-sdk/phone-control.d.ts",
      "default": "./dist/plugin-sdk/phone-control.js"
    },
    "./plugin-sdk/qwen-portal-auth": {
      "types": "./dist/plugin-sdk/qwen-portal-auth.d.ts",
      "default": "./dist/plugin-sdk/qwen-portal-auth.js"
    },
    "./plugin-sdk/synology-chat": {
      "types": "./dist/plugin-sdk/synology-chat.d.ts",
      "default": "./dist/plugin-sdk/synology-chat.js"
    },
    "./plugin-sdk/talk-voice": {
      "types": "./dist/plugin-sdk/talk-voice.d.ts",
      "default": "./dist/plugin-sdk/talk-voice.js"
    },
    "./plugin-sdk/test-utils": {
      "types": "./dist/plugin-sdk/test-utils.d.ts",
      "default": "./dist/plugin-sdk/test-utils.js"
    },
    "./plugin-sdk/thread-ownership": {
      "types": "./dist/plugin-sdk/thread-ownership.d.ts",
      "default": "./dist/plugin-sdk/thread-ownership.js"
    },
    "./plugin-sdk/tlon": {
      "types": "./dist/plugin-sdk/tlon.d.ts",
      "default": "./dist/plugin-sdk/tlon.js"
    },
    "./plugin-sdk/twitch": {
      "types": "./dist/plugin-sdk/twitch.d.ts",
      "default": "./dist/plugin-sdk/twitch.js"
    },
    "./plugin-sdk/voice-call": {
      "types": "./dist/plugin-sdk/voice-call.d.ts",
      "default": "./dist/plugin-sdk/voice-call.js"
    },
    "./plugin-sdk/zalo": {
      "types": "./dist/plugin-sdk/zalo.d.ts",
      "default": "./dist/plugin-sdk/zalo.js"
    },
    "./plugin-sdk/zalouser": {
      "types": "./dist/plugin-sdk/zalouser.d.ts",
```

```
      "default": "./dist/plugin-sdk/zalouser.js"
    },
    "./plugin-sdk/account-id": {
      "types": "./dist/plugin-sdk/account-id.d.ts",
      "default": "./dist/plugin-sdk/account-id.js"
    },
    "./plugin-sdk/keyed-async-queue": {
      "types": "./dist/plugin-sdk/keyed-async-queue.d.ts",
      "default": "./dist/plugin-sdk/keyed-async-queue.js"
    },
    "./cli-entry": "./openclaw.mjs"
  },
  "scripts": {
    "android:assemble": "cd apps/android && ./gradlew :app:assembleDebug",
    "android:format": "cd apps/android && ./gradlew :app:ktlintFormat :benchmark:ktlintFormat",
    "android:install": "cd apps/android && ./gradlew :app:installDebug",
    "android:lint": "cd apps/android && ./gradlew :app:ktlintCheck :benchmark:ktlintCheck",
    "android:lint:android": "cd apps/android && ./gradlew :app:lintDebug",
    "android:run": "cd apps/android && ./gradlew :app:installDebug && adb shell am start -n ai.openclaw.app/.MainActivity",
    "android:test": "cd apps/android && ./gradlew :app:testDebugUnitTest",
    "android:test:integration": "OPENCLAW_LIVE_TEST=1 OPENCLAW_LIVE_ANDROID_NODE=1 vitest run --config vitest.live.config.ts src/gateway/android-node.capabilities.live.test.ts",
    "build": "pnpm canvas:a2ui:bundle && node scripts/tsdown-build.mjs && node scripts/copy-plugin-sdk-root-alias.mjs && pnpm build:plugin-sdk:dts && node --import tsx scripts/write-plugin-sdk-entry-dts.ts && node --import tsx scripts/canvas-a2ui-copy.ts && node --import tsx scripts/copy-hook-metadata.ts && node --import tsx scripts/copy-export-html-templates.ts && node --import tsx scripts/write-build-info.ts && node --import tsx scripts/write-cli-startup-metadata.ts && node --import tsx scripts/write-cli-compat.ts",
    "build:docker": "node scripts/tsdown-build.mjs && node scripts/copy-plugin-sdk-root-alias.mjs && pnpm build:plugin-sdk:dts && node --import tsx scripts/write-plugin-sdk-entry-dts.ts && node --import tsx scripts/canvas-a2ui-copy.ts && node --import tsx scripts/copy-hook-metadata.ts && node --import tsx scripts/copy-export-html-templates.ts && node --import tsx scripts/write-build-info.ts && node --import tsx scripts/write-cli-startup-metadata.ts && node --import tsx scripts/write-cli-compat.ts",
    "build:plugin-sdk:dts": "tsc -p tsconfig.plugin-sdk.dts.json",
    "build:strict-smoke": "pnpm canvas:a2ui:bundle && node scripts/tsdown-build.mjs && node scripts/copy-plugin-sdk-root-alias.mjs && pnpm build:plugin-sdk:dts",
    "canvas:a2ui:bundle": "bash scripts/bundle-a2ui.sh",
    "check": "pnpm check:host-env-policy:swift && pnpm format:check && pnpm tsgo && pnpm lint && pnpm lint:tmp:no-random-messaging && pnpm lint:tmp:channel-agnostic-boundaries && pnpm lint:tmp:no-raw-channel-fetch && pnpm lint:agent:ingress-owner && pnpm lint:plugins:no-register-http-handler && pnpm lint:plugins:no-monolithic-plugin-sdk-entry-imports && pnpm lint:webhook:no-low-level-body-read && pnpm lint:auth:no-pairing-store-group && pnpm lint:auth:pairing-account-scope",
    "check:docs": "pnpm format:docs:check && pnpm lint:docs && pnpm docs:check-links",
```

```
233     "check:host-env-policy:swift": "node scripts/generate-host-env-security-policy-swift.mjs
         --check",
234     "check:loc": "node --import tsx scripts/check-ts-max-loc.ts --max 500",
235     "deadcode:ci": "pnpm deadcode:report:ci:knip",
236     "deadcode:knip": "pnpm dlx knip --config knip.config.ts --isolate-workspaces --
        production --no-progress --reporter compact --files --dependencies",
237     "deadcode:report": "pnpm deadcode:knip; pnpm deadcode:ts-prune; pnpm deadcode:ts-unused
        ",
238     "deadcode:report:ci:knip": "mkdir -p .artifacts/deadcode && pnpm deadcode:knip > .
        artifacts/deadcode/knip.txt 2>&1 || true",
239     "deadcode:report:ci:ts-prune": "mkdir -p .artifacts/deadcode && pnpm deadcode:ts-prune >
         .artifacts/deadcode/ts-prune.txt 2>&1 || true",
240     "deadcode:report:ci:ts-unused": "mkdir -p .artifacts/deadcode && pnpm deadcode:ts-unused
         > .artifacts/deadcode/ts-unused-exports.txt 2>&1 || true",
241     "deadcode:ts-prune": "pnpm dlx ts-prune src extensions scripts",
242     "deadcode:ts-unused": "pnpm dlx ts-unused-exports tsconfig.json --ignoreTestFiles --
        exitWithCount",
243     "dev": "node scripts/run-node.mjs",
244     "docs:bin": "node scripts/build-docs-list.mjs",
245     "docs:check-links": "node scripts/docs-link-audit.mjs",
246     "docs:dev": "cd docs && mint dev",
247     "docs:list": "node scripts/docs-list.js",
248     "docs:spellcheck": "bash scripts/docs-spellcheck.sh",
249     "docs:spellcheck:fix": "bash scripts/docs-spellcheck.sh --write",
250     "dup:check": "jscpd src extensions test scripts --format typescript,javascript --pattern
         \"**/*.{ts,tsx,js,mjs,cjs}\" --gitignore --noSymlinks --ignore \"**/node_modules/**,**/
        dist/**,**/.git/**,**/coverage/**,**/build/**,**/.build/**,**/.artifacts/**\" --min-
        lines 12 --min-tokens 80 --reporters console",
251     "dup:check:json": "jscpd src extensions test scripts --format typescript,javascript --
        pattern \"**/*.{ts,tsx,js,mjs,cjs}\" --gitignore --noSymlinks --ignore \"**/node_modules
        /**,**/dist/**,**/.git/**,**/coverage/**,**/build/**,**/.build/**,**/.artifacts/**\" --
        min-lines 12 --min-tokens 80 --reporters json --output .artifacts/jscpd",
252     "format": "oxfmt --write",
253     "format:all": "pnpm format && pnpm format:swift",
254     "format:check": "oxfmt --check",
255     "format:diff": "oxfmt --write && git --no-pager diff",
256     "format:docs": "git ls-files 'docs/**/*.md' 'docs/**/*.mdx' 'README.md' | xargs oxfmt --
        write",
257     "format:docs:check": "git ls-files 'docs/**/*.md' 'docs/**/*.mdx' 'README.md' | xargs
        oxfmt --check",
258     "format:fix": "oxfmt --write",
259     "format:swift": "swiftformat --lint --config .swiftformat apps/macos/Sources apps/ios/
        Sources apps/shared/OpenClawKit/Sources",
260     "gateway:dev": "OPENCLAW_SKIP_CHANNELS=1 CLAWDBOT_SKIP_CHANNELS=1 node scripts/run-node.
        mjs --dev gateway",
261     "gateway:dev:reset": "OPENCLAW_SKIP_CHANNELS=1 CLAWDBOT_SKIP_CHANNELS=1 node scripts/run
        -node.mjs --dev gateway --reset",
```

```
262     "gateway:watch": "node scripts/watch-node.mjs gateway --force",
263     "gen:host-env-policy:swift": "node scripts/generate-host-env-security-policy-swift.mjs
        --write",
264     "ghsa:patch": "node scripts/ghsa-patch.mjs",
265     "ios:build": "bash -lc './scripts/ios-configure-signing.sh && cd apps/ios && xcodegen
        generate && xcodebuild -project OpenClaw.xcodeproj -scheme OpenClaw -destination \"${
        IOS_DEST:-platform=iOS Simulator,name=iPhone 17}\" -configuration Debug build'",
266     "ios:gen": "bash -lc './scripts/ios-configure-signing.sh && cd apps/ios && xcodegen
        generate'",
267     "ios:open": "bash -lc './scripts/ios-configure-signing.sh && cd apps/ios && xcodegen
        generate && open OpenClaw.xcodeproj'",
268     "ios:run": "bash -lc './scripts/ios-configure-signing.sh && cd apps/ios && xcodegen
        generate && xcodebuild -project OpenClaw.xcodeproj -scheme OpenClaw -destination \"${
        IOS_DEST:-platform=iOS Simulator,name=iPhone 17}\" -configuration Debug build && xcrun
        simctl boot \"${IOS_SIM:-iPhone 17}\" || true && xcrun simctl launch booted ai.openclaw.
        ios'",
269     "lint": "oxlint --type-aware",
270     "lint:agent:ingress-owner": "node scripts/check-ingress-agent-owner-context.mjs",
271     "lint:all": "pnpm lint && pnpm lint:swift",
272     "lint:auth:no-pairing-store-group": "node scripts/check-no-pairing-store-group-auth.mjs
        ",
273     "lint:auth:pairing-account-scope": "node scripts/check-pairing-account-scope.mjs",
274     "lint:docs": "pnpm dlx markdownlint-cli2",
275     "lint:docs:fix": "pnpm dlx markdownlint-cli2 --fix",
276     "lint:fix": "oxlint --type-aware --fix && pnpm format",
277     "lint:plugins:no-monolithic-plugin-sdk-entry-imports": "node --import tsx scripts/check-
        no-monolithic-plugin-sdk-entry-imports.ts",
278     "lint:plugins:no-register-http-handler": "node scripts/check-no-register-http-handler.
        mjs",
279     "lint:swift": "swiftlint lint --config .swiftlint.yml && (cd apps/ios && swiftlint lint
        --config .swiftlint.yml)",
280     "lint:tmp:channel-agnostic-boundaries": "node scripts/check-channel-agnostic-boundaries.
        mjs",
281     "lint:tmp:no-random-messaging": "node scripts/check-no-random-messaging-tmp.mjs",
282     "lint:tmp:no-raw-channel-fetch": "node scripts/check-no-raw-channel-fetch.mjs",
283     "lint:ui:no-raw-window-open": "node scripts/check-no-raw-window-open.mjs",
284     "lint:webhook:no-low-level-body-read": "node scripts/check-webhook-auth-body-order.mjs",
285     "mac:open": "open dist/OpenClaw.app",
286     "mac:package": "bash scripts/package-mac-app.sh",
287     "mac:restart": "bash scripts/restart-mac.sh",
288     "moltbot:rpc": "node scripts/run-node.mjs agent --mode rpc --json",
289     "openclaw": "node scripts/run-node.mjs",
290     "openclaw:rpc": "node scripts/run-node.mjs agent --mode rpc --json",
291     "plugins:sync": "node --import tsx scripts/sync-plugin-versions.ts",
292     "prepack": "pnpm build && pnpm ui:build",
293     "prepare": "command -v git >/dev/null 2>&1 && git rev-parse --is-inside-work-tree >/dev/
        null 2>&1 && git config core.hooksPath git-hooks || exit 0",
```

```
294     "protocol:check": "pnpm protocol:gen && pnpm protocol:gen:swift && git diff --exit-code
        -- dist/protocol.schema.json apps/macos/Sources/OpenClawProtocol/GatewayModels.swift",
295     "protocol:gen": "node --import tsx scripts/protocol-gen.ts",
296     "protocol:gen:swift": "node --import tsx scripts/protocol-gen-swift.ts",
297     "release:check": "node --import tsx scripts/release-check.ts",
298     "release:openclaw:npm:check": "node --import tsx scripts/openclaw-npm-release-check.ts",
299     "start": "node scripts/run-node.mjs",
300     "test": "node scripts/test-parallel.mjs",
301     "test:all": "pnpm lint && pnpm build && pnpm test && pnpm test:e2e && pnpm test:live &&
        pnpm test:docker:all",
302     "test:auth:compat": "vitest run --config vitest.gateway.config.ts src/gateway/server.
        auth.compat-baseline.test.ts src/gateway/client.test.ts src/gateway/reconnect-gating.
        test.ts src/gateway/protocol/connect-error-details.test.ts",
303     "test:channels": "vitest run --config vitest.channels.config.ts",
304     "test:coverage": "vitest run --config vitest.unit.config.ts --coverage",
305     "test:docker:all": "pnpm test:docker:live-models && pnpm test:docker:live-gateway &&
        pnpm test:docker:onboard && pnpm test:docker:gateway-network && pnpm test:docker:qr &&
        pnpm test:docker:doctor-switch && pnpm test:docker:plugins && pnpm test:docker:cleanup",
306     "test:docker:cleanup": "bash scripts/test-cleanup-docker.sh",
307     "test:docker:doctor-switch": "bash scripts/e2e/doctor-install-switch-docker.sh",
308     "test:docker:gateway-network": "bash scripts/e2e/gateway-network-docker.sh",
309     "test:docker:live-gateway": "bash scripts/test-live-gateway-models-docker.sh",
310     "test:docker:live-models": "bash scripts/test-live-models-docker.sh",
311     "test:docker:onboard": "bash scripts/e2e/onboard-docker.sh",
312     "test:docker:plugins": "bash scripts/e2e/plugins-docker.sh",
313     "test:docker:qr": "bash scripts/e2e/qr-import-docker.sh",
314     "test:e2e": "vitest run --config vitest.e2e.config.ts",
315     "test:extensions": "vitest run --config vitest.extensions.config.ts",
316     "test:fast": "vitest run --config vitest.unit.config.ts",
317     "test:force": "node --import tsx scripts/test-force.ts",
318     "test:gateway": "vitest run --config vitest.gateway.config.ts --pool=forks",
319     "test:install:e2e": "bash scripts/test-install-sh-e2e-docker.sh",
320     "test:install:e2e:anthropic": "OPENCLAW_E2E_MODELS=anthropic CLAWDBOT_E2E_MODELS=
        anthropic bash scripts/test-install-sh-e2e-docker.sh",
321     "test:install:e2e:openai": "OPENCLAW_E2E_MODELS=openai CLAWDBOT_E2E_MODELS=openai bash
        scripts/test-install-sh-e2e-docker.sh",
322     "test:install:smoke": "bash scripts/test-install-sh-docker.sh",
323     "test:live": "OPENCLAW_LIVE_TEST=1 CLAWDBOT_LIVE_TEST=1 vitest run --config vitest.live.
        config.ts",
324     "test:macmini": "OPENCLAW_TEST_VM_FORKS=0 OPENCLAW_TEST_PROFILE=serial node scripts/test
        -parallel.mjs",
325     "test:perf:budget": "node scripts/test-perf-budget.mjs",
326     "test:perf:hotspots": "node scripts/test-hotspots.mjs",
327     "test:sectriage": "pnpm exec vitest run --config vitest.gateway.config.ts && vitest run
        --config vitest.unit.config.ts --exclude src/daemon/launchd.integration.test.ts --
        exclude src/process/exec.test.ts",
328     "test:ui": "pnpm lint:ui:no-raw-window-open && pnpm --dir ui test",
```

```
    "test:voicecall:closedloop": "vitest run extensions/voice-call/src/manager.test.ts extensions/voice-call/src/media-stream.test.ts src/plugins/voice-call.plugin.test.ts --maxWorkers=1",
    "test:watch": "vitest",
    "tui": "node scripts/run-node.mjs tui",
    "tui:dev": "OPENCLAW_PROFILE=dev CLAWDBOT_PROFILE=dev node scripts/run-node.mjs --dev tui",
    "ui:build": "node scripts/ui.js build",
    "ui:dev": "node scripts/ui.js dev",
    "ui:install": "node scripts/ui.js install"
  },
  "dependencies": {
    "@agentclientprotocol/sdk": "0.15.0",
    "@aws-sdk/client-bedrock": "^3.1004.0",
    "@buape/carbon": "0.0.0-beta-20260216184201",
    "@clack/prompts": "^1.1.0",
    "@discordjs/voice": "^0.19.0",
    "@grammyjs/runner": "^2.0.3",
    "@grammyjs/transformer-throttler": "^1.2.1",
    "@homebridge/ciao": "^1.3.5",
    "@larksuiteoapi/node-sdk": "^1.59.0",
    "@line/bot-sdk": "^10.6.0",
    "@lydell/node-pty": "1.2.0-beta.3",
    "@mariozechner/pi-agent-core": "0.57.1",
    "@mariozechner/pi-ai": "0.57.1",
    "@mariozechner/pi-coding-agent": "0.57.1",
    "@mariozechner/pi-tui": "0.57.1",
    "@mozilla/readability": "^0.6.0",
    "@sinclair/typebox": "0.34.48",
    "@slack/bolt": "^4.6.0",
    "@slack/web-api": "^7.14.1",
    "@whiskeysockets/baileys": "7.0.0-rc.9",
    "ajv": "^8.18.0",
    "chalk": "^5.6.2",
    "chokidar": "^5.0.0",
    "cli-highlight": "^2.1.11",
    "commander": "^14.0.3",
    "croner": "^10.0.1",
    "discord-api-types": "^0.38.41",
    "dotenv": "^17.3.1",
    "express": "^5.2.1",
    "file-type": "^21.3.1",
    "grammy": "^1.41.1",
    "hono": "4.12.7",
    "https-proxy-agent": "^7.0.6",
    "ipaddr.js": "^2.3.0",
    "jiti": "^2.6.1",
```

```
    "json5": "^2.2.3",
    "jszip": "^3.10.1",
    "linkedom": "^0.18.12",
    "long": "^5.3.2",
    "markdown-it": "^14.1.1",
    "node-edge-tts": "^1.2.10",
    "opusscript": "^0.1.1",
    "osc-progress": "^0.3.0",
    "pdfjs-dist": "^5.5.207",
    "playwright-core": "1.58.2",
    "qrcode-terminal": "^0.12.0",
    "sharp": "^0.34.5",
    "sqlite-vec": "0.1.7-alpha.2",
    "tar": "7.5.11",
    "tslog": "^4.10.2",
    "undici": "^7.22.0",
    "ws": "^8.19.0",
    "yaml": "^2.8.2",
    "zod": "^4.3.6"
  },
  "devDependencies": {
    "@grammyjs/types": "^3.25.0",
    "@lit-labs/signals": "^0.2.0",
    "@lit/context": "^1.1.6",
    "@types/express": "^5.0.6",
    "@types/markdown-it": "^14.1.2",
    "@types/node": "^25.3.5",
    "@types/qrcode-terminal": "^0.12.2",
    "@types/ws": "^8.18.1",
    "@typescript/native-preview": "7.0.0-dev.20260308.1",
    "@vitest/coverage-v8": "^4.0.18",
    "jscpd": "4.0.8",
    "lit": "^3.3.2",
    "oxfmt": "0.36.0",
    "oxlint": "^1.51.0",
    "oxlint-tsgolint": "^0.16.0",
    "signal-utils": "0.21.1",
    "tsdown": "0.21.0",
    "tsx": "^4.21.0",
    "typescript": "^5.9.3",
    "vitest": "^4.0.18"
  },
  "peerDependencies": {
    "@napi-rs/canvas": "^0.1.89",
    "node-llama-cpp": "3.16.2"
  },
  "engines": {
```

```
    "node": ">=22.12.0"
  },
  "packageManager": "pnpm@10.23.0",
  "pnpm": {
    "minimumReleaseAge": 2880,
    "overrides": {
      "hono": "4.12.7",
      "@hono/node-server": "1.19.10",
      "fast-xml-parser": "5.3.8",
      "request": "npm:@cypress/request@3.0.10",
      "request-promise": "npm:@cypress/request-promise@5.0.0",
      "file-type": "21.3.1",
      "form-data": "2.5.4",
      "minimatch": "10.2.4",
      "qs": "6.14.2",
      "node-domexception": "npm:@nolyfill/domexception@^1.0.28",
      "@sinclair/typebox": "0.34.48",
      "tar": "7.5.11",
      "tough-cookie": "4.1.3"
    },
    "onlyBuiltDependencies": [
      "@lydell/node-pty",
      "@matrix-org/matrix-sdk-crypto-nodejs",
      "@napi-rs/canvas",
      "@tloncorp/api",
      "@whiskeysockets/baileys",
      "authenticate-pam",
      "esbuild",
      "koffi",
      "node-llama-cpp",
      "protobufjs",
      "sharp"
    ],
    "packageExtensions": {
      "@mariozechner/pi-coding-agent": {
        "dependencies": {
          "strip-ansi": "^7.2.0"
        }
      }
    }
  }
}
```

B.3　src/cli/gateway-cli/run.ts

作用说明

定义 `gateway run` 及其参数解析和 server 启动入口。

完整内容

```
import fs from "node:fs";
import path from "node:path";
import type { Command } from "commander";
import { readSecretFromFile } from "../../acp/secret-file.js";
import type { GatewayAuthMode, GatewayTailscaleMode } from "../../config/config.js";
import {
  CONFIG_PATH,
  loadConfig,
  readConfigFileSnapshot,
  resolveStateDir,
  resolveGatewayPort,
} from "../../config/config.js";
import { hasConfiguredSecretInput } from "../../config/types.secrets.js";
import { resolveGatewayAuth } from "../../gateway/auth.js";
import { startGatewayServer } from "../../gateway/server.js";
import type { GatewayWsLogStyle } from "../../gateway/ws-logging.js";
import { setGatewayWsLogStyle } from "../../gateway/ws-logging.js";
import { setVerbose } from "../../globals.js";
import { GatewayLockError } from "../../infra/gateway-lock.js";
import { formatPortDiagnostics, inspectPortUsage } from "../../infra/ports.js";
import { cleanStaleGatewayProcessesSync } from "../../infra/restart-stale-pids.js";
import { setConsoleSubsystemFilter, setConsoleTimestampPrefix } from "../../logging/console.
    js";
import { createSubsystemLogger } from "../../logging/subsystem.js";
import { defaultRuntime } from "../../runtime.js";
import { formatCliCommand } from "../command-format.js";
import { inheritOptionFromParent } from "../command-options.js";
import { forceFreePortAndWait, waitForPortBindable } from "../ports.js";
import { ensureDevGatewayConfig } from "./dev.js";
import { runGatewayLoop } from "./run-loop.js";
import {
  describeUnknownError,
  extractGatewayMiskeys,
  maybeExplainGatewayServiceStop,
  parsePort,
  toOptionString,
} from "./shared.js";

type GatewayRunOpts = {
```

```
  port?: unknown;
  bind?: unknown;
  token?: unknown;
  auth?: unknown;
  password?: unknown;
  passwordFile?: unknown;
  tailscale?: unknown;
  tailscaleResetOnExit?: boolean;
  allowUnconfigured?: boolean;
  force?: boolean;
  verbose?: boolean;
  claudeCliLogs?: boolean;
  wsLog?: unknown;
  compact?: boolean;
  rawStream?: boolean;
  rawStreamPath?: unknown;
  dev?: boolean;
  reset?: boolean;
};

const gatewayLog = createSubsystemLogger("gateway");

const GATEWAY_RUN_VALUE_KEYS = [
  "port",
  "bind",
  "token",
  "auth",
  "password",
  "passwordFile",
  "tailscale",
  "wsLog",
  "rawStreamPath",
] as const;

const GATEWAY_RUN_BOOLEAN_KEYS = [
  "tailscaleResetOnExit",
  "allowUnconfigured",
  "dev",
  "reset",
  "force",
  "verbose",
  "claudeCliLogs",
  "compact",
  "rawStream",
] as const;

const GATEWAY_AUTH_MODES: readonly GatewayAuthMode[] = [
```

```
  "none",
  "token",
  "password",
  "trusted-proxy",
];
const GATEWAY_TAILSCALE_MODES: readonly GatewayTailscaleMode[] = ["off", "serve", "funnel"];

function warnInlinePasswordFlag() {
  defaultRuntime.error(
    "Warning: --password can be exposed via process listings. Prefer --password-file or
    OPENCLAW_GATEWAY_PASSWORD.",
  );
}

function resolveGatewayPasswordOption(opts: GatewayRunOpts): string | undefined {
  const direct = toOptionString(opts.password);
  const file = toOptionString(opts.passwordFile);
  if (direct && file) {
    throw new Error("Use either --password or --password-file.");
  }
  if (file) {
    return readSecretFromFile(file, "Gateway password");
  }
  return direct;
}

function parseEnumOption<T extends string>(
  raw: string | undefined,
  allowed: readonly T[],
): T | null {
  if (!raw) {
    return null;
  }
  return (allowed as readonly string[]).includes(raw) ? (raw as T) : null;
}

function formatModeChoices<T extends string>(modes: readonly T[]): string {
  return modes.map((mode) => `"${mode}"`).join("|");
}

function formatModeErrorList<T extends string>(modes: readonly T[]): string {
  const quoted = modes.map((mode) => `"${mode}"`);
  if (quoted.length === 0) {
    return "";
  }
  if (quoted.length === 1) {
    return quoted[0];
```

```
  }
  if (quoted.length === 2) {
    return `${quoted[0]} or ${quoted[1]}`;
  }
  return `${quoted.slice(0, -1).join(", ")}, or ${quoted[quoted.length - 1]}`;
}

function resolveGatewayRunOptions(opts: GatewayRunOpts, command?: Command): GatewayRunOpts {
  const resolved: GatewayRunOpts = { ...opts };

  for (const key of GATEWAY_RUN_VALUE_KEYS) {
    const inherited = inheritOptionFromParent(command, key);
    if (key === "wsLog") {
      // wsLog has a child default ("auto"), so prefer inherited parent CLI value when
    present.
      resolved[key] = inherited ?? resolved[key];
      continue;
    }
    resolved[key] = resolved[key] ?? inherited;
  }

  for (const key of GATEWAY_RUN_BOOLEAN_KEYS) {
    const inherited = inheritOptionFromParent<boolean>(command, key);
    resolved[key] = Boolean(resolved[key] || inherited);
  }

  return resolved;
}

async function runGatewayCommand(opts: GatewayRunOpts) {
  const isDevProfile = process.env.OPENCLAW_PROFILE?.trim().toLowerCase() === "dev";
  const devMode = Boolean(opts.dev) || isDevProfile;
  if (opts.reset && !devMode) {
    defaultRuntime.error("Use --reset with --dev.");
    defaultRuntime.exit(1);
    return;
  }

  setConsoleTimestampPrefix(true);
  setVerbose(Boolean(opts.verbose));
  if (opts.claudeCliLogs) {
    setConsoleSubsystemFilter(["agent/claude-cli"]);
    process.env.OPENCLAW_CLAUDE_CLI_LOG_OUTPUT = "1";
  }
  const wsLogRaw = (opts.compact ? "compact" : opts.wsLog) as string | undefined;
  const wsLogStyle: GatewayWsLogStyle =
    wsLogRaw === "compact" ? "compact" : wsLogRaw === "full" ? "full" : "auto";
```

```
178  if (
179    wsLogRaw !== undefined &&
180    wsLogRaw !== "auto" &&
181    wsLogRaw !== "compact" &&
182    wsLogRaw !== "full"
183  ) {
184    defaultRuntime.error('Invalid --ws-log (use "auto", "full", "compact")');
185    defaultRuntime.exit(1);
186  }
187  setGatewayWsLogStyle(wsLogStyle);
188
189  if (opts.rawStream) {
190    process.env.OPENCLAW_RAW_STREAM = "1";
191  }
192  const rawStreamPath = toOptionString(opts.rawStreamPath);
193  if (rawStreamPath) {
194    process.env.OPENCLAW_RAW_STREAM_PATH = rawStreamPath;
195  }
196
197  if (devMode) {
198    await ensureDevGatewayConfig({ reset: Boolean(opts.reset) });
199  }
200
201  const cfg = loadConfig();
202  const portOverride = parsePort(opts.port);
203  if (opts.port !== undefined && portOverride === null) {
204    defaultRuntime.error("Invalid port");
205    defaultRuntime.exit(1);
206  }
207  const port = portOverride ?? resolveGatewayPort(cfg);
208  if (!Number.isFinite(port) || port <= 0) {
209    defaultRuntime.error("Invalid port");
210    defaultRuntime.exit(1);
211  }
212  const bindRaw = toOptionString(opts.bind) ?? cfg.gateway?.bind ?? "loopback";
213  const bind =
214    bindRaw === "loopback" ||
215    bindRaw === "lan" ||
216    bindRaw === "auto" ||
217    bindRaw === "custom" ||
218    bindRaw === "tailnet"
219      ? bindRaw
220      : null;
221  if (!bind) {
222    defaultRuntime.error('Invalid --bind (use "loopback", "lan", "tailnet", "auto", or "
     custom")');
223    defaultRuntime.exit(1);
```

```
    return;
  }
  if (process.env.OPENCLAW_SERVICE_MARKER?.trim()) {
    const stale = cleanStaleGatewayProcessesSync(port);
    if (stale.length > 0) {
      gatewayLog.info(
        `service-mode: cleared ${stale.length} stale gateway pid(s) before bind on port ${port}`,
      );
    }
  }
  if (opts.force) {
    try {
      const { killed, waitedMs, escalatedToSigkill } = await forceFreePortAndWait(port, {
        timeoutMs: 2000,
        intervalMs: 100,
        sigtermTimeoutMs: 700,
      });
      if (killed.length === 0) {
        gatewayLog.info(`force: no listeners on port ${port}`);
      } else {
        for (const proc of killed) {
          gatewayLog.info(
            `force: killed pid ${proc.pid}${proc.command ? ` (${proc.command})` : ""} on port ${port}`,
          );
        }
        if (escalatedToSigkill) {
          gatewayLog.info(`force: escalated to SIGKILL while freeing port ${port}`);
        }
        if (waitedMs > 0) {
          gatewayLog.info(`force: waited ${waitedMs}ms for port ${port} to free`);
        }
      }
      // After killing, verify the port is actually bindable (handles TIME_WAIT).
      const bindProbeHost =
        bind === "loopback"
          ? "127.0.0.1"
          : bind === "lan"
            ? "0.0.0.0"
            : bind === "custom"
              ? toOptionString(cfg.gateway?.customBindHost)
              : undefined;
      const bindWaitMs = await waitForPortBindable(port, {
        timeoutMs: 3000,
        intervalMs: 150,
        host: bindProbeHost,
```

```
      });
      if (bindWaitMs > 0) {
        gatewayLog.info(`force: waited ${bindWaitMs}ms for port ${port} to become bindable`)
    ;
      }
    } catch (err) {
      defaultRuntime.error(`Force: ${String(err)}`);
      defaultRuntime.exit(1);
      return;
    }
  }
  if (opts.token) {
    const token = toOptionString(opts.token);
    if (token) {
      process.env.OPENCLAW_GATEWAY_TOKEN = token;
    }
  }
  const authModeRaw = toOptionString(opts.auth);
  const authMode = parseEnumOption(authModeRaw, GATEWAY_AUTH_MODES);
  if (authModeRaw && !authMode) {
    defaultRuntime.error(`Invalid --auth (use ${formatModeErrorList(GATEWAY_AUTH_MODES)})`);
    defaultRuntime.exit(1);
    return;
  }
  const tailscaleRaw = toOptionString(opts.tailscale);
  const tailscaleMode = parseEnumOption(tailscaleRaw, GATEWAY_TAILSCALE_MODES);
  if (tailscaleRaw && !tailscaleMode) {
    defaultRuntime.error(
      `Invalid --tailscale (use ${formatModeErrorList(GATEWAY_TAILSCALE_MODES)})`,
    );
    defaultRuntime.exit(1);
    return;
  }
  let passwordRaw: string | undefined;
  try {
    passwordRaw = resolveGatewayPasswordOption(opts);
  } catch (err) {
    defaultRuntime.error(err instanceof Error ? err.message : String(err));
    defaultRuntime.exit(1);
    return;
  }
  if (toOptionString(opts.password)) {
    warnInlinePasswordFlag();
  }
  const tokenRaw = toOptionString(opts.token);

  const snapshot = await readConfigFileSnapshot().catch(() => null);
```

```
315  const configExists = snapshot?.exists ?? fs.existsSync(CONFIG_PATH);
316  const configAuditPath = path.join(resolveStateDir(process.env), "logs", "config-audit.
       jsonl");
317  const mode = cfg.gateway?.mode;
318  if (!opts.allowUnconfigured && mode !== "local") {
319    if (!configExists) {
320      defaultRuntime.error(
321        `Missing config. Run \`${formatCliCommand("openclaw setup")}\` or set gateway.mode=
       local (or pass --allow-unconfigured).`,
322      );
323    } else {
324      defaultRuntime.error(
325        `Gateway start blocked: set gateway.mode=local (current: ${mode ?? "unset"}) or pass
       --allow-unconfigured.`,
326      );
327      defaultRuntime.error(`Config write audit: ${configAuditPath}`);
328    }
329    defaultRuntime.exit(1);
330    return;
331  }
332  const miskeys = extractGatewayMiskeys(snapshot?.parsed);
333  const authOverride =
334    authMode || passwordRaw || tokenRaw || authModeRaw
335      ? {
336          ...(authMode ? { mode: authMode } : {}),
337          ...(tokenRaw ? { token: tokenRaw } : {}),
338          ...(passwordRaw ? { password: passwordRaw } : {}),
339        }
340      : undefined;
341  const resolvedAuth = resolveGatewayAuth({
342    authConfig: cfg.gateway?.auth,
343    authOverride,
344    env: process.env,
345    tailscaleMode: tailscaleMode ?? cfg.gateway?.tailscale?.mode ?? "off",
346  });
347  const resolvedAuthMode = resolvedAuth.mode;
348  const tokenValue = resolvedAuth.token;
349  const passwordValue = resolvedAuth.password;
350  const hasToken = typeof tokenValue === "string" && tokenValue.trim().length > 0;
351  const hasPassword = typeof passwordValue === "string" && passwordValue.trim().length > 0;
352  const tokenConfigured =
353    hasToken ||
354    hasConfiguredSecretInput(
355      authOverride?.token ?? cfg.gateway?.auth?.token,
356      cfg.secrets?.defaults,
357    );
358  const passwordConfigured =
```

```
    hasPassword ||
    hasConfiguredSecretInput(
      authOverride?.password ?? cfg.gateway?.auth?.password,
      cfg.secrets?.defaults,
    );
  const hasSharedSecret =
    (resolvedAuthMode === "token" && tokenConfigured) ||
    (resolvedAuthMode === "password" && passwordConfigured);
  const canBootstrapToken = resolvedAuthMode === "token" && !tokenConfigured;
  const authHints: string[] = [];
  if (miskeys.hasGatewayToken) {
    authHints.push('Found "gateway.token" in config. Use "gateway.auth.token" instead.');
  }
  if (miskeys.hasRemoteToken) {
    authHints.push(
      '"gateway.remote.token" is for remote CLI calls; it does not enable local gateway auth.',
    );
  }
  if (resolvedAuthMode === "password" && !passwordConfigured) {
    defaultRuntime.error(
      [
        "Gateway auth is set to password, but no password is configured.",
        "Set gateway.auth.password (or OPENCLAW_GATEWAY_PASSWORD), or pass --password.",
        ...authHints,
      ]
        .filter(Boolean)
        .join("\n"),
    );
    defaultRuntime.exit(1);
    return;
  }
  if (resolvedAuthMode === "none") {
    gatewayLog.warn(
      "Gateway auth mode=none explicitly configured; all gateway connections are unauthenticated.",
    );
  }
  if (
    bind !== "loopback" &&
    !hasSharedSecret &&
    !canBootstrapToken &&
    resolvedAuthMode !== "trusted-proxy"
  ) {
    defaultRuntime.error(
      [
        `Refusing to bind gateway to ${bind} without auth.`,
```

```
      "Set gateway.auth.token/password (or OPENCLAW_GATEWAY_TOKEN/OPENCLAW_GATEWAY_PASSWORD) or pass --token/--password.",
      ...authHints,
    ]
      .filter(Boolean)
      .join("\n"),
  );
  defaultRuntime.exit(1);
  return;
}
const tailscaleOverride =
  tailscaleMode || opts.tailscaleResetOnExit
    ? {
        ...(tailscaleMode ? { mode: tailscaleMode } : {}),
        ...(opts.tailscaleResetOnExit ? { resetOnExit: true } : {}),
      }
    : undefined;

try {
  await runGatewayLoop({
    runtime: defaultRuntime,
    lockPort: port,
    start: async () =>
      await startGatewayServer(port, {
        bind,
        auth: authOverride,
        tailscale: tailscaleOverride,
      }),
  });
} catch (err) {
  if (
    err instanceof GatewayLockError ||
    (err && typeof err === "object" && (err as { name?: string }).name === "GatewayLockError")
  ) {
    const errMessage = describeUnknownError(err);
    defaultRuntime.error(
      `Gateway failed to start: ${errMessage}\nIf the gateway is supervised, stop it with: ${formatCliCommand("openclaw gateway stop")}`,
    );
    try {
      const diagnostics = await inspectPortUsage(port);
      if (diagnostics.status === "busy") {
        for (const line of formatPortDiagnostics(diagnostics)) {
          defaultRuntime.error(line);
        }
      }
```

```
      } catch {
        // ignore diagnostics failures
      }
      await maybeExplainGatewayServiceStop();
      defaultRuntime.exit(1);
      return;
    }
    defaultRuntime.error(`Gateway failed to start: ${String(err)}`);
    defaultRuntime.exit(1);
  }
}

export function addGatewayRunCommand(cmd: Command): Command {
  return cmd
    .option("--port <port>", "Port for the gateway WebSocket")
    .option(
      "--bind <mode>",
      'Bind mode ("loopback"|"lan"|"tailnet"|"auto"|"custom"). Defaults to config gateway.bind (or loopback).',
    )
    .option(
      "--token <token>",
      "Shared token required in connect.params.auth.token (default: OPENCLAW_GATEWAY_TOKEN env if set)",
    )
    .option("--auth <mode>", `Gateway auth mode (${formatModeChoices(GATEWAY_AUTH_MODES)})`)
    .option("--password <password>", "Password for auth mode=password")
    .option("--password-file <path>", "Read gateway password from file")
    .option(
      "--tailscale <mode>",
      `Tailscale exposure mode (${formatModeChoices(GATEWAY_TAILSCALE_MODES)})`,
    )
    .option(
      "--tailscale-reset-on-exit",
      "Reset Tailscale serve/funnel configuration on shutdown",
      false,
    )
    .option(
      "--allow-unconfigured",
      "Allow gateway start without gateway.mode=local in config",
      false,
    )
    .option("--dev", "Create a dev config + workspace if missing (no BOOTSTRAP.md)", false)
    .option(
      "--reset",
      "Reset dev config + credentials + sessions + workspace (requires --dev)",
      false,
```

```
    )
    .option("--force", "Kill any existing listener on the target port before starting",
    false)
    .option("--verbose", "Verbose logging to stdout/stderr", false)
    .option(
      "--claude-cli-logs",
      "Only show claude-cli logs in the console (includes stdout/stderr)",
      false,
    )
    .option("--ws-log <style>", 'WebSocket log style ("auto"|"full"|"compact")', "auto")
    .option("--compact", 'Alias for "--ws-log compact"', false)
    .option("--raw-stream", "Log raw model stream events to jsonl", false)
    .option("--raw-stream-path <path>", "Raw stream jsonl path")
    .action(async (opts, command) => {
      await runGatewayCommand(resolveGatewayRunOptions(opts, command));
    });
}
```

B.4　src/cli/gateway-cli/run-loop.ts

作用说明

Gateway 生命周期循环、重启、关停、信号处理和锁管理。

完整内容

```
import {
  abortEmbeddedPiRun,
  getActiveEmbeddedRunCount,
  waitForActiveEmbeddedRuns,
} from "../../agents/pi-embedded-runner/runs.js";
import type { startGatewayServer } from "../../gateway/server.js";
import { acquireGatewayLock } from "../../infra/gateway-lock.js";
import { restartGatewayProcessWithFreshPid } from "../../infra/process-respawn.js";
import {
  consumeGatewaySigusr1RestartAuthorization,
  isGatewaySigusr1RestartExternallyAllowed,
  markGatewaySigusr1RestartHandled,
} from "../../infra/restart.js";
import { createSubsystemLogger } from "../../logging/subsystem.js";
import {
  getActiveTaskCount,
  markGatewayDraining,
  resetAllLanes,
  waitForActiveTasks,
} from "../../process/command-queue.js";
```

```
import { createRestartIterationHook } from "../../process/restart-recovery.js";
import type { defaultRuntime } from "../../runtime.js";

const gatewayLog = createSubsystemLogger("gateway");

type GatewayRunSignalAction = "stop" | "restart";

export async function runGatewayLoop(params: {
  start: () => Promise<Awaited<ReturnType<typeof startGatewayServer>>>;
  runtime: typeof defaultRuntime;
  lockPort?: number;
}) {
  let lock = await acquireGatewayLock({ port: params.lockPort });
  let server: Awaited<ReturnType<typeof startGatewayServer>> | null = null;
  let shuttingDown = false;
  let restartResolver: (() => void) | null = null;

  const cleanupSignals = () => {
    process.removeListener("SIGTERM", onSigterm);
    process.removeListener("SIGINT", onSigint);
    process.removeListener("SIGUSR1", onSigusr1);
  };
  const exitProcess = (code: number) => {
    cleanupSignals();
    params.runtime.exit(code);
  };
  const releaseLockIfHeld = async (): Promise<boolean> => {
    if (!lock) {
      return false;
    }
    await lock.release();
    lock = null;
    return true;
  };
  const reacquireLockForInProcessRestart = async (): Promise<boolean> => {
    try {
      lock = await acquireGatewayLock({ port: params.lockPort });
      return true;
    } catch (err) {
      gatewayLog.error(`failed to reacquire gateway lock for in-process restart: ${String(
    err)}`);
      exitProcess(1);
      return false;
    }
  };
  const handleRestartAfterServerClose = async () => {
    const hadLock = await releaseLockIfHeld();
```

```
    // Release the lock BEFORE spawning so the child can acquire it immediately.
    const respawn = restartGatewayProcessWithFreshPid();
    if (respawn.mode === "spawned" || respawn.mode === "supervised") {
      const modeLabel =
        respawn.mode === "spawned"
          ? `spawned pid ${respawn.pid ?? "unknown"}`
          : "supervisor restart";
      gatewayLog.info(`restart mode: full process restart (${modeLabel})`);
      exitProcess(0);
      return;
    }
    if (respawn.mode === "failed") {
      gatewayLog.warn(
        `full process restart failed (${respawn.detail ?? "unknown error"}); falling back to in-process restart`,
      );
    } else {
      gatewayLog.info(
        `restart mode: in-process restart (${respawn.detail ?? "OPENCLAW_NO_RESPAWN"})`,
      );
    }
    if (hadLock && !(await reacquireLockForInProcessRestart())) {
      return;
    }
    shuttingDown = false;
    restartResolver?.();
  };
  const handleStopAfterServerClose = async () => {
    await releaseLockIfHeld();
    exitProcess(0);
  };

  const DRAIN_TIMEOUT_MS = 90_000;
  const SHUTDOWN_TIMEOUT_MS = 5_000;

  const request = (action: GatewayRunSignalAction, signal: string) => {
    if (shuttingDown) {
      gatewayLog.info(`received ${signal} during shutdown; ignoring`);
      return;
    }
    shuttingDown = true;
    const isRestart = action === "restart";
    gatewayLog.info(`received ${signal}; ${isRestart ? "restarting" : "shutting down"}`);

    // Allow extra time for draining active turns on restart.
    const forceExitMs = isRestart ? DRAIN_TIMEOUT_MS + SHUTDOWN_TIMEOUT_MS : SHUTDOWN_TIMEOUT_MS;
```

```
    const forceExitTimer = setTimeout(() => {
      gatewayLog.error("shutdown timed out; exiting without full cleanup");
      // Exit non-zero on restart timeout so launchd/systemd treats it as a
      // failure and triggers a clean process restart instead of assuming the
      // shutdown was intentional. Stop-timeout stays at 0 (graceful). (#36822)
      exitProcess(isRestart ? 1 : 0);
    }, forceExitMs);

    void (async () => {
      try {
        // On restart, wait for in-flight agent turns to finish before
        // tearing down the server so buffered messages are delivered.
        if (isRestart) {
          // Reject new enqueues immediately during the drain window so
          // sessions get an explicit restart error instead of silent task loss.
          markGatewayDraining();
          const activeTasks = getActiveTaskCount();
          const activeRuns = getActiveEmbeddedRunCount();

          // Best-effort abort for compacting runs so long compaction operations
          // don't hold session write locks across restart boundaries.
          if (activeRuns > 0) {
            abortEmbeddedPiRun(undefined, { mode: "compacting" });
          }

          if (activeTasks > 0 || activeRuns > 0) {
            gatewayLog.info(
              `draining ${activeTasks} active task(s) and ${activeRuns} active embedded run(s) before restart (timeout ${DRAIN_TIMEOUT_MS}ms)`,
            );
            const [tasksDrain, runsDrain] = await Promise.all([
              activeTasks > 0
                ? waitForActiveTasks(DRAIN_TIMEOUT_MS)
                : Promise.resolve({ drained: true }),
              activeRuns > 0
                ? waitForActiveEmbeddedRuns(DRAIN_TIMEOUT_MS)
                : Promise.resolve({ drained: true }),
            ]);
            if (tasksDrain.drained && runsDrain.drained) {
              gatewayLog.info("all active work drained");
            } else {
              gatewayLog.warn("drain timeout reached; proceeding with restart");
              // Final best-effort abort to avoid carrying active runs into the
              // next lifecycle when drain time budget is exhausted.
              abortEmbeddedPiRun(undefined, { mode: "all" });
            }
          }
```

```
      }

      await server?.close({
        reason: isRestart ? "gateway restarting" : "gateway stopping",
        restartExpectedMs: isRestart ? 1500 : null,
      });
    } catch (err) {
      gatewayLog.error(`shutdown error: ${String(err)}`);
    } finally {
      clearTimeout(forceExitTimer);
      server = null;
      if (isRestart) {
        await handleRestartAfterServerClose();
      } else {
        await handleStopAfterServerClose();
      }
    }
  })();
};

const onSigterm = () => {
  gatewayLog.info("signal SIGTERM received");
  request("stop", "SIGTERM");
};
const onSigint = () => {
  gatewayLog.info("signal SIGINT received");
  request("stop", "SIGINT");
};
const onSigusr1 = () => {
  gatewayLog.info("signal SIGUSR1 received");
  const authorized = consumeGatewaySigusr1RestartAuthorization();
  if (!authorized && !isGatewaySigusr1RestartExternallyAllowed()) {
    gatewayLog.warn(
      "SIGUSR1 restart ignored (not authorized; commands.restart=false or use gateway tool
  ).",
    );
    return;
  }
  markGatewaySigusr1RestartHandled();
  request("restart", "SIGUSR1");
};

process.on("SIGTERM", onSigterm);
process.on("SIGINT", onSigint);
process.on("SIGUSR1", onSigusr1);

try {
```

```
    const onIteration = createRestartIterationHook(() => {
      // After an in-process restart (SIGUSR1), reset command-queue lane state.
      // Interrupted tasks from the previous lifecycle may have left `active`
      // counts elevated (their finally blocks never ran), permanently blocking
      // new work from draining. This must happen here — at the restart
      // coordinator level — rather than inside individual subsystem init
      // functions, to avoid surprising cross-cutting side effects.
      resetAllLanes();
    });

    // Keep process alive; SIGUSR1 triggers an in-process restart (no supervisor required).
    // SIGTERM/SIGINT still exit after a graceful shutdown.
    let isFirstStart = true;
    // eslint-disable-next-line no-constant-condition
    while (true) {
      onIteration();
      try {
        server = await params.start();
        isFirstStart = false;
      } catch (err) {
        // On initial startup, let the error propagate so the outer handler
        // can report "Gateway failed to start" and exit non-zero. Only
        // swallow errors on subsequent in-process restarts to keep the
        // process alive (a crash would lose macOS TCC permissions). (#35862)
        if (isFirstStart) {
          throw err;
        }
        server = null;
        // Release the gateway lock so that `daemon restart/stop` (which
        // discovers PIDs via the gateway port) can still manage the process.
        // Without this, the process holds the lock but is not listening,
        // forcing manual cleanup. (#35862)
        await releaseLockIfHeld();
        const errMsg = err instanceof Error ? err.message : String(err);
        const errStack = err instanceof Error && err.stack ? `\n${err.stack}` : "";
        gatewayLog.error(
          `gateway startup failed: ${errMsg}. ` +
            `Process will stay alive; fix the issue and restart.${errStack}`,
        );
      }
      await new Promise<void>((resolve) => {
        restartResolver = resolve;
      });
    }
  } finally {
    await releaseLockIfHeld();
    cleanupSignals();
```

```
  }
}
```

B.5　src/infra/external-memory-kernel.ts

作用说明

当前 sidecar bridge 的完整实现。它决定何时、如何从 WSL 调用 Windows 侧 OpenClaw Memory Sidecar。

完整内容

```
import { spawn } from "node:child_process";
import fs from "node:fs";
import path from "node:path";
import { createSubsystemLogger } from "../logging/subsystem.js";

const memoryLog = createSubsystemLogger("memory");
const DEFAULT_MEMORY_ROOT = "/mnt/d/openclaw";
const DEFAULT_PYTHON = "python3";
const MIN_RUN_INTERVAL_MS = 2_000;

let activeRun: Promise<void> | null = null;
let lastAttemptAt = 0;

function resolveMemoryRoot(): string | null {
  const root = process.env.OPENCLAW_EXTERNAL_MEMORY_ROOT?.trim() || DEFAULT_MEMORY_ROOT;
  const mainPath = path.join(root, "main.py");
  if (!fs.existsSync(mainPath)) {
    return null;
  }
  return root;
}

function buildMemoryEnv(params: {
  source?: string;
  status?: string;
  sessionKey?: string;
  agentId?: string;
}): NodeJS.ProcessEnv {
  const env: NodeJS.ProcessEnv = { ...process.env };
  if (params.source) {
    env.OPENCLAW_MEMORY_SOURCE = params.source;
  }
  if (params.status) {
    env.OPENCLAW_MEMORY_STATUS = params.status;
```

```
  }
  if (params.sessionKey) {
    env.OPENCLAW_MEMORY_SESSION_KEY = params.sessionKey;
  }
  if (params.agentId) {
    env.OPENCLAW_MEMORY_AGENT_ID = params.agentId;
  }
  return env;
}

export function queueExternalMemoryKernelRun(params: {
  source: string;
  status: "success" | "final" | "error";
  sessionKey?: string;
  agentId?: string;
}): boolean {
  if (process.env.OPENCLAW_DISABLE_EXTERNAL_MEMORY === "1") {
    return false;
  }
  const root = resolveMemoryRoot();
  if (!root) {
    return false;
  }
  const now = Date.now();
  if (activeRun || now - lastAttemptAt < MIN_RUN_INTERVAL_MS) {
    return false;
  }
  lastAttemptAt = now;

  activeRun = new Promise<void>((resolve) => {
    const child = spawn(
      process.env.OPENCLAW_EXTERNAL_MEMORY_PYTHON?.trim() || DEFAULT_PYTHON,
      ["main.py"],
      {
        cwd: root,
        env: buildMemoryEnv(params),
        stdio: "ignore",
      },
    );

    child.once("error", (err) => {
      memoryLog.warn(`external memory kernel launch failed: ${String(err)}`);
      resolve();
    });

    child.once("exit", (code) => {
      if (code !== 0) {
```

```
        memoryLog.warn(`external memory kernel exited with code ${String(code)}`);
      }
      resolve();
    });
  }).finally(() => {
    activeRun = null;
  });

  return true;
}

export async function waitForExternalMemoryKernelIdleForTest(): Promise<void> {
  await activeRun;
}
```

B.6　src/gateway/server-methods/chat.ts

作用说明

真实聊天主路径，用户一句话最终会走到这里。当前已接入 `gateway-chat` sidecar 调用。

完整内容

```
import fs from "node:fs";
import path from "node:path";
import { CURRENT_SESSION_VERSION } from "@mariozechner/pi-coding-agent";
import { resolveSessionAgentId } from "../../agents/agent-scope.js";
import { resolveThinkingDefault } from "../../agents/model-selection.js";
import { resolveAgentTimeoutMs } from "../../agents/timeout.js";
import { dispatchInboundMessage } from "../../auto-reply/dispatch.js";
import { createReplyDispatcher } from "../../auto-reply/reply/reply-dispatcher.js";
import type { MsgContext } from "../../auto-reply/templating.js";
import { isSilentReplyText, SILENT_REPLY_TOKEN } from "../../auto-reply/tokens.js";
import { createReplyPrefixOptions } from "../../channels/reply-prefix.js";
import { resolveSessionFilePath } from "../../config/sessions.js";
import { queueExternalMemoryKernelRun } from "../../infra/external-memory-kernel.js";
import { jsonUtf8Bytes } from "../../infra/json-utf8-bytes.js";
import { normalizeInputProvenance, type InputProvenance } from "../../sessions/input-
    provenance.js";
import { resolveSendPolicy } from "../../sessions/send-policy.js";
import { parseAgentSessionKey } from "../../sessions/session-key-utils.js";
import {
  stripInlineDirectiveTagsForDisplay,
  stripInlineDirectiveTagsFromMessageForDisplay,
} from "../../utils/directive-tags.js";
import {
```

```
  INTERNAL_MESSAGE_CHANNEL,
  isWebchatClient,
  normalizeMessageChannel,
} from "../../utils/message-channel.js";
import {
  abortChatRunById,
  abortChatRunsForSessionKey,
  type ChatAbortControllerEntry,
  type ChatAbortOps,
  isChatStopCommandText,
  resolveChatRunExpiresAtMs,
} from "../chat-abort.js";
import { type ChatImageContent, parseMessageWithAttachments } from "../chat-attachments.js";
import { stripEnvelopeFromMessage, stripEnvelopeFromMessages } from "../chat-sanitize.js";
import {
  GATEWAY_CLIENT_CAPS,
  GATEWAY_CLIENT_MODES,
  GATEWAY_CLIENT_NAMES,
  hasGatewayClientCap,
} from "../protocol/client-info.js";
import {
  ErrorCodes,
  errorShape,
  formatValidationErrors,
  validateChatAbortParams,
  validateChatHistoryParams,
  validateChatInjectParams,
  validateChatSendParams,
} from "../protocol/index.js";
import { CHAT_SEND_SESSION_KEY_MAX_LENGTH } from "../protocol/schema/primitives.js";
import { getMaxChatHistoryMessagesBytes } from "../server-constants.js";
import {
  capArrayByJsonBytes,
  loadSessionEntry,
  readSessionMessages,
  resolveSessionModelRef,
} from "../session-utils.js";
import { formatForLog } from "../ws-log.js";
import { injectTimestamp, timestampOptsFromConfig } from "./agent-timestamp.js";
import { setGatewayDedupeEntry } from "./agent-wait-dedupe.js";
import { normalizeRpcAttachmentsToChatAttachments } from "./attachment-normalize.js";
import { appendInjectedAssistantMessageToTranscript } from "./chat-transcript-inject.js";
import type {
  GatewayRequestContext,
  GatewayRequestHandlerOptions,
  GatewayRequestHandlers,
} from "./types.js";
```

```

type TranscriptAppendResult = {
  ok: boolean;
  messageId?: string;
  message?: Record<string, unknown>;
  error?: string;
};

type AbortOrigin = "rpc" | "stop-command";

type AbortedPartialSnapshot = {
  runId: string;
  sessionId: string;
  text: string;
  abortOrigin: AbortOrigin;
};

const CHAT_HISTORY_TEXT_MAX_CHARS = 12_000;
const CHAT_HISTORY_MAX_SINGLE_MESSAGE_BYTES = 128 * 1024;
const CHAT_HISTORY_OVERSIZED_PLACEHOLDER = "[chat.history omitted: message too large]";
let chatHistoryPlaceholderEmitCount = 0;
const CHANNEL_AGNOSTIC_SESSION_SCOPES = new Set([
  "main",
  "direct",
  "dm",
  "group",
  "channel",
  "cron",
  "run",
  "subagent",
  "acp",
  "thread",
  "topic",
]);
const CHANNEL_SCOPED_SESSION_SHAPES = new Set(["direct", "dm", "group", "channel"]);

type ChatSendDeliveryEntry = {
  deliveryContext?: {
    channel?: string;
    to?: string;
    accountId?: string;
    threadId?: string | number;
  };
  lastChannel?: string;
  lastTo?: string;
  lastAccountId?: string;
  lastThreadId?: string | number;
```

```
};

type ChatSendOriginatingRoute = {
  originatingChannel: string;
  originatingTo?: string;
  accountId?: string;
  messageThreadId?: string | number;
  explicitDeliverRoute: boolean;
};

function resolveChatSendOriginatingRoute(params: {
  client?: { mode?: string | null; id?: string | null } | null;
  deliver?: boolean;
  entry?: ChatSendDeliveryEntry;
  hasConnectedClient?: boolean;
  mainKey?: string;
  sessionKey: string;
}): ChatSendOriginatingRoute {
  const shouldDeliverExternally = params.deliver === true;
  if (!shouldDeliverExternally) {
    return {
      originatingChannel: INTERNAL_MESSAGE_CHANNEL,
      explicitDeliverRoute: false,
    };
  }

  const routeChannelCandidate = normalizeMessageChannel(
    params.entry?.deliveryContext?.channel ?? params.entry?.lastChannel,
  );
  const routeToCandidate = params.entry?.deliveryContext?.to ?? params.entry?.lastTo;
  const routeAccountIdCandidate =
    params.entry?.deliveryContext?.accountId ?? params.entry?.lastAccountId ?? undefined;
  const routeThreadIdCandidate =
    params.entry?.deliveryContext?.threadId ?? params.entry?.lastThreadId;
  if (params.sessionKey.length > CHAT_SEND_SESSION_KEY_MAX_LENGTH) {
    return {
      originatingChannel: INTERNAL_MESSAGE_CHANNEL,
      explicitDeliverRoute: false,
    };
  }

  const parsedSessionKey = parseAgentSessionKey(params.sessionKey);
  const sessionScopeParts = (parsedSessionKey?.rest ?? params.sessionKey)
    .split(":", 3)
    .filter(Boolean);
  const sessionScopeHead = sessionScopeParts[0];
  const sessionChannelHint = normalizeMessageChannel(sessionScopeHead);
```

```
164  const normalizedSessionScopeHead = (sessionScopeHead ?? "").trim().toLowerCase();
165  const sessionPeerShapeCandidates = [sessionScopeParts[1], sessionScopeParts[2]]
166    .map((part) => (part ?? "").trim().toLowerCase())
167    .filter(Boolean);
168  const isChannelAgnosticSessionScope = CHANNEL_AGNOSTIC_SESSION_SCOPES.has(
169    normalizedSessionScopeHead,
170  );
171  const isChannelScopedSession = sessionPeerShapeCandidates.some((part) =>
172    CHANNEL_SCOPED_SESSION_SHAPES.has(part),
173  );
174  const hasLegacyChannelPeerShape =
175    !isChannelScopedSession &&
176    typeof sessionScopeParts[1] === "string" &&
177    sessionChannelHint === routeChannelCandidate;
178  const isFromWebchatClient = isWebchatClient(params.client);
179  const configuredMainKey = (params.mainKey ?? "main").trim().toLowerCase();
180  const isConfiguredMainSessionScope =
181    normalizedSessionScopeHead.length > 0 && normalizedSessionScopeHead ===
     configuredMainKey;
182
183  // Webchat/Control UI clients never inherit external delivery routes, even when
184  // accessing channel-scoped sessions. External routes are only for non-webchat
185  // clients where the session key explicitly encodes an external target.
186  // Preserve the old configured-main contract: any connected non-webchat client
187  // may inherit the last external route even when client metadata is absent.
188  const canInheritDeliverableRoute = Boolean(
189    !isFromWebchatClient &&
190    sessionChannelHint &&
191    sessionChannelHint !== INTERNAL_MESSAGE_CHANNEL &&
192    ((!isChannelAgnosticSessionScope && (isChannelScopedSession || hasLegacyChannelPeerShape
     )) ||
193      (isConfiguredMainSessionScope && params.hasConnectedClient)),
194  );
195  const hasDeliverableRoute =
196    canInheritDeliverableRoute &&
197    routeChannelCandidate &&
198    routeChannelCandidate !== INTERNAL_MESSAGE_CHANNEL &&
199    typeof routeToCandidate === "string" &&
200    routeToCandidate.trim().length > 0;
201
202  if (!hasDeliverableRoute) {
203    return {
204      originatingChannel: INTERNAL_MESSAGE_CHANNEL,
205      explicitDeliverRoute: false,
206    };
207  }
208
```

```
  return {
    originatingChannel: routeChannelCandidate,
    originatingTo: routeToCandidate,
    accountId: routeAccountIdCandidate,
    messageThreadId: routeThreadIdCandidate,
    explicitDeliverRoute: true,
  };
}

function stripDisallowedChatControlChars(message: string): string {
  let output = "";
  for (const char of message) {
    const code = char.charCodeAt(0);
    if (code === 9 || code === 10 || code === 13 || (code >= 32 && code !== 127)) {
      output += char;
    }
  }
  return output;
}

export function sanitizeChatSendMessageInput(
  message: string,
): { ok: true; message: string } | { ok: false; error: string } {
  const normalized = message.normalize("NFC");
  if (normalized.includes("\u0000")) {
    return { ok: false, error: "message must not contain null bytes" };
  }
  return { ok: true, message: stripDisallowedChatControlChars(normalized) };
}

function normalizeOptionalChatSystemReceipt(
  value: unknown,
): { ok: true; receipt?: string } | { ok: false; error: string } {
  if (value == null) {
    return { ok: true };
  }
  if (typeof value !== "string") {
    return { ok: false, error: "systemProvenanceReceipt must be a string" };
  }
  const sanitized = sanitizeChatSendMessageInput(value);
  if (!sanitized.ok) {
    return sanitized;
  }
  const receipt = sanitized.message.trim();
  return { ok: true, receipt: receipt || undefined };
}

```

```
function isAcpBridgeClient(client: GatewayRequestHandlerOptions["client"]): boolean {
  const info = client?.connect?.client;
  return (
    info?.id === GATEWAY_CLIENT_NAMES.CLI &&
    info?.mode === GATEWAY_CLIENT_MODES.CLI &&
    info?.displayName === "ACP" &&
    info?.version === "acp"
  );
}

function truncateChatHistoryText(text: string): { text: string; truncated: boolean } {
  if (text.length <= CHAT_HISTORY_TEXT_MAX_CHARS) {
    return { text, truncated: false };
  }
  return {
    text: `${text.slice(0, CHAT_HISTORY_TEXT_MAX_CHARS)}\n...(truncated)...`,
    truncated: true,
  };
}

function sanitizeChatHistoryContentBlock(block: unknown): { block: unknown; changed: boolean
     } {
  if (!block || typeof block !== "object") {
    return { block, changed: false };
  }
  const entry = { ...(block as Record<string, unknown>) };
  let changed = false;
  if (typeof entry.text === "string") {
    const stripped = stripInlineDirectiveTagsForDisplay(entry.text);
    const res = truncateChatHistoryText(stripped.text);
    entry.text = res.text;
    changed ||= stripped.changed || res.truncated;
  }
  if (typeof entry.partialJson === "string") {
    const res = truncateChatHistoryText(entry.partialJson);
    entry.partialJson = res.text;
    changed ||= res.truncated;
  }
  if (typeof entry.arguments === "string") {
    const res = truncateChatHistoryText(entry.arguments);
    entry.arguments = res.text;
    changed ||= res.truncated;
  }
  if (typeof entry.thinking === "string") {
    const res = truncateChatHistoryText(entry.thinking);
    entry.thinking = res.text;
    changed ||= res.truncated;
```

```
  }
  if ("thinkingSignature" in entry) {
    delete entry.thinkingSignature;
    changed = true;
  }
  const type = typeof entry.type === "string" ? entry.type : "";
  if (type === "image" && typeof entry.data === "string") {
    const bytes = Buffer.byteLength(entry.data, "utf8");
    delete entry.data;
    entry.omitted = true;
    entry.bytes = bytes;
    changed = true;
  }
  return { block: changed ? entry : block, changed };
}

function sanitizeChatHistoryMessage(message: unknown): { message: unknown; changed: boolean
    } {
  if (!message || typeof message !== "object") {
    return { message, changed: false };
  }
  const entry = { ...(message as Record<string, unknown>) };
  let changed = false;

  if ("details" in entry) {
    delete entry.details;
    changed = true;
  }
  if ("usage" in entry) {
    delete entry.usage;
    changed = true;
  }
  if ("cost" in entry) {
    delete entry.cost;
    changed = true;
  }

  if (typeof entry.content === "string") {
    const stripped = stripInlineDirectiveTagsForDisplay(entry.content);
    const res = truncateChatHistoryText(stripped.text);
    entry.content = res.text;
    changed ||= stripped.changed || res.truncated;
  } else if (Array.isArray(entry.content)) {
    const updated = entry.content.map((block) => sanitizeChatHistoryContentBlock(block));
    if (updated.some((item) => item.changed)) {
      entry.content = updated.map((item) => item.block);
      changed = true;
```

```
    }
  }

  if (typeof entry.text === "string") {
    const stripped = stripInlineDirectiveTagsForDisplay(entry.text);
    const res = truncateChatHistoryText(stripped.text);
    entry.text = res.text;
    changed ||= stripped.changed || res.truncated;
  }

  return { message: changed ? entry : message, changed };
}

/**
 * Extract the visible text from an assistant history message for silent-token checks.
 * Returns `undefined` for non-assistant messages or messages with no extractable text.
 * When `entry.text` is present it takes precedence over `entry.content` to avoid
 * dropping messages that carry real text alongside a stale `content: "NO_REPLY"`.
 */
function extractAssistantTextForSilentCheck(message: unknown): string | undefined {
  if (!message || typeof message !== "object") {
    return undefined;
  }
  const entry = message as Record<string, unknown>;
  if (entry.role !== "assistant") {
    return undefined;
  }
  if (typeof entry.text === "string") {
    return entry.text;
  }
  if (typeof entry.content === "string") {
    return entry.content;
  }
  if (!Array.isArray(entry.content) || entry.content.length === 0) {
    return undefined;
  }

  const texts: string[] = [];
  for (const block of entry.content) {
    if (!block || typeof block !== "object") {
      return undefined;
    }
    const typed = block as { type?: unknown; text?: unknown };
    if (typed.type !== "text" || typeof typed.text !== "string") {
      return undefined;
    }
    texts.push(typed.text);
```

```
  }
  return texts.length > 0 ? texts.join("\n") : undefined;
}

function sanitizeChatHistoryMessages(messages: unknown[]): unknown[] {
  if (messages.length === 0) {
    return messages;
  }
  let changed = false;
  const next: unknown[] = [];
  for (const message of messages) {
    const res = sanitizeChatHistoryMessage(message);
    changed ||= res.changed;
    // Drop assistant messages whose entire visible text is the silent reply token.
    const text = extractAssistantTextForSilentCheck(res.message);
    if (text !== undefined && isSilentReplyText(text, SILENT_REPLY_TOKEN)) {
      changed = true;
      continue;
    }
    next.push(res.message);
  }
  return changed ? next : messages;
}

function buildOversizedHistoryPlaceholder(message?: unknown): Record<string, unknown> {
  const role =
    message &&
    typeof message === "object" &&
    typeof (message as { role?: unknown }).role === "string"
      ? (message as { role: string }).role
      : "assistant";
  const timestamp =
    message &&
    typeof message === "object" &&
    typeof (message as { timestamp?: unknown }).timestamp === "number"
      ? (message as { timestamp: number }).timestamp
      : Date.now();
  return {
    role,
    timestamp,
    content: [{ type: "text", text: CHAT_HISTORY_OVERSIZED_PLACEHOLDER }],
    __openclaw: { truncated: true, reason: "oversized" },
  };
}

function replaceOversizedChatHistoryMessages(params: {
  messages: unknown[];
```

```
  maxSingleMessageBytes: number;
}): { messages: unknown[]; replacedCount: number } {
  const { messages, maxSingleMessageBytes } = params;
  if (messages.length === 0) {
    return { messages, replacedCount: 0 };
  }
  let replacedCount = 0;
  const next = messages.map((message) => {
    if (jsonUtf8Bytes(message) <= maxSingleMessageBytes) {
      return message;
    }
    replacedCount += 1;
    return buildOversizedHistoryPlaceholder(message);
  });
  return { messages: replacedCount > 0 ? next : messages, replacedCount };
}

function enforceChatHistoryFinalBudget(params: { messages: unknown[]; maxBytes: number }): {
  messages: unknown[];
  placeholderCount: number;
} {
  const { messages, maxBytes } = params;
  if (messages.length === 0) {
    return { messages, placeholderCount: 0 };
  }
  if (jsonUtf8Bytes(messages) <= maxBytes) {
    return { messages, placeholderCount: 0 };
  }
  const last = messages.at(-1);
  if (last && jsonUtf8Bytes([last]) <= maxBytes) {
    return { messages: [last], placeholderCount: 0 };
  }
  const placeholder = buildOversizedHistoryPlaceholder(last);
  if (jsonUtf8Bytes([placeholder]) <= maxBytes) {
    return { messages: [placeholder], placeholderCount: 1 };
  }
  return { messages: [], placeholderCount: 0 };
}

function resolveTranscriptPath(params: {
  sessionId: string;
  storePath: string | undefined;
  sessionFile?: string;
  agentId?: string;
}): string | null {
  const { sessionId, storePath, sessionFile, agentId } = params;
  if (!storePath && !sessionFile) {
```

```
    return null;
  }
  try {
    const sessionsDir = storePath ? path.dirname(storePath) : undefined;
    return resolveSessionFilePath(
      sessionId,
      sessionFile ? { sessionFile } : undefined,
      sessionsDir || agentId ? { sessionsDir, agentId } : undefined,
    );
  } catch {
    return null;
  }
}

function ensureTranscriptFile(params: { transcriptPath: string; sessionId: string }): {
  ok: boolean;
  error?: string;
} {
  if (fs.existsSync(params.transcriptPath)) {
    return { ok: true };
  }
  try {
    fs.mkdirSync(path.dirname(params.transcriptPath), { recursive: true });
    const header = {
      type: "session",
      version: CURRENT_SESSION_VERSION,
      id: params.sessionId,
      timestamp: new Date().toISOString(),
      cwd: process.cwd(),
    };
    fs.writeFileSync(params.transcriptPath, `${JSON.stringify(header)}\n`, {
      encoding: "utf-8",
      mode: 0o600,
    });
    return { ok: true };
  } catch (err) {
    return { ok: false, error: err instanceof Error ? err.message : String(err) };
  }
}

function transcriptHasIdempotencyKey(transcriptPath: string, idempotencyKey: string):
    boolean {
  try {
    const lines = fs.readFileSync(transcriptPath, "utf-8").split(/\r?\n/);
    for (const line of lines) {
      if (!line.trim()) {
        continue;
```

```
535       }
536       const parsed = JSON.parse(line) as { message?: { idempotencyKey?: unknown } };
537       if (parsed?.message?.idempotencyKey === idempotencyKey) {
538         return true;
539       }
540     }
541     return false;
542   } catch {
543     return false;
544   }
545 }
546
547 function appendAssistantTranscriptMessage(params: {
548   message: string;
549   label?: string;
550   sessionId: string;
551   storePath: string | undefined;
552   sessionFile?: string;
553   agentId?: string;
554   createIfMissing?: boolean;
555   idempotencyKey?: string;
556   abortMeta?: {
557     aborted: true;
558     origin: AbortOrigin;
559     runId: string;
560   };
561 }): TranscriptAppendResult {
562   const transcriptPath = resolveTranscriptPath({
563     sessionId: params.sessionId,
564     storePath: params.storePath,
565     sessionFile: params.sessionFile,
566     agentId: params.agentId,
567   });
568   if (!transcriptPath) {
569     return { ok: false, error: "transcript path not resolved" };
570   }
571
572   if (!fs.existsSync(transcriptPath)) {
573     if (!params.createIfMissing) {
574       return { ok: false, error: "transcript file not found" };
575     }
576     const ensured = ensureTranscriptFile({
577       transcriptPath,
578       sessionId: params.sessionId,
579     });
580     if (!ensured.ok) {
581       return { ok: false, error: ensured.error ?? "failed to create transcript file" };
```

```
    }
  }

  if (params.idempotencyKey && transcriptHasIdempotencyKey(transcriptPath, params.
    idempotencyKey)) {
    return { ok: true };
  }

  return appendInjectedAssistantMessageToTranscript({
    transcriptPath,
    message: params.message,
    label: params.label,
    idempotencyKey: params.idempotencyKey,
    abortMeta: params.abortMeta,
  });
}

function collectSessionAbortPartials(params: {
  chatAbortControllers: Map<string, ChatAbortControllerEntry>;
  chatRunBuffers: Map<string, string>;
  sessionKey: string;
  abortOrigin: AbortOrigin;
}): AbortedPartialSnapshot[] {
  const out: AbortedPartialSnapshot[] = [];
  for (const [runId, active] of params.chatAbortControllers) {
    if (active.sessionKey !== params.sessionKey) {
      continue;
    }
    const text = params.chatRunBuffers.get(runId);
    if (!text || !text.trim()) {
      continue;
    }
    out.push({
      runId,
      sessionId: active.sessionId,
      text,
      abortOrigin: params.abortOrigin,
    });
  }
  return out;
}

function persistAbortedPartials(params: {
  context: Pick<GatewayRequestContext, "logGateway">;
  sessionKey: string;
  snapshots: AbortedPartialSnapshot[];
}) {
```

```
  if (params.snapshots.length === 0) {
    return;
  }
  const { storePath, entry } = loadSessionEntry(params.sessionKey);
  for (const snapshot of params.snapshots) {
    const sessionId = entry?.sessionId ?? snapshot.sessionId ?? snapshot.runId;
    const appended = appendAssistantTranscriptMessage({
      message: snapshot.text,
      sessionId,
      storePath,
      sessionFile: entry?.sessionFile,
      createIfMissing: true,
      idempotencyKey: `${snapshot.runId}:assistant`,
      abortMeta: {
        aborted: true,
        origin: snapshot.abortOrigin,
        runId: snapshot.runId,
      },
    });
    if (!appended.ok) {
      params.context.logGateway.warn(
        `chat.abort transcript append failed: ${appended.error ?? "unknown error"}`,
      );
    }
  }
}

function createChatAbortOps(context: GatewayRequestContext): ChatAbortOps {
  return {
    chatAbortControllers: context.chatAbortControllers,
    chatRunBuffers: context.chatRunBuffers,
    chatDeltaSentAt: context.chatDeltaSentAt,
    chatAbortedRuns: context.chatAbortedRuns,
    removeChatRun: context.removeChatRun,
    agentRunSeq: context.agentRunSeq,
    broadcast: context.broadcast,
    nodeSendToSession: context.nodeSendToSession,
  };
}

function abortChatRunsForSessionKeyWithPartials(params: {
  context: GatewayRequestContext;
  ops: ChatAbortOps;
  sessionKey: string;
  abortOrigin: AbortOrigin;
  stopReason?: string;
}) {
```

```
675   const snapshots = collectSessionAbortPartials({
676     chatAbortControllers: params.context.chatAbortControllers,
677     chatRunBuffers: params.context.chatRunBuffers,
678     sessionKey: params.sessionKey,
679     abortOrigin: params.abortOrigin,
680   });
681   const res = abortChatRunsForSessionKey(params.ops, {
682     sessionKey: params.sessionKey,
683     stopReason: params.stopReason,
684   });
685   if (res.aborted) {
686     persistAbortedPartials({
687       context: params.context,
688       sessionKey: params.sessionKey,
689       snapshots,
690     });
691   }
692   return res;
693 }
694
695 function nextChatSeq(context: { agentRunSeq: Map<string, number> }, runId: string) {
696   const next = (context.agentRunSeq.get(runId) ?? 0) + 1;
697   context.agentRunSeq.set(runId, next);
698   return next;
699 }
700
701 function broadcastChatFinal(params: {
702   context: Pick<GatewayRequestContext, "broadcast" | "nodeSendToSession" | "agentRunSeq">;
703   runId: string;
704   sessionKey: string;
705   message?: Record<string, unknown>;
706 }) {
707   const seq = nextChatSeq({ agentRunSeq: params.context.agentRunSeq }, params.runId);
708   const strippedEnvelopeMessage = stripEnvelopeFromMessage(params.message) as
709     | Record<string, unknown>
710     | undefined;
711   const payload = {
712     runId: params.runId,
713     sessionKey: params.sessionKey,
714     seq,
715     state: "final" as const,
716     message: stripInlineDirectiveTagsFromMessageForDisplay(strippedEnvelopeMessage),
717   };
718   params.context.broadcast("chat", payload);
719   params.context.nodeSendToSession(params.sessionKey, "chat", payload);
720   params.context.agentRunSeq.delete(params.runId);
721 }
```

```

function broadcastChatError(params: {
  context: Pick<GatewayRequestContext, "broadcast" | "nodeSendToSession" | "agentRunSeq">;
  runId: string;
  sessionKey: string;
  errorMessage?: string;
}) {
  const seq = nextChatSeq({ agentRunSeq: params.context.agentRunSeq }, params.runId);
  const payload = {
    runId: params.runId,
    sessionKey: params.sessionKey,
    seq,
    state: "error" as const,
    errorMessage: params.errorMessage,
  };
  params.context.broadcast("chat", payload);
  params.context.nodeSendToSession(params.sessionKey, "chat", payload);
  params.context.agentRunSeq.delete(params.runId);
}

export const chatHandlers: GatewayRequestHandlers = {
  "chat.history": async ({ params, respond, context }) => {
    if (!validateChatHistoryParams(params)) {
      respond(
        false,
        undefined,
        errorShape(
          ErrorCodes.INVALID_REQUEST,
          `invalid chat.history params: ${formatValidationErrors(validateChatHistoryParams.errors)}`,
        ),
      );
      return;
    }
    const { sessionKey, limit } = params as {
      sessionKey: string;
      limit?: number;
    };
    const { cfg, storePath, entry } = loadSessionEntry(sessionKey);
    const sessionId = entry?.sessionId;
    const rawMessages =
      sessionId && storePath ? readSessionMessages(sessionId, storePath, entry?.sessionFile) : [];
    const hardMax = 1000;
    const defaultLimit = 200;
    const requested = typeof limit === "number" ? limit : defaultLimit;
    const max = Math.min(hardMax, requested);
```

```
767     const sliced = rawMessages.length > max ? rawMessages.slice(-max) : rawMessages;
768     const sanitized = stripEnvelopeFromMessages(sliced);
769     const normalized = sanitizeChatHistoryMessages(sanitized);
770     const maxHistoryBytes = getMaxChatHistoryMessagesBytes();
771     const perMessageHardCap = Math.min(CHAT_HISTORY_MAX_SINGLE_MESSAGE_BYTES,
      maxHistoryBytes);
772     const replaced = replaceOversizedChatHistoryMessages({
773       messages: normalized,
774       maxSingleMessageBytes: perMessageHardCap,
775     });
776     const capped = capArrayByJsonBytes(replaced.messages, maxHistoryBytes).items;
777     const bounded = enforceChatHistoryFinalBudget({ messages: capped, maxBytes:
      maxHistoryBytes });
778     const placeholderCount = replaced.replacedCount + bounded.placeholderCount;
779     if (placeholderCount > 0) {
780       chatHistoryPlaceholderEmitCount += placeholderCount;
781       context.logGateway.debug(
782         `chat.history omitted oversized payloads placeholders=${placeholderCount} total=${
      chatHistoryPlaceholderEmitCount}`,
783       );
784     }
785     let thinkingLevel = entry?.thinkingLevel;
786     if (!thinkingLevel) {
787       const sessionAgentId = resolveSessionAgentId({ sessionKey, config: cfg });
788       const { provider, model } = resolveSessionModelRef(cfg, entry, sessionAgentId);
789       const catalog = await context.loadGatewayModelCatalog();
790       thinkingLevel = resolveThinkingDefault({
791         cfg,
792         provider,
793         model,
794         catalog,
795       });
796     }
797     const verboseLevel = entry?.verboseLevel ?? cfg.agents?.defaults?.verboseDefault;
798     respond(true, {
799       sessionKey,
800       sessionId,
801       messages: bounded.messages,
802       thinkingLevel,
803       verboseLevel,
804     });
805   },
806   "chat.abort": ({ params, respond, context }) => {
807     if (!validateChatAbortParams(params)) {
808       respond(
809         false,
810         undefined,
```

```
      errorShape(
        ErrorCodes.INVALID_REQUEST,
        `invalid chat.abort params: ${formatValidationErrors(validateChatAbortParams.
errors)}`,
      ),
    );
    return;
  }
  const { sessionKey: rawSessionKey, runId } = params as {
    sessionKey: string;
    runId?: string;
  };

  const ops = createChatAbortOps(context);

  if (!runId) {
    const res = abortChatRunsForSessionKeyWithPartials({
      context,
      ops,
      sessionKey: rawSessionKey,
      abortOrigin: "rpc",
      stopReason: "rpc",
    });
    respond(true, { ok: true, aborted: res.aborted, runIds: res.runIds });
    return;
  }

  const active = context.chatAbortControllers.get(runId);
  if (!active) {
    respond(true, { ok: true, aborted: false, runIds: [] });
    return;
  }
  if (active.sessionKey !== rawSessionKey) {
    respond(
      false,
      undefined,
      errorShape(ErrorCodes.INVALID_REQUEST, "runId does not match sessionKey"),
    );
    return;
  }

  const partialText = context.chatRunBuffers.get(runId);
  const res = abortChatRunById(ops, {
    runId,
    sessionKey: rawSessionKey,
    stopReason: "rpc",
  });
```

```
857     if (res.aborted && partialText && partialText.trim()) {
858       persistAbortedPartials({
859         context,
860         sessionKey: rawSessionKey,
861         snapshots: [
862           {
863             runId,
864             sessionId: active.sessionId,
865             text: partialText,
866             abortOrigin: "rpc",
867           },
868         ],
869       });
870     }
871     respond(true, {
872       ok: true,
873       aborted: res.aborted,
874       runIds: res.aborted ? [runId] : [],
875     });
876   },
877   "chat.send": async ({ params, respond, context, client }) => {
878     if (!validateChatSendParams(params)) {
879       respond(
880         false,
881         undefined,
882         errorShape(
883           ErrorCodes.INVALID_REQUEST,
884           `invalid chat.send params: ${formatValidationErrors(validateChatSendParams.errors)
      }`,
885         ),
886       );
887       return;
888     }
889     const p = params as {
890       sessionKey: string;
891       message: string;
892       thinking?: string;
893       deliver?: boolean;
894       attachments?: Array<{
895         type?: string;
896         mimeType?: string;
897         fileName?: string;
898         content?: unknown;
899       }>;
900       timeoutMs?: number;
901       systemInputProvenance?: InputProvenance;
902       systemProvenanceReceipt?: string;
```

```
903       idempotencyKey: string;
904     };
905     if ((p.systemInputProvenance || p.systemProvenanceReceipt) && !isAcpBridgeClient(client)
        ) {
906       respond(
907         false,
908         undefined,
909         errorShape(
910           ErrorCodes.INVALID_REQUEST,
911           "system provenance fields are reserved for the ACP bridge",
912         ),
913       );
914       return;
915     }
916     const sanitizedMessageResult = sanitizeChatSendMessageInput(p.message);
917     if (!sanitizedMessageResult.ok) {
918       respond(
919         false,
920         undefined,
921         errorShape(ErrorCodes.INVALID_REQUEST, sanitizedMessageResult.error),
922       );
923       return;
924     }
925     const systemReceiptResult = normalizeOptionalChatSystemReceipt(p.systemProvenanceReceipt
        );
926     if (!systemReceiptResult.ok) {
927       respond(false, undefined, errorShape(ErrorCodes.INVALID_REQUEST, systemReceiptResult.
        error));
928       return;
929     }
930     const inboundMessage = sanitizedMessageResult.message;
931     const systemInputProvenance = normalizeInputProvenance(p.systemInputProvenance);
932     const systemProvenanceReceipt = systemReceiptResult.receipt;
933     const stopCommand = isChatStopCommandText(inboundMessage);
934     const normalizedAttachments = normalizeRpcAttachmentsToChatAttachments(p.attachments);
935     const rawMessage = inboundMessage.trim();
936     if (!rawMessage && normalizedAttachments.length === 0) {
937       respond(
938         false,
939         undefined,
940         errorShape(ErrorCodes.INVALID_REQUEST, "message or attachment required"),
941       );
942       return;
943     }
944     let parsedMessage = inboundMessage;
945     let parsedImages: ChatImageContent[] = [];
946     if (normalizedAttachments.length > 0) {
```

```
      try {
        const parsed = await parseMessageWithAttachments(inboundMessage,
    normalizedAttachments, {
          maxBytes: 5_000_000,
          log: context.logGateway,
        });
        parsedMessage = parsed.message;
        parsedImages = parsed.images;
      } catch (err) {
        respond(false, undefined, errorShape(ErrorCodes.INVALID_REQUEST, String(err)));
        return;
      }
    }
    const rawSessionKey = p.sessionKey;
    const { cfg, entry, canonicalKey: sessionKey } = loadSessionEntry(rawSessionKey);
    const timeoutMs = resolveAgentTimeoutMs({
      cfg,
      overrideMs: p.timeoutMs,
    });
    const now = Date.now();
    const clientRunId = p.idempotencyKey;

    const sendPolicy = resolveSendPolicy({
      cfg,
      entry,
      sessionKey,
      channel: entry?.channel,
      chatType: entry?.chatType,
    });
    if (sendPolicy === "deny") {
      respond(
        false,
        undefined,
        errorShape(ErrorCodes.INVALID_REQUEST, "send blocked by session policy"),
      );
      return;
    }

    if (stopCommand) {
      const res = abortChatRunsForSessionKeyWithPartials({
        context,
        ops: createChatAbortOps(context),
        sessionKey: rawSessionKey,
        abortOrigin: "stop-command",
        stopReason: "stop",
      });
      respond(true, { ok: true, aborted: res.aborted, runIds: res.runIds });
```

```
      return;
    }

    const cached = context.dedupe.get(`chat:${clientRunId}`);
    if (cached) {
      respond(cached.ok, cached.payload, cached.error, {
        cached: true,
      });
      return;
    }

    const activeExisting = context.chatAbortControllers.get(clientRunId);
    if (activeExisting) {
      respond(true, { runId: clientRunId, status: "in_flight" as const }, undefined, {
        cached: true,
        runId: clientRunId,
      });
      return;
    }

    try {
      const abortController = new AbortController();
      context.chatAbortControllers.set(clientRunId, {
        controller: abortController,
        sessionId: entry?.sessionId ?? clientRunId,
        sessionKey: rawSessionKey,
        startedAtMs: now,
        expiresAtMs: resolveChatRunExpiresAtMs({ now, timeoutMs }),
      });
      const ackPayload = {
        runId: clientRunId,
        status: "started" as const,
      };
      respond(true, ackPayload, undefined, { runId: clientRunId });

      const trimmedMessage = parsedMessage.trim();
      const injectThinking = Boolean(
        p.thinking && trimmedMessage && !trimmedMessage.startsWith("/"),
      );
      const commandBody = injectThinking ? `/think ${p.thinking} ${parsedMessage}` :
    parsedMessage;
      const messageForAgent = systemProvenanceReceipt
        ? [systemProvenanceReceipt, parsedMessage].filter(Boolean).join("\n\n")
        : parsedMessage;
      const clientInfo = client?.connect?.client;
      const {
        originatingChannel,
```

```
1039        originatingTo,
1040        accountId,
1041        messageThreadId,
1042        explicitDeliverRoute,
1043      } = resolveChatSendOriginatingRoute({
1044        client: clientInfo,
1045        deliver: p.deliver,
1046        entry,
1047        hasConnectedClient: client?.connect !== undefined,
1048        mainKey: cfg.session?.mainKey,
1049        sessionKey,
1050      });
1051      // Inject timestamp so agents know the current date/time.
1052      // Only BodyForAgent gets the timestamp — Body stays raw for UI display.
1053      // See: https://github.com/moltbot/moltbot/issues/3658
1054      const stampedMessage = injectTimestamp(messageForAgent, timestampOptsFromConfig(cfg));
1055
1056      const ctx: MsgContext = {
1057        Body: messageForAgent,
1058        BodyForAgent: stampedMessage,
1059        BodyForCommands: commandBody,
1060        RawBody: parsedMessage,
1061        CommandBody: commandBody,
1062        InputProvenance: systemInputProvenance,
1063        SessionKey: sessionKey,
1064        Provider: INTERNAL_MESSAGE_CHANNEL,
1065        Surface: INTERNAL_MESSAGE_CHANNEL,
1066        OriginatingChannel: originatingChannel,
1067        OriginatingTo: originatingTo,
1068        ExplicitDeliverRoute: explicitDeliverRoute,
1069        AccountId: accountId,
1070        MessageThreadId: messageThreadId,
1071        ChatType: "direct",
1072        CommandAuthorized: true,
1073        MessageSid: clientRunId,
1074        SenderId: clientInfo?.id,
1075        SenderName: clientInfo?.displayName,
1076        SenderUsername: clientInfo?.displayName,
1077        GatewayClientScopes: client?.connect?.scopes,
1078      };
1079
1080      const agentId = resolveSessionAgentId({
1081        sessionKey,
1082        config: cfg,
1083      });
1084      const { onModelSelected, ...prefixOptions } = createReplyPrefixOptions({
1085        cfg,
```

```
1086        agentId,
1087        channel: INTERNAL_MESSAGE_CHANNEL,
1088      });
1089      const finalReplyParts: string[] = [];
1090      const dispatcher = createReplyDispatcher({
1091        ...prefixOptions,
1092        onError: (err) => {
1093          context.logGateway.warn(`webchat dispatch failed: ${formatForLog(err)}`);
1094        },
1095        deliver: async (payload, info) => {
1096          if (info.kind !== "final") {
1097            return;
1098          }
1099          const text = payload.text?.trim() ?? "";
1100          if (!text) {
1101            return;
1102          }
1103          finalReplyParts.push(text);
1104        },
1105      });
1106
1107      let agentRunStarted = false;
1108      void dispatchInboundMessage({
1109        ctx,
1110        cfg,
1111        dispatcher,
1112        replyOptions: {
1113          runId: clientRunId,
1114          abortSignal: abortController.signal,
1115          images: parsedImages.length > 0 ? parsedImages : undefined,
1116          onAgentRunStart: (runId) => {
1117            agentRunStarted = true;
1118            const connId = typeof client?.connId === "string" ? client.connId : undefined;
1119            const wantsToolEvents = hasGatewayClientCap(
1120              client?.connect?.caps,
1121              GATEWAY_CLIENT_CAPS.TOOL_EVENTS,
1122            );
1123            if (connId && wantsToolEvents) {
1124              context.registerToolEventRecipient(runId, connId);
1125              // Register for any other active runs *in the same session* so
1126              // late-joining clients (e.g. page refresh mid-response) receive
1127              // in-progress tool events without leaking cross-session data.
1128              for (const [activeRunId, active] of context.chatAbortControllers) {
1129                if (activeRunId !== runId && active.sessionKey === p.sessionKey) {
1130                  context.registerToolEventRecipient(activeRunId, connId);
1131                }
1132              }
```

```
1133            }
1134          },
1135          onModelSelected,
1136        },
1137      })
1138        .then(() => {
1139          if (!agentRunStarted) {
1140            const combinedReply = finalReplyParts
1141              .map((part) => part.trim())
1142              .filter(Boolean)
1143              .join("\n\n")
1144              .trim();
1145            let message: Record<string, unknown> | undefined;
1146            if (combinedReply) {
1147              const { storePath: latestStorePath, entry: latestEntry } =
1148                loadSessionEntry(sessionKey);
1149              const sessionId = latestEntry?.sessionId ?? entry?.sessionId ?? clientRunId;
1150              const appended = appendAssistantTranscriptMessage({
1151                message: combinedReply,
1152                sessionId,
1153                storePath: latestStorePath,
1154                sessionFile: latestEntry?.sessionFile,
1155                agentId,
1156                createIfMissing: true,
1157              });
1158              if (appended.ok) {
1159                message = appended.message;
1160              } else {
1161                context.logGateway.warn(
1162                  `webchat transcript append failed: ${appended.error ?? "unknown error"}`,
1163                );
1164                const now = Date.now();
1165                message = {
1166                  role: "assistant",
1167                  content: [{ type: "text", text: combinedReply }],
1168                  timestamp: now,
1169                  // Keep this compatible with Pi stopReason enums even though this message
    isn't
1170                  // persisted to the transcript due to the append failure.
1171                  stopReason: "stop",
1172                  usage: { input: 0, output: 0, totalTokens: 0 },
1173                };
1174              }
1175            }
1176            broadcastChatFinal({
1177              context,
1178              runId: clientRunId,
```

```
1179            sessionKey: rawSessionKey,
1180            message,
1181          });
1182        }
1183        setGatewayDedupeEntry({
1184          dedupe: context.dedupe,
1185          key: `chat:${clientRunId}`,
1186          entry: {
1187            ts: Date.now(),
1188            ok: true,
1189            payload: { runId: clientRunId, status: "ok" as const },
1190          },
1191        });
1192        queueExternalMemoryKernelRun({
1193          source: "gateway-chat",
1194          status: "success",
1195          sessionKey: p.sessionKey,
1196        });
1197      })
1198      .catch((err) => {
1199        const error = errorShape(ErrorCodes.UNAVAILABLE, String(err));
1200        setGatewayDedupeEntry({
1201          dedupe: context.dedupe,
1202          key: `chat:${clientRunId}`,
1203          entry: {
1204            ts: Date.now(),
1205            ok: false,
1206            payload: {
1207              runId: clientRunId,
1208              status: "error" as const,
1209              summary: String(err),
1210            },
1211            error,
1212          },
1213        });
1214        broadcastChatError({
1215          context,
1216          runId: clientRunId,
1217          sessionKey: rawSessionKey,
1218          errorMessage: String(err),
1219        });
1220        queueExternalMemoryKernelRun({
1221          source: "gateway-chat",
1222          status: "error",
1223          sessionKey: p.sessionKey,
1224        });
1225      })
```

```
1226        .finally(() => {
1227          context.chatAbortControllers.delete(clientRunId);
1228        });
1229    } catch (err) {
1230      const error = errorShape(ErrorCodes.UNAVAILABLE, String(err));
1231      const payload = {
1232        runId: clientRunId,
1233        status: "error" as const,
1234        summary: String(err),
1235      };
1236      setGatewayDedupeEntry({
1237        dedupe: context.dedupe,
1238        key: `chat:${clientRunId}`,
1239        entry: {
1240          ts: Date.now(),
1241          ok: false,
1242          payload,
1243          error,
1244        },
1245      });
1246      respond(false, payload, error, {
1247        runId: clientRunId,
1248        error: formatForLog(err),
1249      });
1250    }
1251  },
1252  "chat.inject": async ({ params, respond, context }) => {
1253    if (!validateChatInjectParams(params)) {
1254      respond(
1255        false,
1256        undefined,
1257        errorShape(
1258          ErrorCodes.INVALID_REQUEST,
1259          `invalid chat.inject params: ${formatValidationErrors(validateChatInjectParams.
    errors)}`,
1260        ),
1261      );
1262      return;
1263    }
1264    const p = params as {
1265      sessionKey: string;
1266      message: string;
1267      label?: string;
1268    };
1269
1270    // Load session to find transcript file
1271    const rawSessionKey = p.sessionKey;
```

```
    const { cfg, storePath, entry } = loadSessionEntry(rawSessionKey);
    const sessionId = entry?.sessionId;
    if (!sessionId || !storePath) {
      respond(false, undefined, errorShape(ErrorCodes.INVALID_REQUEST, "session not found"))
    ;
      return;
    }

    const appended = appendAssistantTranscriptMessage({
      message: p.message,
      label: p.label,
      sessionId,
      storePath,
      sessionFile: entry?.sessionFile,
      agentId: resolveSessionAgentId({ sessionKey: rawSessionKey, config: cfg }),
      createIfMissing: false,
    });
    if (!appended.ok || !appended.messageId || !appended.message) {
      respond(
        false,
        undefined,
        errorShape(
          ErrorCodes.UNAVAILABLE,
          `failed to write transcript: ${appended.error ?? "unknown error"}`,
        ),
      );
      return;
    }

    // Broadcast to webchat for immediate UI update
    const chatPayload = {
      runId: `inject-${appended.messageId}`,
      sessionKey: rawSessionKey,
      seq: 0,
      state: "final" as const,
      message: stripInlineDirectiveTagsFromMessageForDisplay(
        stripEnvelopeFromMessage(appended.message) as Record<string, unknown>,
      ),
    };
    context.broadcast("chat", chatPayload);
    context.nodeSendToSession(rawSessionKey, "chat", chatPayload);

    respond(true, { ok: true, messageId: appended.messageId });
  },
};
```

B.7　src/gateway/server-methods/agent.ts

作用说明

真实 agent RPC 主路径，当前已接入 `gateway-agent` sidecar 调用。

完整内容

```
import { randomUUID } from "node:crypto";
import { listAgentIds } from "../../agents/agent-scope.js";
import type { AgentInternalEvent } from "../../agents/internal-events.js";
import {
  normalizeSpawnedRunMetadata,
  resolveIngressWorkspaceOverrideForSpawnedRun,
} from "../../agents/spawned-context.js";
import { buildBareSessionResetPrompt } from "../../auto-reply/reply/session-reset-prompt.js
    ";
import { agentCommandFromIngress } from "../../commands/agent.js";
import { loadConfig } from "../../config/config.js";
import {
  mergeSessionEntry,
  resolveAgentIdFromSessionKey,
  resolveExplicitAgentSessionKey,
  resolveAgentMainSessionKey,
  type SessionEntry,
  updateSessionStore,
} from "../../config/sessions.js";
import { registerAgentRunContext } from "../../infra/agent-events.js";
import {
  resolveAgentDeliveryPlan,
  resolveAgentOutboundTarget,
} from "../../infra/outbound/agent-delivery.js";
import { resolveMessageChannelSelection } from "../../infra/outbound/channel-selection.js";
import { queueExternalMemoryKernelRun } from "../../infra/external-memory-kernel.js";
import { classifySessionKeyShape, normalizeAgentId } from "../../routing/session-key.js";
import { defaultRuntime } from "../../runtime.js";
import { normalizeInputProvenance, type InputProvenance } from "../../sessions/input-
    provenance.js";
import { resolveSendPolicy } from "../../sessions/send-policy.js";
import { normalizeSessionDeliveryFields } from "../../utils/delivery-context.js";
import {
  INTERNAL_MESSAGE_CHANNEL,
  isDeliverableMessageChannel,
  isGatewayMessageChannel,
  normalizeMessageChannel,
} from "../../utils/message-channel.js";
import { resolveAssistantIdentity } from "../assistant-identity.js";
```

```
import { parseMessageWithAttachments } from "../chat-attachments.js";
import { resolveAssistantAvatarUrl } from "../control-ui-shared.js";
import { ADMIN_SCOPE } from "../method-scopes.js";
import { GATEWAY_CLIENT_CAPS, hasGatewayClientCap } from "../protocol/client-info.js";
import {
  ErrorCodes,
  errorShape,
  formatValidationErrors,
  validateAgentIdentityParams,
  validateAgentParams,
  validateAgentWaitParams,
} from "../protocol/index.js";
import { performGatewaySessionReset } from "../session-reset-service.js";
import {
  canonicalizeSpawnedByForAgent,
  loadSessionEntry,
  pruneLegacyStoreKeys,
  resolveGatewaySessionStoreTarget,
} from "../session-utils.js";
import { formatForLog } from "../ws-log.js";
import { waitForAgentJob } from "./agent-job.js";
import { injectTimestamp, timestampOptsFromConfig } from "./agent-timestamp.js";
import {
  readTerminalSnapshotFromGatewayDedupe,
  setGatewayDedupeEntry,
  type AgentWaitTerminalSnapshot,
  waitForTerminalGatewayDedupe,
} from "./agent-wait-dedupe.js";
import { normalizeRpcAttachmentsToChatAttachments } from "./attachment-normalize.js";
import type { GatewayRequestHandlerOptions, GatewayRequestHandlers } from "./types.js";

const RESET_COMMAND_RE = /^\/(new|reset)(?:\s+([\s\S]*))?$/i;

function resolveSenderIsOwnerFromClient(client: GatewayRequestHandlerOptions["client"]):
    boolean {
  const scopes = Array.isArray(client?.connect?.scopes) ? client.connect.scopes : [];
  return scopes.includes(ADMIN_SCOPE);
}

async function runSessionResetFromAgent(params: {
  key: string;
  reason: "new" | "reset";
}): Promise<
  | { ok: true; key: string; sessionId?: string }
  | { ok: false; error: ReturnType<typeof errorShape> }
> {
  const result = await performGatewaySessionReset({
```

```
    key: params.key,
    reason: params.reason,
    commandSource: "gateway:agent",
  });
  if (!result.ok) {
    return result;
  }
  return {
    ok: true,
    key: result.key,
    sessionId: result.entry.sessionId,
  };
}

function dispatchAgentRunFromGateway(params: {
  ingressOpts: Parameters<typeof agentCommandFromIngress>[0];
  runId: string;
  idempotencyKey: string;
  respond: GatewayRequestHandlerOptions["respond"];
  context: GatewayRequestHandlerOptions["context"];
}) {
  void agentCommandFromIngress(params.ingressOpts, defaultRuntime, params.context.deps)
    .then((result) => {
      const payload = {
        runId: params.runId,
        status: "ok" as const,
        summary: "completed",
        result,
      };
      setGatewayDedupeEntry({
        dedupe: params.context.dedupe,
        key: `agent:${params.idempotencyKey}`,
        entry: {
          ts: Date.now(),
          ok: true,
          payload,
        },
      });
      queueExternalMemoryKernelRun({
        source: "gateway-agent",
        status: "success",
        sessionKey: params.ingressOpts.sessionKey,
      });
      // Send a second res frame (same id) so TS clients with expectFinal can wait.
      // Swift clients will typically treat the first res as the result and ignore this.
      params.respond(true, payload, undefined, { runId: params.runId });
    })
```

```
    .catch((err) => {
      const error = errorShape(ErrorCodes.UNAVAILABLE, String(err));
      const payload = {
        runId: params.runId,
        status: "error" as const,
        summary: String(err),
      };
      setGatewayDedupeEntry({
        dedupe: params.context.dedupe,
        key: `agent:${params.idempotencyKey}`,
        entry: {
          ts: Date.now(),
          ok: false,
          payload,
          error,
        },
      });
      queueExternalMemoryKernelRun({
        source: "gateway-agent",
        status: "error",
        sessionKey: params.ingressOpts.sessionKey,
      });
      params.respond(false, payload, error, {
        runId: params.runId,
        error: formatForLog(err),
      });
    });
}

export const agentHandlers: GatewayRequestHandlers = {
  agent: async ({ params, respond, context, client, isWebchatConnect }) => {
    const p = params;
    if (!validateAgentParams(p)) {
      respond(
        false,
        undefined,
        errorShape(
          ErrorCodes.INVALID_REQUEST,
          `invalid agent params: ${formatValidationErrors(validateAgentParams.errors)}`,
        ),
      );
      return;
    }
    const request = p as {
      message: string;
      agentId?: string;
      to?: string;
```

```
178       replyTo?: string;
179       sessionId?: string;
180       sessionKey?: string;
181       thinking?: string;
182       deliver?: boolean;
183       attachments?: Array<{
184         type?: string;
185         mimeType?: string;
186         fileName?: string;
187         content?: unknown;
188       }>;
189       channel?: string;
190       replyChannel?: string;
191       accountId?: string;
192       replyAccountId?: string;
193       threadId?: string;
194       groupId?: string;
195       groupChannel?: string;
196       groupSpace?: string;
197       lane?: string;
198       extraSystemPrompt?: string;
199       internalEvents?: AgentInternalEvent[];
200       idempotencyKey: string;
201       timeout?: number;
202       bestEffortDeliver?: boolean;
203       label?: string;
204       spawnedBy?: string;
205       inputProvenance?: InputProvenance;
206       workspaceDir?: string;
207     };
208     const senderIsOwner = resolveSenderIsOwnerFromClient(client);
209     const cfg = loadConfig();
210     const idem = request.idempotencyKey;
211     const normalizedSpawned = normalizeSpawnedRunMetadata({
212       spawnedBy: request.spawnedBy,
213       groupId: request.groupId,
214       groupChannel: request.groupChannel,
215       groupSpace: request.groupSpace,
216       workspaceDir: request.workspaceDir,
217     });
218     let resolvedGroupId: string | undefined = normalizedSpawned.groupId;
219     let resolvedGroupChannel: string | undefined = normalizedSpawned.groupChannel;
220     let resolvedGroupSpace: string | undefined = normalizedSpawned.groupSpace;
221     let spawnedByValue = normalizedSpawned.spawnedBy;
222     const inputProvenance = normalizeInputProvenance(request.inputProvenance);
223     const cached = context.dedupe.get(`agent:${idem}`);
224     if (cached) {
```

```
      respond(cached.ok, cached.payload, cached.error, {
        cached: true,
      });
      return;
    }
    const normalizedAttachments = normalizeRpcAttachmentsToChatAttachments(request.attachments);
    const requestedBestEffortDeliver =
      typeof request.bestEffortDeliver === "boolean" ? request.bestEffortDeliver : undefined;

    let message = (request.message ?? "").trim();
    let images: Array<{ type: "image"; data: string; mimeType: string }> = [];
    if (normalizedAttachments.length > 0) {
      try {
        const parsed = await parseMessageWithAttachments(message, normalizedAttachments, {
          maxBytes: 5_000_000,
          log: context.logGateway,
        });
        message = parsed.message.trim();
        images = parsed.images;
      } catch (err) {
        respond(false, undefined, errorShape(ErrorCodes.INVALID_REQUEST, String(err)));
        return;
      }
    }

    const isKnownGatewayChannel = (value: string): boolean => isGatewayMessageChannel(value);
    const channelHints = [request.channel, request.replyChannel]
      .filter((value): value is string => typeof value === "string")
      .map((value) => value.trim())
      .filter(Boolean);
    for (const rawChannel of channelHints) {
      const normalized = normalizeMessageChannel(rawChannel);
      if (normalized && normalized !== "last" && !isKnownGatewayChannel(normalized)) {
        respond(
          false,
          undefined,
          errorShape(
            ErrorCodes.INVALID_REQUEST,
            `invalid agent params: unknown channel: ${String(normalized)}`,
          ),
        );
        return;
      }
    }
```

```
269
270     const agentIdRaw = typeof request.agentId === "string" ? request.agentId.trim() : "";
271     const agentId = agentIdRaw ? normalizeAgentId(agentIdRaw) : undefined;
272     if (agentId) {
273       const knownAgents = listAgentIds(cfg);
274       if (!knownAgents.includes(agentId)) {
275         respond(
276           false,
277           undefined,
278           errorShape(
279             ErrorCodes.INVALID_REQUEST,
280             `invalid agent params: unknown agent id "${request.agentId}"`,
281           ),
282         );
283         return;
284       }
285     }
286
287     const requestedSessionKeyRaw =
288       typeof request.sessionKey === "string" && request.sessionKey.trim()
289         ? request.sessionKey.trim()
290         : undefined;
291     if (
292       requestedSessionKeyRaw &&
293       classifySessionKeyShape(requestedSessionKeyRaw) === "malformed_agent"
294     ) {
295       respond(
296         false,
297         undefined,
298         errorShape(
299           ErrorCodes.INVALID_REQUEST,
300           `invalid agent params: malformed session key "${requestedSessionKeyRaw}"`,
301         ),
302       );
303       return;
304     }
305     let requestedSessionKey =
306       requestedSessionKeyRaw ??
307       resolveExplicitAgentSessionKey({
308         cfg,
309         agentId,
310       });
311     if (agentId && requestedSessionKeyRaw) {
312       const sessionAgentId = resolveAgentIdFromSessionKey(requestedSessionKeyRaw);
313       if (sessionAgentId !== agentId) {
314         respond(
315           false,
```

```
316         undefined,
317         errorShape(
318           ErrorCodes.INVALID_REQUEST,
319           `invalid agent params: agent "${request.agentId}" does not match session key
    agent "${sessionAgentId}"`,
320         ),
321       );
322       return;
323     }
324   }
325   let resolvedSessionId = request.sessionId?.trim() || undefined;
326   let sessionEntry: SessionEntry | undefined;
327   let bestEffortDeliver = requestedBestEffortDeliver ?? false;
328   let cfgForAgent: ReturnType<typeof loadConfig> | undefined;
329   let resolvedSessionKey = requestedSessionKey;
330   let skipTimestampInjection = false;
331
332   const resetCommandMatch = message.match(RESET_COMMAND_RE);
333   if (resetCommandMatch && requestedSessionKey) {
334     const resetReason = resetCommandMatch[1]?.toLowerCase() === "new" ? "new" : "reset";
335     const resetResult = await runSessionResetFromAgent({
336       key: requestedSessionKey,
337       reason: resetReason,
338     });
339     if (!resetResult.ok) {
340       respond(false, undefined, resetResult.error);
341       return;
342     }
343     requestedSessionKey = resetResult.key;
344     resolvedSessionId = resetResult.sessionId ?? resolvedSessionId;
345     const postResetMessage = resetCommandMatch[2]?.trim() ?? "";
346     if (postResetMessage) {
347       message = postResetMessage;
348     } else {
349       // Keep bare /new and /reset behavior aligned with chat.send:
350       // reset first, then run a fresh-session greeting prompt in-place.
351       // Date is embedded in the prompt so agents read the correct daily
352       // memory files; skip further timestamp injection to avoid duplication.
353       message = buildBareSessionResetPrompt(cfg);
354       skipTimestampInjection = true;
355     }
356   }
357
358   // Inject timestamp into user-authored messages that don't already have one.
359   // Channel messages (Discord, Telegram, etc.) get timestamps via envelope
360   // formatting in a separate code path — they never reach this handler.
361   // See: https://github.com/moltbot/moltbot/issues/3658
```

```
362     if (!skipTimestampInjection) {
363       message = injectTimestamp(message, timestampOptsFromConfig(cfg));
364     }
365
366     if (requestedSessionKey) {
367       const { cfg, storePath, entry, canonicalKey } = loadSessionEntry(requestedSessionKey);
368       cfgForAgent = cfg;
369       const now = Date.now();
370       const sessionId = entry?.sessionId ?? randomUUID();
371       const labelValue = request.label?.trim() || entry?.label;
372       const sessionAgent = resolveAgentIdFromSessionKey(canonicalKey);
373       spawnedByValue = canonicalizeSpawnedByForAgent(
374         cfg,
375         sessionAgent,
376         spawnedByValue || entry?.spawnedBy,
377       );
378       let inheritedGroup:
379         | { groupId?: string; groupChannel?: string; groupSpace?: string }
380         | undefined;
381       if (spawnedByValue && (!resolvedGroupId || !resolvedGroupChannel || !
    resolvedGroupSpace)) {
382         try {
383           const parentEntry = loadSessionEntry(spawnedByValue)?.entry;
384           inheritedGroup = {
385             groupId: parentEntry?.groupId,
386             groupChannel: parentEntry?.groupChannel,
387             groupSpace: parentEntry?.space,
388           };
389         } catch {
390           inheritedGroup = undefined;
391         }
392       }
393       resolvedGroupId = resolvedGroupId || inheritedGroup?.groupId;
394       resolvedGroupChannel = resolvedGroupChannel || inheritedGroup?.groupChannel;
395       resolvedGroupSpace = resolvedGroupSpace || inheritedGroup?.groupSpace;
396       const deliveryFields = normalizeSessionDeliveryFields(entry);
397       const nextEntryPatch: SessionEntry = {
398         sessionId,
399         updatedAt: now,
400         thinkingLevel: entry?.thinkingLevel,
401         verboseLevel: entry?.verboseLevel,
402         reasoningLevel: entry?.reasoningLevel,
403         systemSent: entry?.systemSent,
404         sendPolicy: entry?.sendPolicy,
405         skillsSnapshot: entry?.skillsSnapshot,
406         deliveryContext: deliveryFields.deliveryContext,
407         lastChannel: deliveryFields.lastChannel ?? entry?.lastChannel,
```

```
408         lastTo: deliveryFields.lastTo ?? entry?.lastTo,
409         lastAccountId: deliveryFields.lastAccountId ?? entry?.lastAccountId,
410         modelOverride: entry?.modelOverride,
411         providerOverride: entry?.providerOverride,
412         label: labelValue,
413         spawnedBy: spawnedByValue,
414         spawnDepth: entry?.spawnDepth,
415         channel: entry?.channel ?? request.channel?.trim(),
416         groupId: resolvedGroupId ?? entry?.groupId,
417         groupChannel: resolvedGroupChannel ?? entry?.groupChannel,
418         space: resolvedGroupSpace ?? entry?.space,
419         cliSessionIds: entry?.cliSessionIds,
420         claudeCliSessionId: entry?.claudeCliSessionId,
421       };
422       sessionEntry = mergeSessionEntry(entry, nextEntryPatch);
423       const sendPolicy = resolveSendPolicy({
424         cfg,
425         entry,
426         sessionKey: canonicalKey,
427         channel: entry?.channel,
428         chatType: entry?.chatType,
429       });
430       if (sendPolicy === "deny") {
431         respond(
432           false,
433           undefined,
434           errorShape(ErrorCodes.INVALID_REQUEST, "send blocked by session policy"),
435         );
436         return;
437       }
438       resolvedSessionId = sessionId;
439       const canonicalSessionKey = canonicalKey;
440       resolvedSessionKey = canonicalSessionKey;
441       const agentId = resolveAgentIdFromSessionKey(canonicalSessionKey);
442       const mainSessionKey = resolveAgentMainSessionKey({ cfg, agentId });
443       if (storePath) {
444         const persisted = await updateSessionStore(storePath, (store) => {
445           const target = resolveGatewaySessionStoreTarget({
446             cfg,
447             key: requestedSessionKey,
448             store,
449           });
450           pruneLegacyStoreKeys({
451             store,
452             canonicalKey: target.canonicalKey,
453             candidates: target.storeKeys,
454           });
```

```
455          const merged = mergeSessionEntry(store[canonicalSessionKey], nextEntryPatch);
456          store[canonicalSessionKey] = merged;
457          return merged;
458        });
459        sessionEntry = persisted;
460      }
461      if (canonicalSessionKey === mainSessionKey || canonicalSessionKey === "global") {
462        context.addChatRun(idem, {
463          sessionKey: canonicalSessionKey,
464          clientRunId: idem,
465        });
466        if (requestedBestEffortDeliver === undefined) {
467          bestEffortDeliver = true;
468        }
469      }
470      registerAgentRunContext(idem, { sessionKey: canonicalSessionKey });
471    }
472
473    const runId = idem;
474    const connId = typeof client?.connId === "string" ? client.connId : undefined;
475    const wantsToolEvents = hasGatewayClientCap(
476      client?.connect?.caps,
477      GATEWAY_CLIENT_CAPS.TOOL_EVENTS,
478    );
479    if (connId && wantsToolEvents) {
480      context.registerToolEventRecipient(runId, connId);
481      // Register for any other active runs *in the same session* so
482      // late-joining clients (e.g. page refresh mid-response) receive
483      // in-progress tool events without leaking cross-session data.
484      for (const [activeRunId, active] of context.chatAbortControllers) {
485        if (activeRunId !== runId && active.sessionKey === requestedSessionKey) {
486          context.registerToolEventRecipient(activeRunId, connId);
487        }
488      }
489    }
490
491    const wantsDelivery = request.deliver === true;
492    const explicitTo =
493      typeof request.replyTo === "string" && request.replyTo.trim()
494        ? request.replyTo.trim()
495        : typeof request.to === "string" && request.to.trim()
496          ? request.to.trim()
497          : undefined;
498    const explicitThreadId =
499      typeof request.threadId === "string" && request.threadId.trim()
500        ? request.threadId.trim()
501        : undefined;
```

```
502     const turnSourceChannel =
503       typeof request.channel === "string" && request.channel.trim()
504         ? request.channel.trim()
505         : undefined;
506     const turnSourceTo =
507       typeof request.to === "string" && request.to.trim() ? request.to.trim() : undefined;
508     const turnSourceAccountId =
509       typeof request.accountId === "string" && request.accountId.trim()
510         ? request.accountId.trim()
511         : undefined;
512     const deliveryPlan = resolveAgentDeliveryPlan({
513       sessionEntry,
514       requestedChannel: request.replyChannel ?? request.channel,
515       explicitTo,
516       explicitThreadId,
517       accountId: request.replyAccountId ?? request.accountId,
518       wantsDelivery,
519       turnSourceChannel,
520       turnSourceTo,
521       turnSourceAccountId,
522       turnSourceThreadId: explicitThreadId,
523     });
524
525     let resolvedChannel = deliveryPlan.resolvedChannel;
526     let deliveryTargetMode = deliveryPlan.deliveryTargetMode;
527     let resolvedAccountId = deliveryPlan.resolvedAccountId;
528     let resolvedTo = deliveryPlan.resolvedTo;
529     let effectivePlan = deliveryPlan;
530
531     if (wantsDelivery && resolvedChannel === INTERNAL_MESSAGE_CHANNEL) {
532       const cfgResolved = cfgForAgent ?? cfg;
533       try {
534         const selection = await resolveMessageChannelSelection({ cfg: cfgResolved });
535         resolvedChannel = selection.channel;
536         deliveryTargetMode = deliveryTargetMode ?? "implicit";
537         effectivePlan = {
538           ...deliveryPlan,
539           resolvedChannel,
540           deliveryTargetMode,
541           resolvedAccountId,
542         };
543       } catch (err) {
544         respond(false, undefined, errorShape(ErrorCodes.INVALID_REQUEST, String(err)));
545         return;
546       }
547     }
548
```

```
549     if (!resolvedTo && isDeliverableMessageChannel(resolvedChannel)) {
550       const cfgResolved = cfgForAgent ?? cfg;
551       const fallback = resolveAgentOutboundTarget({
552         cfg: cfgResolved,
553         plan: effectivePlan,
554         targetMode: deliveryTargetMode ?? "implicit",
555         validateExplicitTarget: false,
556       });
557       if (fallback.resolvedTarget?.ok) {
558         resolvedTo = fallback.resolvedTo;
559       }
560     }
561
562     if (wantsDelivery && resolvedChannel === INTERNAL_MESSAGE_CHANNEL) {
563       respond(
564         false,
565         undefined,
566         errorShape(
567           ErrorCodes.INVALID_REQUEST,
568           "delivery channel is required: pass --channel/--reply-channel or use a main
    session with a previous channel",
569         ),
570       );
571       return;
572     }
573
574     const normalizedTurnSource = normalizeMessageChannel(turnSourceChannel);
575     const turnSourceMessageChannel =
576       normalizedTurnSource && isGatewayMessageChannel(normalizedTurnSource)
577         ? normalizedTurnSource
578         : undefined;
579     const originMessageChannel =
580       turnSourceMessageChannel ??
581       (client?.connect && isWebchatConnect(client.connect)
582         ? INTERNAL_MESSAGE_CHANNEL
583         : resolvedChannel);
584
585     const deliver = request.deliver === true && resolvedChannel !== INTERNAL_MESSAGE_CHANNEL
    ;
586
587     const accepted = {
588       runId,
589       status: "accepted" as const,
590       acceptedAt: Date.now(),
591     };
592     // Store an in-flight ack so retries do not spawn a second run.
593     setGatewayDedupeEntry({
```

```
      dedupe: context.dedupe,
      key: `agent:${idem}`,
      entry: {
        ts: Date.now(),
        ok: true,
        payload: accepted,
      },
    });
    respond(true, accepted, undefined, { runId });

    const resolvedThreadId = explicitThreadId ?? deliveryPlan.resolvedThreadId;

    dispatchAgentRunFromGateway({
      ingressOpts: {
        message,
        images,
        to: resolvedTo,
        sessionId: resolvedSessionId,
        sessionKey: resolvedSessionKey,
        thinking: request.thinking,
        deliver,
        deliveryTargetMode,
        channel: resolvedChannel,
        accountId: resolvedAccountId,
        threadId: resolvedThreadId,
        runContext: {
          messageChannel: originMessageChannel,
          accountId: resolvedAccountId,
          groupId: resolvedGroupId,
          groupChannel: resolvedGroupChannel,
          groupSpace: resolvedGroupSpace,
          currentThreadTs: resolvedThreadId != null ? String(resolvedThreadId) : undefined,
        },
        groupId: resolvedGroupId,
        groupChannel: resolvedGroupChannel,
        groupSpace: resolvedGroupSpace,
        spawnedBy: spawnedByValue,
        timeout: request.timeout?.toString(),
        bestEffortDeliver,
        messageChannel: originMessageChannel,
        runId,
        lane: request.lane,
        extraSystemPrompt: request.extraSystemPrompt,
        internalEvents: request.internalEvents,
        inputProvenance,
        // Internal-only: allow workspace override for spawned subagent runs.
        workspaceDir: resolveIngressWorkspaceOverrideForSpawnedRun({
```

```
641          spawnedBy: spawnedByValue,
642          workspaceDir: request.workspaceDir,
643        }),
644        senderIsOwner,
645      },
646      runId,
647      idempotencyKey: idem,
648      respond,
649      context,
650    });
651  },
652  "agent.identity.get": ({ params, respond }) => {
653    if (!validateAgentIdentityParams(params)) {
654      respond(
655        false,
656        undefined,
657        errorShape(
658          ErrorCodes.INVALID_REQUEST,
659          `invalid agent.identity.get params: ${formatValidationErrors(
660            validateAgentIdentityParams.errors,
661          )}`,
662        ),
663      );
664      return;
665    }
666    const p = params;
667    const agentIdRaw = typeof p.agentId === "string" ? p.agentId.trim() : "";
668    const sessionKeyRaw = typeof p.sessionKey === "string" ? p.sessionKey.trim() : "";
669    let agentId = agentIdRaw ? normalizeAgentId(agentIdRaw) : undefined;
670    if (sessionKeyRaw) {
671      if (classifySessionKeyShape(sessionKeyRaw) === "malformed_agent") {
672        respond(
673          false,
674          undefined,
675          errorShape(
676            ErrorCodes.INVALID_REQUEST,
677            `invalid agent.identity.get params: malformed session key "${sessionKeyRaw}"`,
678          ),
679        );
680        return;
681      }
682      const resolved = resolveAgentIdFromSessionKey(sessionKeyRaw);
683      if (agentId && resolved !== agentId) {
684        respond(
685          false,
686          undefined,
687          errorShape(
```

```
688            ErrorCodes.INVALID_REQUEST,
689            `invalid agent.identity.get params: agent "${agentIdRaw}" does not match session
     key agent "${resolved}"`,
690          ),
691        );
692        return;
693      }
694      agentId = resolved;
695    }
696    const cfg = loadConfig();
697    const identity = resolveAssistantIdentity({ cfg, agentId });
698    const avatarValue =
699      resolveAssistantAvatarUrl({
700        avatar: identity.avatar,
701        agentId: identity.agentId,
702        basePath: cfg.gateway?.controlUi?.basePath,
703      }) ?? identity.avatar;
704    respond(true, { ...identity, avatar: avatarValue }, undefined);
705  },
706  "agent.wait": async ({ params, respond, context }) => {
707    if (!validateAgentWaitParams(params)) {
708      respond(
709        false,
710        undefined,
711        errorShape(
712          ErrorCodes.INVALID_REQUEST,
713          `invalid agent.wait params: ${formatValidationErrors(validateAgentWaitParams.
     errors)}`,
714        ),
715      );
716      return;
717    }
718    const p = params;
719    const runId = (p.runId ?? "").trim();
720    const timeoutMs =
721      typeof p.timeoutMs === "number" && Number.isFinite(p.timeoutMs)
722        ? Math.max(0, Math.floor(p.timeoutMs))
723        : 30_000;
724    const hasActiveChatRun = context.chatAbortControllers.has(runId);
725
726    const cachedGatewaySnapshot = readTerminalSnapshotFromGatewayDedupe({
727      dedupe: context.dedupe,
728      runId,
729      ignoreAgentTerminalSnapshot: hasActiveChatRun,
730    });
731    if (cachedGatewaySnapshot) {
732      respond(true, {
```

```
733         runId,
734         status: cachedGatewaySnapshot.status,
735         startedAt: cachedGatewaySnapshot.startedAt,
736         endedAt: cachedGatewaySnapshot.endedAt,
737         error: cachedGatewaySnapshot.error,
738       });
739       return;
740     }
741
742     const lifecycleAbortController = new AbortController();
743     const dedupeAbortController = new AbortController();
744     const lifecyclePromise = waitForAgentJob({
745       runId,
746       timeoutMs,
747       signal: lifecycleAbortController.signal,
748       // When chat.send is active with the same runId, ignore cached lifecycle
749       // snapshots so stale agent results do not preempt the active chat run.
750       ignoreCachedSnapshot: hasActiveChatRun,
751     });
752     const dedupePromise = waitForTerminalGatewayDedupe({
753       dedupe: context.dedupe,
754       runId,
755       timeoutMs,
756       signal: dedupeAbortController.signal,
757       ignoreAgentTerminalSnapshot: hasActiveChatRun,
758     });
759
760     const first = await Promise.race([
761       lifecyclePromise.then((snapshot) => ({ source: "lifecycle" as const, snapshot })),
762       dedupePromise.then((snapshot) => ({ source: "dedupe" as const, snapshot })),
763     ]);
764
765     let snapshot: AgentWaitTerminalSnapshot | Awaited<ReturnType<typeof waitForAgentJob>> =
766       first.snapshot;
767     if (snapshot) {
768       if (first.source === "lifecycle") {
769         dedupeAbortController.abort();
770       } else {
771         lifecycleAbortController.abort();
772       }
773     } else {
774       snapshot = first.source === "lifecycle" ? await dedupePromise : await lifecyclePromise
    ;
775       lifecycleAbortController.abort();
776       dedupeAbortController.abort();
777     }
778
```

```
    if (!snapshot) {
      respond(true, {
        runId,
        status: "timeout",
      });
      return;
    }
    respond(true, {
      runId,
      status: snapshot.status,
      startedAt: snapshot.startedAt,
      endedAt: snapshot.endedAt,
      error: snapshot.error,
    });
  },
};
```

B.8　src/auto-reply/reply/agent-runner.ts

作用说明

真实 reply-agent 主路径，当前已接入 `reply-agent` sidecar 调用。

完整内容

```
import fs from "node:fs";
import { lookupContextTokens } from "../../agents/context.js";
import { DEFAULT_CONTEXT_TOKENS } from "../../agents/defaults.js";
import { resolveModelAuthMode } from "../../agents/model-auth.js";
import { isCliProvider } from "../../agents/model-selection.js";
import { queueEmbeddedPiMessage } from "../../agents/pi-embedded.js";
import { hasNonzeroUsage } from "../../agents/usage.js";
import {
  resolveAgentIdFromSessionKey,
  resolveSessionFilePath,
  resolveSessionFilePathOptions,
  resolveSessionTranscriptPath,
  type SessionEntry,
  updateSessionStore,
  updateSessionStoreEntry,
} from "../../config/sessions.js";
import type { TypingMode } from "../../config/types.js";
import { emitAgentEvent } from "../../infra/agent-events.js";
import { emitDiagnosticEvent, isDiagnosticsEnabled } from "../../infra/diagnostic-events.js
    ";
import { queueExternalMemoryKernelRun } from "../../infra/external-memory-kernel.js";
```

```
import { generateSecureUuid } from "../../infra/secure-random.js";
import { enqueueSystemEvent } from "../../infra/system-events.js";
import { defaultRuntime } from "../../runtime.js";
import { estimateUsageCost, resolveModelCostConfig } from "../../utils/usage-format.js";
import {
  buildFallbackClearedNotice,
  buildFallbackNotice,
  resolveFallbackTransition,
} from "../fallback-state.js";
import type { OriginatingChannelType, TemplateContext } from "../templating.js";
import { resolveResponseUsageMode, type VerboseLevel } from "../thinking.js";
import type { GetReplyOptions, ReplyPayload } from "../types.js";
import { runAgentTurnWithFallback } from "./agent-runner-execution.js";
import {
  createShouldEmitToolOutput,
  createShouldEmitToolResult,
  finalizeWithFollowup,
  isAudioPayload,
  signalTypingIfNeeded,
} from "./agent-runner-helpers.js";
import { runMemoryFlushIfNeeded } from "./agent-runner-memory.js";
import { buildReplyPayloads } from "./agent-runner-payloads.js";
import {
  appendUnscheduledReminderNote,
  hasSessionRelatedCronJobs,
  hasUnbackedReminderCommitment,
} from "./agent-runner-reminder-guard.js";
import { appendUsageLine, formatResponseUsageLine } from "./agent-runner-utils.js";
import { createAudioAsVoiceBuffer, createBlockReplyPipeline } from "./block-reply-pipeline.
    js";
import { resolveEffectiveBlockStreamingConfig } from "./block-streaming.js";
import { createFollowupRunner } from "./followup-runner.js";
import { resolveOriginMessageProvider, resolveOriginMessageTo } from "./origin-routing.js";
import { readPostCompactionContext } from "./post-compaction-context.js";
import { resolveActiveRunQueueAction } from "./queue-policy.js";
import { enqueueFollowupRun, type FollowupRun, type QueueSettings } from "./queue.js";
import { createReplyMediaPathNormalizer } from "./reply-media-paths.js";
import { createReplyToModeFilterForChannel, resolveReplyToMode } from "./reply-threading.js
    ";
import { incrementRunCompactionCount, persistRunSessionUsage } from "./session-run-
    accounting.js";
import { createTypingSignaler } from "./typing-mode.js";
import type { TypingController } from "./typing.js";

const BLOCK_REPLY_SEND_TIMEOUT_MS = 15_000;

export async function runReplyAgent(params: {
```

```
  commandBody: string;
  followupRun: FollowupRun;
  queueKey: string;
  resolvedQueue: QueueSettings;
  shouldSteer: boolean;
  shouldFollowup: boolean;
  isActive: boolean;
  isStreaming: boolean;
  opts?: GetReplyOptions;
  typing: TypingController;
  sessionEntry?: SessionEntry;
  sessionStore?: Record<string, SessionEntry>;
  sessionKey?: string;
  storePath?: string;
  defaultModel: string;
  agentCfgContextTokens?: number;
  resolvedVerboseLevel: VerboseLevel;
  isNewSession: boolean;
  blockStreamingEnabled: boolean;
  blockReplyChunking?: {
    minChars: number;
    maxChars: number;
    breakPreference: "paragraph" | "newline" | "sentence";
    flushOnParagraph?: boolean;
  };
  resolvedBlockStreamingBreak: "text_end" | "message_end";
  sessionCtx: TemplateContext;
  shouldInjectGroupIntro: boolean;
  typingMode: TypingMode;
}): Promise<ReplyPayload | ReplyPayload[] | undefined> {
  const {
    commandBody,
    followupRun,
    queueKey,
    resolvedQueue,
    shouldSteer,
    shouldFollowup,
    isActive,
    isStreaming,
    opts,
    typing,
    sessionEntry,
    sessionStore,
    sessionKey,
    storePath,
    defaultModel,
    agentCfgContextTokens,
```

```
    resolvedVerboseLevel,
    isNewSession,
    blockStreamingEnabled,
    blockReplyChunking,
    resolvedBlockStreamingBreak,
    sessionCtx,
    shouldInjectGroupIntro,
    typingMode,
  } = params;

  let activeSessionEntry = sessionEntry;
  const activeSessionStore = sessionStore;
  let activeIsNewSession = isNewSession;

  const isHeartbeat = opts?.isHeartbeat === true;
  const typingSignals = createTypingSignaler({
    typing,
    mode: typingMode,
    isHeartbeat,
  });

  const shouldEmitToolResult = createShouldEmitToolResult({
    sessionKey,
    storePath,
    resolvedVerboseLevel,
  });
  const shouldEmitToolOutput = createShouldEmitToolOutput({
    sessionKey,
    storePath,
    resolvedVerboseLevel,
  });

  const pendingToolTasks = new Set<Promise<void>>();
  const blockReplyTimeoutMs = opts?.blockReplyTimeoutMs ?? BLOCK_REPLY_SEND_TIMEOUT_MS;

  const replyToChannel = resolveOriginMessageProvider({
    originatingChannel: sessionCtx.OriginatingChannel,
    provider: sessionCtx.Surface ?? sessionCtx.Provider,
  }) as OriginatingChannelType | undefined;
  const replyToMode = resolveReplyToMode(
    followupRun.run.config,
    replyToChannel,
    sessionCtx.AccountId,
    sessionCtx.ChatType,
  );
  const applyReplyToMode = createReplyToModeFilterForChannel(replyToMode, replyToChannel);
  const cfg = followupRun.run.config;
```

```
159  const normalizeReplyMediaPaths = createReplyMediaPathNormalizer({
160    cfg,
161    sessionKey,
162    workspaceDir: followupRun.run.workspaceDir,
163  });
164  const blockReplyCoalescing =
165    blockStreamingEnabled && opts?.onBlockReply
166      ? resolveEffectiveBlockStreamingConfig({
167          cfg,
168          provider: sessionCtx.Provider,
169          accountId: sessionCtx.AccountId,
170          chunking: blockReplyChunking,
171        }).coalescing
172      : undefined;
173  const blockReplyPipeline =
174    blockStreamingEnabled && opts?.onBlockReply
175      ? createBlockReplyPipeline({
176          onBlockReply: opts.onBlockReply,
177          timeoutMs: blockReplyTimeoutMs,
178          coalescing: blockReplyCoalescing,
179          buffer: createAudioAsVoiceBuffer({ isAudioPayload }),
180        })
181      : null;
182  const touchActiveSessionEntry = async () => {
183    if (!activeSessionEntry || !activeSessionStore || !sessionKey) {
184      return;
185    }
186    const updatedAt = Date.now();
187    activeSessionEntry.updatedAt = updatedAt;
188    activeSessionStore[sessionKey] = activeSessionEntry;
189    if (storePath) {
190      await updateSessionStoreEntry({
191        storePath,
192        sessionKey,
193        update: async () => ({ updatedAt }),
194      });
195    }
196  };
197
198  if (shouldSteer && isStreaming) {
199    const steered = queueEmbeddedPiMessage(followupRun.run.sessionId, followupRun.prompt);
200    if (steered && !shouldFollowup) {
201      await touchActiveSessionEntry();
202      typing.cleanup();
203      return undefined;
204    }
205  }
```

```
206
207   const activeRunQueueAction = resolveActiveRunQueueAction({
208     isActive,
209     isHeartbeat,
210     shouldFollowup,
211     queueMode: resolvedQueue.mode,
212   });
213
214   if (activeRunQueueAction === "drop") {
215     typing.cleanup();
216     return undefined;
217   }
218
219   if (activeRunQueueAction === "enqueue-followup") {
220     enqueueFollowupRun(queueKey, followupRun, resolvedQueue);
221     await touchActiveSessionEntry();
222     typing.cleanup();
223     return undefined;
224   }
225
226   await typingSignals.signalRunStart();
227
228   activeSessionEntry = await runMemoryFlushIfNeeded({
229     cfg,
230     followupRun,
231     promptForEstimate: followupRun.prompt,
232     sessionCtx,
233     opts,
234     defaultModel,
235     agentCfgContextTokens,
236     resolvedVerboseLevel,
237     sessionEntry: activeSessionEntry,
238     sessionStore: activeSessionStore,
239     sessionKey,
240     storePath,
241     isHeartbeat,
242   });
243
244   const runFollowupTurn = createFollowupRunner({
245     opts,
246     typing,
247     typingMode,
248     sessionEntry: activeSessionEntry,
249     sessionStore: activeSessionStore,
250     sessionKey,
251     storePath,
252     defaultModel,
```

```
253     agentCfgContextTokens,
254   });
255
256   let responseUsageLine: string | undefined;
257   type SessionResetOptions = {
258     failureLabel: string;
259     buildLogMessage: (nextSessionId: string) => string;
260     cleanupTranscripts?: boolean;
261   };
262   const resetSession = async ({
263     failureLabel,
264     buildLogMessage,
265     cleanupTranscripts,
266   }: SessionResetOptions): Promise<boolean> => {
267     if (!sessionKey || !activeSessionStore || !storePath) {
268       return false;
269     }
270     const prevEntry = activeSessionStore[sessionKey] ?? activeSessionEntry;
271     if (!prevEntry) {
272       return false;
273     }
274     const prevSessionId = cleanupTranscripts ? prevEntry.sessionId : undefined;
275     const nextSessionId = generateSecureUuid();
276     const nextEntry: SessionEntry = {
277       ...prevEntry,
278       sessionId: nextSessionId,
279       updatedAt: Date.now(),
280       systemSent: false,
281       abortedLastRun: false,
282       modelProvider: undefined,
283       model: undefined,
284       contextTokens: undefined,
285       systemPromptReport: undefined,
286       fallbackNoticeSelectedModel: undefined,
287       fallbackNoticeActiveModel: undefined,
288       fallbackNoticeReason: undefined,
289     };
290     const agentId = resolveAgentIdFromSessionKey(sessionKey);
291     const nextSessionFile = resolveSessionTranscriptPath(
292       nextSessionId,
293       agentId,
294       sessionCtx.MessageThreadId,
295     );
296     nextEntry.sessionFile = nextSessionFile;
297     activeSessionStore[sessionKey] = nextEntry;
298     try {
299       await updateSessionStore(storePath, (store) => {
```

```
300         store[sessionKey] = nextEntry;
301       });
302     } catch (err) {
303       defaultRuntime.error(
304         `Failed to persist session reset after ${failureLabel} (${sessionKey}): ${String(err
    )}`,
305       );
306     }
307     followupRun.run.sessionId = nextSessionId;
308     followupRun.run.sessionFile = nextSessionFile;
309     activeSessionEntry = nextEntry;
310     activeIsNewSession = true;
311     defaultRuntime.error(buildLogMessage(nextSessionId));
312     if (cleanupTranscripts && prevSessionId) {
313       const transcriptCandidates = new Set<string>();
314       const resolved = resolveSessionFilePath(
315         prevSessionId,
316         prevEntry,
317         resolveSessionFilePathOptions({ agentId, storePath }),
318       );
319       if (resolved) {
320         transcriptCandidates.add(resolved);
321       }
322       transcriptCandidates.add(resolveSessionTranscriptPath(prevSessionId, agentId));
323       for (const candidate of transcriptCandidates) {
324         try {
325           fs.unlinkSync(candidate);
326         } catch {
327           // Best-effort cleanup.
328         }
329       }
330     }
331     return true;
332   };
333   const resetSessionAfterCompactionFailure = async (reason: string): Promise<boolean> =>
334     resetSession({
335       failureLabel: "compaction failure",
336       buildLogMessage: (nextSessionId) =>
337         `Auto-compaction failed (${reason}). Restarting session ${sessionKey} -> ${
    nextSessionId} and retrying.`,
338     });
339   const resetSessionAfterRoleOrderingConflict = async (reason: string): Promise<boolean> =>
340     resetSession({
341       failureLabel: "role ordering conflict",
342       buildLogMessage: (nextSessionId) =>
343         `Role ordering conflict (${reason}). Restarting session ${sessionKey} -> ${
    nextSessionId}.`,
```

```
      cleanupTranscripts: true,
    });
  try {
    const runStartedAt = Date.now();
    const runOutcome = await runAgentTurnWithFallback({
      commandBody,
      followupRun,
      sessionCtx,
      opts,
      typingSignals,
      blockReplyPipeline,
      blockStreamingEnabled,
      blockReplyChunking,
      resolvedBlockStreamingBreak,
      applyReplyToMode,
      shouldEmitToolResult,
      shouldEmitToolOutput,
      pendingToolTasks,
      resetSessionAfterCompactionFailure,
      resetSessionAfterRoleOrderingConflict,
      isHeartbeat,
      sessionKey,
      getActiveSessionEntry: () => activeSessionEntry,
      activeSessionStore,
      storePath,
      resolvedVerboseLevel,
    });

    if (!isHeartbeat) {
      queueExternalMemoryKernelRun({
        source: "reply-agent",
        status: runOutcome.kind === "success" ? "success" : "final",
        sessionKey,
        agentId: followupRun.run.agentId,
      });
    }

    if (runOutcome.kind === "final") {
      return finalizeWithFollowup(runOutcome.payload, queueKey, runFollowupTurn);
    }

    const {
      runId,
      runResult,
      fallbackProvider,
      fallbackModel,
      fallbackAttempts,
```

```
      directlySentBlockKeys,
    } = runOutcome;
    let { didLogHeartbeatStrip, autoCompactionCompleted } = runOutcome;

    if (
      shouldInjectGroupIntro &&
      activeSessionEntry &&
      activeSessionStore &&
      sessionKey &&
      activeSessionEntry.groupActivationNeedsSystemIntro
    ) {
      const updatedAt = Date.now();
      activeSessionEntry.groupActivationNeedsSystemIntro = false;
      activeSessionEntry.updatedAt = updatedAt;
      activeSessionStore[sessionKey] = activeSessionEntry;
      if (storePath) {
        await updateSessionStoreEntry({
          storePath,
          sessionKey,
          update: async () => ({
            groupActivationNeedsSystemIntro: false,
            updatedAt,
          }),
        });
      }
    }

    const payloadArray = runResult.payloads ?? [];

    if (blockReplyPipeline) {
      await blockReplyPipeline.flush({ force: true });
      blockReplyPipeline.stop();
    }
    if (pendingToolTasks.size > 0) {
      await Promise.allSettled(pendingToolTasks);
    }

    const usage = runResult.meta?.agentMeta?.usage;
    const promptTokens = runResult.meta?.agentMeta?.promptTokens;
    const modelUsed = runResult.meta?.agentMeta?.model ?? fallbackModel ?? defaultModel;
    const providerUsed =
      runResult.meta?.agentMeta?.provider ?? fallbackProvider ?? followupRun.run.provider;
    const verboseEnabled = resolvedVerboseLevel !== "off";
    const selectedProvider = followupRun.run.provider;
    const selectedModel = followupRun.run.model;
    const fallbackStateEntry =
      activeSessionEntry ?? (sessionKey ? activeSessionStore?.[sessionKey] : undefined);
```

```
438     const fallbackTransition = resolveFallbackTransition({
439       selectedProvider,
440       selectedModel,
441       activeProvider: providerUsed,
442       activeModel: modelUsed,
443       attempts: fallbackAttempts,
444       state: fallbackStateEntry,
445     });
446     if (fallbackTransition.stateChanged) {
447       if (fallbackStateEntry) {
448         fallbackStateEntry.fallbackNoticeSelectedModel = fallbackTransition.nextState.
    selectedModel;
449         fallbackStateEntry.fallbackNoticeActiveModel = fallbackTransition.nextState.
    activeModel;
450         fallbackStateEntry.fallbackNoticeReason = fallbackTransition.nextState.reason;
451         fallbackStateEntry.updatedAt = Date.now();
452         activeSessionEntry = fallbackStateEntry;
453       }
454       if (sessionKey && fallbackStateEntry && activeSessionStore) {
455         activeSessionStore[sessionKey] = fallbackStateEntry;
456       }
457       if (sessionKey && storePath) {
458         await updateSessionStoreEntry({
459           storePath,
460           sessionKey,
461           update: async () => ({
462             fallbackNoticeSelectedModel: fallbackTransition.nextState.selectedModel,
463             fallbackNoticeActiveModel: fallbackTransition.nextState.activeModel,
464             fallbackNoticeReason: fallbackTransition.nextState.reason,
465           }),
466         });
467       }
468     }
469     const cliSessionId = isCliProvider(providerUsed, cfg)
470       ? runResult.meta?.agentMeta?.sessionId?.trim()
471       : undefined;
472     const contextTokensUsed =
473       agentCfgContextTokens ??
474       lookupContextTokens(modelUsed) ??
475       activeSessionEntry?.contextTokens ??
476       DEFAULT_CONTEXT_TOKENS;
477
478     await persistRunSessionUsage({
479       storePath,
480       sessionKey,
481       usage,
482       lastCallUsage: runResult.meta?.agentMeta?.lastCallUsage,
```

```
483       promptTokens,
484       modelUsed,
485       providerUsed,
486       contextTokensUsed,
487       systemPromptReport: runResult.meta?.systemPromptReport,
488       cliSessionId,
489     });
490 
491     // Drain any late tool/block deliveries before deciding there's "nothing to send".
492     // Otherwise, a late typing trigger (e.g. from a tool callback) can outlive the run and
493     // keep the typing indicator stuck.
494     if (payloadArray.length === 0) {
495       return finalizeWithFollowup(undefined, queueKey, runFollowupTurn);
496     }
497 
498     const payloadResult = await buildReplyPayloads({
499       payloads: payloadArray,
500       isHeartbeat,
501       didLogHeartbeatStrip,
502       blockStreamingEnabled,
503       blockReplyPipeline,
504       directlySentBlockKeys,
505       replyToMode,
506       replyToChannel,
507       currentMessageId: sessionCtx.MessageSidFull ?? sessionCtx.MessageSid,
508       messageProvider: followupRun.run.messageProvider,
509       messagingToolSentTexts: runResult.messagingToolSentTexts,
510       messagingToolSentMediaUrls: runResult.messagingToolSentMediaUrls,
511       messagingToolSentTargets: runResult.messagingToolSentTargets,
512       originatingChannel: sessionCtx.OriginatingChannel,
513       originatingTo: resolveOriginMessageTo({
514         originatingTo: sessionCtx.OriginatingTo,
515         to: sessionCtx.To,
516       }),
517       accountId: sessionCtx.AccountId,
518       normalizeMediaPaths: normalizeReplyMediaPaths,
519     });
520     const { replyPayloads } = payloadResult;
521     didLogHeartbeatStrip = payloadResult.didLogHeartbeatStrip;
522 
523     if (replyPayloads.length === 0) {
524       return finalizeWithFollowup(undefined, queueKey, runFollowupTurn);
525     }
526 
527     const successfulCronAdds = runResult.successfulCronAdds ?? 0;
528     const hasReminderCommitment = replyPayloads.some(
529       (payload) =>
```

```
530         !payload.isError &&
531         typeof payload.text === "string" &&
532         hasUnbackedReminderCommitment(payload.text),
533     );
534     // Suppress the guard note when an existing cron job (created in a prior
535     // turn) already covers the commitment — avoids false positives (#32228).
536     const coveredByExistingCron =
537       hasReminderCommitment && successfulCronAdds === 0
538         ? await hasSessionRelatedCronJobs({
539             cronStorePath: cfg.cron?.store,
540             sessionKey,
541           })
542         : false;
543     const guardedReplyPayloads =
544       hasReminderCommitment && successfulCronAdds === 0 && !coveredByExistingCron
545         ? appendUnscheduledReminderNote(replyPayloads)
546         : replyPayloads;
547
548     await signalTypingIfNeeded(guardedReplyPayloads, typingSignals);
549
550     if (isDiagnosticsEnabled(cfg) && hasNonzeroUsage(usage)) {
551       const input = usage.input ?? 0;
552       const output = usage.output ?? 0;
553       const cacheRead = usage.cacheRead ?? 0;
554       const cacheWrite = usage.cacheWrite ?? 0;
555       const promptTokens = input + cacheRead + cacheWrite;
556       const totalTokens = usage.total ?? promptTokens + output;
557       const costConfig = resolveModelCostConfig({
558         provider: providerUsed,
559         model: modelUsed,
560         config: cfg,
561       });
562       const costUsd = estimateUsageCost({ usage, cost: costConfig });
563       emitDiagnosticEvent({
564         type: "model.usage",
565         sessionKey,
566         sessionId: followupRun.run.sessionId,
567         channel: replyToChannel,
568         provider: providerUsed,
569         model: modelUsed,
570         usage: {
571           input,
572           output,
573           cacheRead,
574           cacheWrite,
575           promptTokens,
576           total: totalTokens,
```

```
      },
      lastCallUsage: runResult.meta?.agentMeta?.lastCallUsage,
      context: {
        limit: contextTokensUsed,
        used: totalTokens,
      },
      costUsd,
      durationMs: Date.now() - runStartedAt,
    });
  }

  const responseUsageRaw =
    activeSessionEntry?.responseUsage ??
    (sessionKey ? activeSessionStore?.[sessionKey]?.responseUsage : undefined);
  const responseUsageMode = resolveResponseUsageMode(responseUsageRaw);
  if (responseUsageMode !== "off" && hasNonzeroUsage(usage)) {
    const authMode = resolveModelAuthMode(providerUsed, cfg);
    const showCost = authMode === "api-key";
    const costConfig = showCost
      ? resolveModelCostConfig({
          provider: providerUsed,
          model: modelUsed,
          config: cfg,
        })
      : undefined;
    let formatted = formatResponseUsageLine({
      usage,
      showCost,
      costConfig,
    });
    if (formatted && responseUsageMode === "full" && sessionKey) {
      formatted = `${formatted} • session \`${sessionKey}\``;
    }
    if (formatted) {
      responseUsageLine = formatted;
    }
  }

  // If verbose is enabled, prepend operational run notices.
  let finalPayloads = guardedReplyPayloads;
  const verboseNotices: ReplyPayload[] = [];

  if (verboseEnabled && activeIsNewSession) {
    verboseNotices.push({ text: `  New session: ${followupRun.run.sessionId}` });
  }

  if (fallbackTransition.fallbackTransitioned) {
```

```
624       emitAgentEvent({
625         runId,
626         sessionKey,
627         stream: "lifecycle",
628         data: {
629           phase: "fallback",
630           selectedProvider,
631           selectedModel,
632           activeProvider: providerUsed,
633           activeModel: modelUsed,
634           reasonSummary: fallbackTransition.reasonSummary,
635           attemptSummaries: fallbackTransition.attemptSummaries,
636           attempts: fallbackAttempts,
637         },
638       });
639       if (verboseEnabled) {
640         const fallbackNotice = buildFallbackNotice({
641           selectedProvider,
642           selectedModel,
643           activeProvider: providerUsed,
644           activeModel: modelUsed,
645           attempts: fallbackAttempts,
646         });
647         if (fallbackNotice) {
648           verboseNotices.push({ text: fallbackNotice });
649         }
650       }
651     }
652     if (fallbackTransition.fallbackCleared) {
653       emitAgentEvent({
654         runId,
655         sessionKey,
656         stream: "lifecycle",
657         data: {
658           phase: "fallback_cleared",
659           selectedProvider,
660           selectedModel,
661           activeProvider: providerUsed,
662           activeModel: modelUsed,
663           previousActiveModel: fallbackTransition.previousState.activeModel,
664         },
665       });
666       if (verboseEnabled) {
667         verboseNotices.push({
668           text: buildFallbackClearedNotice({
669             selectedProvider,
670             selectedModel,
```

```
671             previousActiveModel: fallbackTransition.previousState.activeModel,
672           }),
673         });
674       }
675     }
676
677     if (autoCompactionCompleted) {
678       const count = await incrementRunCompactionCount({
679         sessionEntry: activeSessionEntry,
680         sessionStore: activeSessionStore,
681         sessionKey,
682         storePath,
683         lastCallUsage: runResult.meta?.agentMeta?.lastCallUsage,
684         contextTokensUsed,
685       });
686
687       // Inject post-compaction workspace context for the next agent turn
688       if (sessionKey) {
689         const workspaceDir = process.cwd();
690         readPostCompactionContext(workspaceDir, cfg)
691           .then((contextContent) => {
692             if (contextContent) {
693               enqueueSystemEvent(contextContent, { sessionKey });
694             }
695           })
696           .catch(() => {
697             // Silent failure — post-compaction context is best-effort
698           });
699       }
700
701       if (verboseEnabled) {
702         const suffix = typeof count === "number" ? ` (count ${count})` : "";
703         verboseNotices.push({ text: `  Auto-compaction complete${suffix}.` });
704       }
705     }
706     if (verboseNotices.length > 0) {
707       finalPayloads = [...verboseNotices, ...finalPayloads];
708     }
709     if (responseUsageLine) {
710       finalPayloads = appendUsageLine(finalPayloads, responseUsageLine);
711     }
712
713     return finalizeWithFollowup(
714       finalPayloads.length === 1 ? finalPayloads[0] : finalPayloads,
715       queueKey,
716       runFollowupTurn,
717     );
```

```
  } catch (error) {
    // Keep the followup queue moving even when an unexpected exception escapes
    // the run path; the caller still receives the original error.
    finalizeWithFollowup(undefined, queueKey, runFollowupTurn);
    throw error;
  } finally {
    blockReplyPipeline?.stop();
    typing.markRunComplete();
    // Safety net: the dispatcher's onIdle callback normally fires
    // markDispatchIdle(), but if the dispatcher exits early, errors,
    // or the reply path doesn't go through it cleanly, the second
    // signal never fires and the typing keepalive loop runs forever.
    // Calling this twice is harmless — cleanup() is guarded by the
    // `active` flag.  Same pattern as the followup runner fix (#26881).
    typing.markDispatchIdle();
  }
}
```

最后说明

这份手册的设计目的不是“把所有文件机械堆一起”，而是让后来维修的人能回答四个问题：

1. 系统现在到底怎么跑
2. 哪些文件是真正关键的
3. 真实消息为什么会更新 memory
4. 出故障时应该先看哪里、先修哪里

因此，这份文档同时保留了：

- 结构解释
- 运行链解释
- 维修流程
- 完整源码与状态快照

如果后续继续演化系统，建议每次大改后重新导出一版手册，以避免“代码已经变了，手册还是旧的”这种二次故障。